

Universitatea Națională de Științe și Tehnologie POLITEHNICA București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației

**Detecția defectelor de fabricație într-un circuit digital,
cu ajutorul vectorilor de test generați automat**

Lucrare de licență

**Prezentată ca cerință parțială pentru obținerea
titlului de *Inginer*
în domeniul *Calculatoare și Tehnologia Informației*
programul de studii *Ingineria Informației***

Conducător științific

**Ș. L. Dr. Ing. Diana-Elena Bucur Brînaru
Ing. Liviu Gugu**

Absolvent

Stancovici Marian

Anul 2024

Universitatea "Politehnica" din București
Facultatea de Electronică, Telecomunicații și Tehnologia Informației
Program de studiu **INF**

Anexa 1

TEMA PROIECTULUI DE DIPLOMĂ
a studentului **STANCOVICI L.I. Marian, 444A**

1. Titlul temei: Detecția defectelor de fabricație într-un circuit digital cu ajutorul vectorilor de test generați automat

2. Descrierea temei și a contribuției personale a studentului (în afara părții de documentare):

Scopul lucrării de față este implementarea unui set de teste pentru un circuit digital, pentru care se generează vectori de test, cu scopul de a depista defectele de fabricație (DFT). Circuitul digital descris în limbaj Verilog constă într-un SoC bazat pe tehnologie RISC-V în care sunt implementate module dedicate de Timer, UART, SPI și periferice GPIO. Se va crea modulul de top care va instanția SoC-ul și, dacă este necesar, logica de test care include folosirea EDT-ul (Embedded Deterministic Test). Se verifică performanțele sistemului (timp de testare) prin diferite variante de test folosind un singur chain și cu mai multe chain-uri cu ajutorul tehnologiei EDT. Vor fi studiate diferite structuri de test și rezultatele obținute (Test Coverage, Test Time). Pentru diferitele structuri de test se face sinteza asupra modulului pentru a ajunge la un Netlist pentru care se generează și se simulează patternuri de test folosind Tessent - Mentor Graphics, program uzual folosit în industria semiconductoarelor.

3. Discipline necesare pt. proiect:

Microcontrolere, Circuite Integrate Digitale

4. Data înregistrării temei: 2024-01-03 17:50:27

Conducător(i) lucrare,
Ș.L.Dr.Ing. Diana BRÎNARU

Student,
STANCOVICI L.I. Marian

Ing. GUGU Liviu, Infineon Technologies România

Director departament,
Ș.L. dr. ing Bogdan FLOREA

Decan,
Prof. dr. ing. Mihnea UDREA

Cod Validare: **1fe20feb92**

Copyright © 2024 , *Stancovici Marian / Infineon Technologies România*

Toate drepturile rezervate

Autorul acordă UPB dreptul de a reproduce și de a distribui public copii pe hîrtie sau electronice ale acestei lucrări, în formă integrală sau parțială.

Declarație de onestitate academică

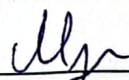
Prin prezenta declar că lucrarea cu titlul "*Detecția defectelor de fabricație într-un circuit digital, cu ajutorul vectorilor de test generați automat*", prezentată în cadrul Facultății de Electronică, Telecomunicații și Tehnologia Informației a Universității Naționale de Științe și Tehnologie "Politehnica" din București ca cerință parțială pentru obținerea titlului de *Inginer în domeniul Calculatoare și Tehnologia Informației*, programul de studii *Ingineria Informației* este scrisă de mine și nu a mai fost prezentată niciodată la o facultate sau instituție de învățământ superior din țară sau străinătate.

Declar că toate sursele utilizate, inclusiv cele de pe Internet, sunt indicate în lucrare, ca referințe bibliografice. Fragmentele de text din alte surse, reproduse exact, chiar și în traducere proprie din altă limbă, sunt scrise între ghilimele și fac referință la sursă. Reformularea în cuvinte proprii a textelor scrise de către alți autori face referință la sursă. Înțeleg că plagiatul constituie infracțiune și se sancționează conform legilor în vigoare.

Declar că toate rezultatele simulărilor, experimentelor și măsurărilor pe care le prezint ca fiind făcute de mine, precum și metodele prin care au fost obținute, sunt reale și provin din respectivele simulări, experimente și măsurători. Înțeleg că falsificarea datelor și rezultatelor constituie fraudă și se sancționează conform regulamentelor în vigoare.

București, 27.06.2024

Absolvent Marian STANCOVICI


(semnătura în original)

Cuprins

Lista figurilor	v
Lista tabelelor	vii
Lista acronimelor	ix
1. Fundamente teoretice	5
1.1. Circuite digitale și defecte de fabricație	5
1.2. Testarea și testabilitatea circuitelor digitale	6
1.2.1. Testare VS Verificare	6
1.3. Modelarea defectelor	8
1.3.1. Defecte funcționale	11
1.3.1.1. Defecte de tranziție (delay faults)	11
1.3.2. Defecte structurale la nivelul porților	13
1.3.2.1. Defecte de tip stuck-at	13
1.4. Simularea defectelor (faulturilor)	14
1.5. Generarea automată a vectorilor de test (ATPG)	15
1.5.1. Generalități	15
1.5.2. Mod de funcționare	15
1.5.3. Metode ATPG	17
1.5.3.1. Metoda aleatoare	17
1.5.3.2. Metoda deterministă	17
1.5.3.3. Metoda de generare externă	18
1.5.4. Avantaje și dezavantaje ATPG	18
1.6. Tehnici de proiectare pentru testabilitate (DFT)	18
1.6.1. Tipuri de tehnici DFT	19
1.6.1.1. Tehnici ad-hoc	19
1.6.1.2. Tehnici structurate	19
1.6.2. Testarea circuitelor combinaționale și secvențiale	19
1.6.2.1. Circuite combinaționale	20
1.6.2.2. Circuite secvențiale	20
1.6.3. Observabilitate și controlabilitate	21
1.6.3.1. Observabilitate	22
1.6.3.2. Controlabilitate	22
1.7. Tehnologia SCAN	23
1.7.1. Transformarea flip-flopilor normali în flip-flopi de Scan [1]	23
1.7.2. Metodologia de proiectare Scan	24
1.8. Embedded Deterministic Test	25
1.8.1. Avantaje și caracteristici EDT	25
1.8.2. Structura EDT	26
1.8.2.1. Decompresor	27

1.8.2.2.	Compactor	28
1.8.2.3.	Controller	29
1.8.3.	Mod de funcționare EDT	30
1.8.4.	Tessent TestKompress	30
1.8.4.1.	Avantaje și caracteristici TestKompress	31
1.8.4.2.	ATPG vs EDT	31
2.	Descrierea sistemului [2]	33
2.1.	Arhitectura SoC bazată pe tehnologia RISC-V	33
2.2.	Module dedicate	34
2.2.1.	Timer	34
2.2.2.	UART	34
2.2.3.	SPI	34
2.2.4.	GPIO	34
2.2.5.	AXI RAM	34
2.3.	Modulul de top și instanțierea SoC-ului	35
2.4.	Logica de test și integrarea EDT	35
3.	Metodologie și implementare	37
3.1.	Generarea EDT-ului și crearea netlist-ului pentru modulul de top	38
3.1.1.	Configurări necesare	38
3.1.2.	Etape pentru generare	40
3.2.	Sinteza circuitului	40
3.2.1.	Cazul fără EDT (sinteză normală)	41
3.2.1.1.	Insertia logicii de SCAN	44
3.2.2.	Cazul cu EDT (mai multe lanțuri de scan)	46
3.3.	Generarea vectorilor de test pentru defecte de tip STUCK-AT și TRANSITION	48
3.3.1.	Cazul fără EDT (ATPG normal)	49
3.3.1.1.	Generarea defectelor de tip Stuck-at	49
3.3.1.2.	Generarea defectelor de tip Transition	56
3.3.2.	Cazul cu EDT (variante cu 30, 40, 50, 60 de lanțuri de SCAN)	57
3.4.	Simularea vectorilor de test cu ajutorul testbench-ului	60
3.4.1.	Pași pentru simularea patternurilor	60
3.4.2.	Vizualizarea vectorilor de test pe forma de undă	61
3.5.	Etapa de diagnoză	63
4.	Rezultate și discuții (ATPG)	67
4.1.	Criterii de performanță ale sistemului pentru diferite variante de test (non-EDT vs EDT-30/40/50/60 chains)	67
4.1.1.	Patternuri de test	67
4.1.2.	Ciclii de shiftare	68
4.1.3.	EDT Offset	68
4.1.4.	Ciclii totali de shiftare (per pattern)	69
4.1.5.	Ciclii totali de shiftare	69
4.1.6.	Timp de testare	69
4.1.7.	Test Coverage	70
4.1.8.	Arie	70

5. Concluzii	71
5.1. Direcții viitoare de cercetare și dezvoltare	71
5.2. Contribuții principale și concluzii	72
Bibliografie	75
Anexa A. Cod	77

Lista figurilor

1.1. Mux 2x1 în care intrarea de selecție prezintă defect	8
1.2. Reprezentare cu porți logice a unui mux 2x1 defect	10
1.3. Vizualizarea sursei problemei pentru circuit, tranzistorul defect	10
1.4. Exemplu de propagare întârziată a semnalului în circuit [3]	11
1.5. Exemplificare a semnalelor pentru a evidenția tranziția întârziată	12
1.6. Exemplificare a defectelor de tip single stuck-at pe o poartă OR	13
1.7. Exemplu de circuit cu 3 intrări, o ieșire și un defect stuck-at-1	16
1.8. Tabelul de adevăr care atestă prezența defectului din figura precedentă	16
1.9. Exemplu de circuit secvențial pe care se aplică tehnici DFT	21
1.10. Exemplificare a logicii secvențiale și combinaționale folosind 4 bistabile	21
1.11. Metode pentru îmbunătățirea observabilității	23
1.12. Modificarea unui flip-flop normal în flip-flop de Scan	23
1.13. Exemplu de lanț de Scan	24
1.14. Exemplu de circuit testabil după inserarea logicii de SCAN	25
1.15. Structura EDT-ului conectat la un nucleu RISC-V	26
1.16. Structura unui decompresor	27
1.17. Un exemplu simplu de Ring LFSR care calculează o funcție polinomială	28
1.18. Structura unui compactor	28
1.19. Controller+decompresor+compactor	29
1.20. Structura unui controller EDT	29
1.21. Comparatie între ATPG normal și ATPG integrat cu EDT	31
2.1. Plăcuța FPGA Digilent Artix-7 Arty	33
3.1. Procesul întreg de testare cu SCAN, ATPG și EDT	37
3.2. Detalii despre fișierul <code>skeleton_input_file</code> din cadrul EDT-ului	39
3.3. Legătura dintre fișierele de intrare și ieșire pentru programul Synopsys Design Compiler	41
3.4. Diagrama pentru pașii de inserție a logicii scan în cadrul sintezei	44
3.5. Raportul privind lungimea, intrarea, ieșirea și elementele lanțului de scan inserat	46
3.6. Modificări în cod pentru inserția Scan cu EDT	46
3.7. Vizualizare a câtorva din cele 70 de lanțuri de scan de dimensiune 80 sau 79	47
3.8. Descrierea procesului de ATPG	48
3.9. Ierarhizarea și interapelarea fișierelor pentru ATPG	49
3.10. Reprezentare a configurării semnalului de ceas în timeplate	52
3.11. Reprezentarea ciclurilor datelor în procedura <i>capt_mul_3pat_1clk</i>	54
3.12. Generare de vectori de test cu procedura <i>capt_basic_clk</i> în timp real	56
3.13. Reprezentare prin semnale a ciclurilor din procedura <i>capture_mul_1slow_2fast</i>	57
3.14. Ierarhizarea și interapelarea fișierelor pentru cazul ATPG folosind EDT	58
3.15. Lungimile, intrările și ieșirile celor 30 de lanțuri de scan, cazul EDT	59
3.16. Logul pentru simularea vectorilor de test în cazul cu 30 de lanțuri de scan	61
3.17. Vizualizarea formelor de undă pentru simularea patternurilor pentru 30 de lanțuri	62

3.18. Exemple de celule conectate în scan chain (Schematic)	62
3.19. Privire mai detaliată asupra semnalelor de scan în cadrul unui pattern	63
3.20. Exemplu de simulare pentru a evidenția deplasarea datelor în lanțul de scan . .	63
3.21. Diferențe între netlist normal și cel defect pentru diagnoză	64
3.22. Logul cu diferențele dintre ieșirile reale și cele așteptate pe netlistul defect . . .	64
3.23. Fișierul final.txt după prelucrare	65
3.24. Rezultatul procesului de diagnoză	66
4.1. Tabelul cu metrici de evaluare pentru generarea patternurilor de test	67

Lista tabelelor

1.1. Tabel de adevăr pentru un mux 2x1 funcționând corect	9
1.2. Tabel de adevăr pentru un mux 2x1 funcționând incorect	9

Lista acronimelor

DFT = Design For Testing
ATPG = Automatic Test Pattern Generation
SoC = System on Chip
RISC = Reduced Instruction Set Computer
EDT = Embedded Deterministic Test
UART = Universal Asynchronous Receiver / Transmitter
SPI = Serial Peripheral Interface
GPIO = General Purpose Input-Output
BIST = Built-In-Self-Test
VHDL = VHSIC Hardware Description Language
VHSIC = Very-High-Speed Integrated Circuit
I2C = Inter-Integrated Circuit
RTL = Register Transfer Level
SA0 = Stuck-at-0
SA1 = Stuck-at-1
JTAG = Joint Test Action Group
FSM = Finite State Machine
FF = Flip Flop
DRC = Design Rule Check
SI = Scan Input
SE = Scan Enable
ATE = Automatic Test Equipment
LSFR = Linear Feedback Shift Register
FPGA = Field Programmable Gate Array
ASIC = Application-Specific Integrated Circuit
FIFO = First-In-First-Out
RAM = Random Access Memory
AXI = Advanced eXtensible Interface
AMBA = Advanced Microcontroller Bus Architecture
HDL = Hardware Description Language
TCL = Tool Command Language
DC = Design Compiler
NCP = Named Capture Procedure

Introducere

Contextul și motivația lucrării

În aceste vremuri ale tehnologiei din ce în ce mai avansate, dispozitivele electrice au ajuns să aibă un rol esențial în viața noastră cotidiană, de la rețele sociale până la aparatura pe care o avem prin casă. Acestea trebuie însă să funcționeze exact așa cum ne dorim, în condiții plăcute sau mai puțin plăcute, iar acest lucru se realizează printr-un proces riguros de testare prin diverse scenarii de utilizare pentru a asigura calitatea și fiabilitatea produsului.

Fiecare din aceste produse electronice conține în interiorul lor un chip/procesor, ”inima” acestuia, care îi conferă modul și toate scenariile de funcționare în mod detaliat, ceea ce necesită mai multe procese speciale de testare și testabilitate.

Scopul lucrării de față este de a explora unul din aceste procese prin tehnici DFT și de a dezvolta și implementa un set de teste automate (ATPG) pentru un modul la alegere, și anume un SoC bazat pe tehnologie RISC-V care prezintă diferite module integrate, pentru a ne familiariza cu conceptele și fluxul de testare. dar și pentru a evalua performanțele diferitelor strategii de testare utilizând tehnologia SCAN și EDT (Embedded Deterministic Test).

Obiectivele lucrării

Lucrarea de față urmărește atingerea următoarelor obiective:

- Crearea unui modul de test pentru SoC-ul RISC-V: Acesta va include modulele dedicate de Timer, UART, SPI și periferice GPIO, precum și logica necesară pentru testarea automată, cât și modulul aferent de EDT în cazul mării performanțelor sistemului.
- Sinteza circuitului pentru a introduce logica DFT (Flip Flopi de SCAN), testa codul de posibile erori de sintaxă și pentru a importa librăriile porților logice aferente structurii noastre pentru utilizare ulterioară.
- Generarea vectorilor de test: Se vor genera automat vectori de test pentru a identifica defectele de fabricație în circuitul digital, defecte de tip STUCKAT și TRANSITION.
- Evaluarea performanțelor sistemului de testare: Se va analiza timpul de testare și test-coverage-ul prin diverse structuri de test, utilizând atât un singur lanț de scan, cât și mai multe lanțuri de scan (scan chain-uri), în contextul tehnologiei EDT.
- Utilizarea Tessent - Mentor Graphics: Se va folosi acest program pentru generarea și simularea vectorilor (patternurilor) de test, evaluându-se astfel eficiența și performanțele sistemului de testare și vizualizând semnalele de interes cu ajutorul testbench-ului.
- Inserarea intenționată a defectelor în circuit (netlist) pentru a putea efectua o etapă de diagnoză în care se efectuează detectarea precisă a defectului în circuit exact la poarta logică aferentă.

Structura lucrării

Lucrarea de față este structurată în mai multe capitole, fiecare din ele abordând un subiect esențial din proiect:

- Capitolul 1: Introducere

În acest capitol se prezintă contextul și motivația lucrării, cât și cum se propune să se redacteze și să se structureze, menționând aplicabilitatea acestora în lumea reală.

- Capitolul 2: Fundamente teoretice

Acest capitol abordează în principal aspectele fundamentale referitoare la partea teoretică a acestei lucrări pentru o mai bună înțelegere a implementării. Se explică în detaliu concepte legate de circuite digitale, defecte de fabricație și tehnici de testare specializate pe domeniul DFT (SCAN, ATPG, EDT).

- Capitolul 3: Descrierea sistemului

În acest capitol se discută despre sistemul cu care se lucrează. Dat fiind faptul că s-a pornit de la un modul scris în Verilog (un SoC RISC-V cu diferite periferice), chiar dacă structura sa pe module nu este atât de relevantă pentru această lucrare, se discută sumar despre aceasta pentru a avea un fundal de la care se pornește.

- Capitolul 4: Metodologie și implementare

În capitolul 4 și cel mai important, se vorbește despre modul de implementare abordat în această lucrare. Se pornește de la modulul Verilog discutat și explicat în capitolul precedent.

Se realizează sinteza acestuia, unde se introduce logica DFT în circuitul pur digital pre-silicon (lanțuri de SCAN) și se importă librăriile necesare fiecărei porți logice.

Apoi intervine etapa generării vectorilor de test. În această etapă se introduc constrângeri în circuit prin fișiere .dofile pentru defectele pe care dorim să le testăm (în acest caz defecte de tip STUCKAT și TRANSITION) specifice tool-ului ATPG, se definesc proceduri, scan-chainuri, semnale, toate acestea pentru cazuri diferite: pentru un singur scan chain (ATPG normal) sau mai multe scan-chainuri, folosind de tehnologia EDT (Embedded Deterministic Test) cu 30,40,50, respectiv 60 de chainuri.

În etapele următoare se realizează simularea vectorilor de test generați anterior, pentru a ajunge la un testbench pentru a vizualiza datele de interes și a vedea dacă nu avem erori.

Ultima etapă este cea de diagnoză, în care se inserează un defect intenționat în netlist pentru a vedea dacă tool-ul îl detectează cu exactitate pentru a putea fi depanată problema în circuit.

- Capitolul 5: Rezultate și discuții

Acest capitol este o continuare și o analiză a capitolului anterior, în care s-au ales mai multe metrice de performanță (test time, test coverage, arie) pentru a putea observa și evidenția diferențele majore dintre cazul non-EDT vs EDT, dar și diferențele minore dintre cazurile în care s-a folosit EDT-ul (30 vs 40 vs 50 vs 60 scan chainuri).

- Capitolul 6: Concluzii

Reflectând la capitolele precedente, acesta sumarizează concluziile referitoare la modul de implementare, sugestii ale acestei lucrări, dar abordează și posibile direcții viitoare de cercetare și dezvoltare în acest domeniu.

Această structură prezentată mai sus asigură o abordare sistematică a subiectului, permițând o înțelegere clară a metodelor de detectare a defectelor de fabricație în circuitele digitale.

Capitolul 1

Fundamente teoretice

1.1 Circuite digitale și defecte de fabricație

În zilele noastre, tehnologia a ajuns din ce în ce mai avansată și mai întâlnită în viața de zi cu zi, prin urmare acest lucru conducând la înmulțirea exponențială și diversificarea circuitelor digitale integrate. Acestea reprezintă inima tehnologiei moderne, stăpânind domenii precum automatizări, comunicații, electrocasnice etc.

Circuitele digitale sunt construite, la baza lor, din componente electronice (tranzistori, diode, condensatori) care și acestea sunt constituite la rândul lor din porți logice (AND, OR, NAND etc) și care lucrează împreună pentru a transmite și manipula date binare (biți de 1 și 0) reprezentând informație digitală. Cu toate acestea, în procesul complex de fabricație pot apărea diverse defecte la nivelul circuitului care pot afecta semnificativ performanța și fiabilitatea acestuia.

Pentru a avea o vedere mai clară, defectele în circuitele digitale/chip-uri se pot clasifica în mai multe categorii: [4] [5]:

1. După natura defectului:

- **Defecte permanente:** Aceste tipuri de defecte sunt prezente permanent, chiar de la fabricarea chip-ului. Ele pot avea ca și cauză:
 - **Defecte de fabricație:** Erori care pot apărea în procesul de fabricație, precum contacte nefavorabile, fire întrerupte, componente defecte, tranzistoare parazitice etc.
 - **Defecte de proiectare:** Erori în proiectarea chip-ului, cum ar fi conexiuni incorecte, calculul greșit al rezistențelor/capacităților etc.
- **Defecte tranzitorii:** Aceste defecte apar în mod temporar și pot avea ca și cauze:
 - **Zgomot electric:** Interferențe care afectează funcționarea circuitului.
 - **Fluctuații de tensiune:** Schimbări bruște de tensiune care afectează funcționarea particularizată a componentelor.
 - **Radiații:** Deși mai rare, acestea pot afecta circuitele digitale, provocând erori temporare/tranzitorii.

2. După localizarea defectului:

- **Defecte la nivel de componentă:** Acestea se întâmplă la nivelul unor componente individuale, cum ar fi un tranzistor defect sau o poartă logică defectă.
- **Defecte la nivel de interconexiune:** Apar la nivelul conexiunilor dintre componente, cum ar fi fire rupte, contacte false etc.
- **Defecte la nivel de sistem (pe de-a-ntregul):** Acestea apar la nivelul întregului sistem de lucru. Pot fi erori de software, la nivel de sursă de alimentare sau chiar defecte la porți care ajung să se propage pe întreg circuitul.

3. După tipul erorii:

- **Defecte pe bază de erori de scurtcircuit:** Conexiuni greșite între două noduri ale circuitului.
- **Defecte pe bază de erori de întrerupere:** Întreruperea unor conexiuni, precum un fir rupt.
- **Defecte pe bază de erori de valoare:** Una din componente prezintă o valoare diferită decât cea specificată în documentație, cum ar fi un condensator cu o capacitate prea mare sau prea mică.
- **Defecte pe bază de erori de sincronizare:** Defecte legate de sincronizarea semnalelor în circuit.
- **Defecte pe bază de erori de memorie:** Erori la nivel de memorie, precum pierderea datelor prin flip-flopi.

4. După gravitatea defectului:

- **Defecte fatale:** Defecte care determină nefuncționarea completă a întregului circuit.
- **Defecte non-fatale:** Defecte care afectează funcționarea normală a circuitului, dar nu îl strică complet.

5. După tipul de circuit:

- **Defecte în circuite combinaționale:** Defecte care afectează funcționarea logicii combinaționale (componentele care nu au nevoie de ceas, precum mux-uri, demux-uri, comparatoare)
- **Defecte în circuite secvențiale:** Defecte care afectează funcționarea logicii secvențiale, precum cele care afectează ceasul sistemului sau logica internă a circuitelor secvențiale.

1.2 Testarea și testabilitatea circuitelor digitale

Testarea și testabilitatea circuitelor digitale sunt esențiale pentru a asigura funcționarea fiabilă și corectă a sistemelor electronice. Acestea sunt formate din mai multe etape și tehnici pentru detecția și corecția de erori ce pot apărea în proiectare, fabricare sau chiar utilizarea zilnică. Printre aceste etape se numără verificarea funcțională, testarea de producție și testarea post-producție.

Cât despre testabilitate, aceasta reprezintă o măsură care ajută să se determine cât de ușor se poate testa un chip/circuit digital. Pentru performanță în această metrică s-au introdus câteva tehnici de testare a circuitelor, și anume tehnici DFT (Design for Testing). Exemple de astfel de tehnici sunt testarea built-in-self-test(BIST) și cea SCAN-path, despre a doua vom vorbi în detaliu în capitolele următoare.

1.2.1 Testare VS Verificare

Aceste două noțiuni fundamentale folosite extrem de des în fabricarea chipurilor și semiconductoarelor, chiar dacă par similare la început, prezintă de fapt conotații diferite și reprezintă două etape diferite, însă complementare una alteia în procesul de asigurare a funcționalității și calității chipurilor.

Diferențele principale între acestea două sunt [6] [7] [8] [9]:

1. Scopul:

- **Testarea:** În principal face referire la evaluarea funcțională și fizică a semiconductoarelor fabricați. Aceasta implică crearea și aplicarea unor teste foarte specifice pentru depistarea anumitor defecțiuni la nivelul chipului, până la cel mai de jos nivel (porți logice defecte). Testarea se poate efectua pe chipuri individuale, creațiile finale ale acestora, dar și în sisteme integrate.
- **Verificarea:** Foarte important de precizat, verificarea se efectuează, spre deosebire de testare, înainte de producția fizică a chipului, atunci când acesta se află încă sub formă digitală (Netlist).

Acest proces se bazează pe crearea și rularea diferitelor tipuri de teste și scripturi în limbaje precum C, Python, System Verilog pentru a evalua corectitudinea schemei electronice/codului firmware al chip-ului. Acesta trebuie să fie în conformitate cu cerințele clientului, de aceea trebuie verificarea oricărui scenariu posibil de utilizare (corner case-uri).

2. Momentul în proces:

- **Testarea:** Așa cum am menționat anterior, testarea se efectuează după ce chipurile sunt fabricate, pentru ca proiectanții să se asigure că acestea funcționează conform cerințelor și specificațiilor finale.
- **Verificarea:** În principiu are loc înaintea fazei de testare, mai precis înaintea procesului de fabricație când încă nu avem forma fizică a chipului, ci doar un prototip (design) în format digital care trebuie verificat înainte să fie transmis pentru producție.

3. Mecanismele utilizate:

- **Testarea:** Se bazează pe utilizarea de teste hardware specializate care aplică semnale de test pe toate intrările și măsoară răspunsurile pe ieșiri, dar și pe programe de test software care evaluează funcționalitatea chipului în diferite condiții și scenarii de lucru.
- **Verificarea:** În absența unei forme fizice a produsului, instrumentele folosite pentru verificare sunt cele software avansate, cum ar fi simulatoare Verilog/VHDL/C care evaluează formal diferite funcționalități având la dispoziție codul pentru Chip.

4. Tipurile de defecțiuni detectate:

- **Testarea:** Detectează defecțiuni (faulturi) de tip fizic, precum conexiuni întrerupte, defecte ale tranzistoarelor sau ale porților logice (transmiterea întârziată a biților). Acestea pot fi însă reproduse la nivel software prin introducerea de constrângeri în design.
- **Verificarea:** Detectează defecte în design, precum erori logice, defecte pe magistralele utilizate (UART, SPI, I2C), conflicte de acces la date sau alte neconcordanțe față de specificațiile dorite.

În concluzie, verificarea și testarea sunt două procese esențiale în industria semiconductoarelor, verificarea asigurând calitatea design-ului în faza de proiect, iar testarea verificând calitatea fizică și funcționalitatea produsului final. Ambele sunt cruciale pentru a asigura fiabilitatea și integritatea componentelor electronice.

1.3 Modelarea defectelor

Un model al unui obiect fizic sau al unui fenomen reprezintă o descriere sau o simulare utilizată pentru a analiza comportamentul acestuia, pentru a studia fenomenul în sine sau pentru a evalua impactul obiectului sau fenomenului asupra mediului înconjurător. [10] Acest aspect dorim să îl analizăm de asemenea și pentru chipurile în curs de producție, folosindu-se modele computerizate pentru a se evalua anumite scenarii și împrejurări ale produsului respectiv pentru a fi supus la stres.

Așa cum în sistemele binare complexe avem ca și modele valorile 0 și 1, în prelucrarea chipurilor vom analiza modelele de defecte și "faulturi" care vor permite introducerea manuală de defecte pentru a putea controla pe deplin circuitul și a-l depăna.

În acest capitol se va discuta despre diverse modele de defecte pentru a eficientiza acest proces, dar mai întâi trebuie să definim clar anumiți termeni referitori: [4]:

- **Fault:** Așa cum s-a precizat anterior, un defect în sisteme electronice se referă la o imperfecțiune la nivel fizic, hardware. Un "fault" pe de altă parte, este o reprezentare a unui defect și este folosit pentru analiza defectelor în software.
- **Eroare:** O eroare este reprezentată la nivel hardware, fizic, și poate afecta propagarea valorilor pe firele sau ieșirile circuitului. La nivel software acestea pot fi interpretate ca și faulturi observate.
- **Defecțiune (failure):** Acestea se întâmplă pe întregul sistem de lucru atunci când un defect se propagă și distruge funcționalitatea acestuia. La nivel software se întâmplă datorită unui fault existent în sistem.

Deci, pentru a putea urmări un circuit defect pe de-a-ntregul este nevoie de un "fault model" bine definit de care ne putem folosi pentru a introduce defecte în mod digital.

În continuare, vom analiza posibilitățile de apariție și percepere a defectelor pentru a avea un fundal mai clar despre cauza acestora. Atunci când un defect fizic apare în sistem, acesta poate fi văzut în 4 nivele de abstractizare: la nivel de switch, poartă logică, RTL (Register Transfer Level) sau la nivel de sistem. [11] Ce este de interes însă în această lucrare de licență sunt defectele la nivel de poartă logică.

Să presupunem că avem un multiplexor 2x1 defect pe porțiunea liniei de selecție ca în figura următoare:

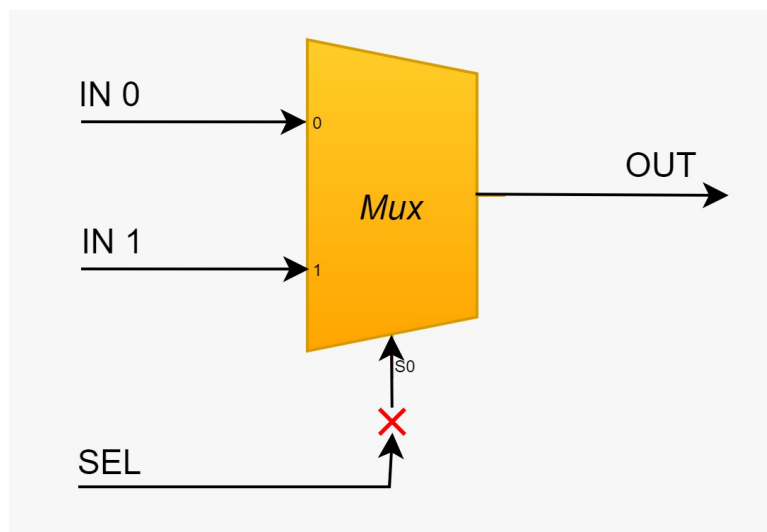


Figura 1.1: Mux 2x1 în care intrarea de selecție prezintă defect

Tabelul de adevăr pentru un multiplexor funcționând corect este următorul:

Intrări			Ieșire
Sel	In0	In1	Out
0	0	X	0
0	1	X	1
1	X	0	0
1	X	1	1

Tabela 1.1: Tabel de adevăr pentru un mux 2x1 funcționând corect

După cum putem observa, atunci când selecția ia valoarea 0 înseamnă că se ia în calcul intrarea in0, iar analog când selecția ia valoarea 1 intrarea de interes va fi in1, ieșirea propagându-se din acestea. Deci una din intrări nu contează în ambele cazuri, de unde și valoarea de "X".

Însă problemele care pot apărea la intrarea de selecție sunt următoarele: aceasta poate să rămână blocată în una din valori (0 sau 1) și atunci jumătate din tabelul de adevăr va fi eronat, sau valoarea propagată pe selecție poate să fie întârziată, și atunci selecția realizează operația corectă dar nu în timpul necesar, iar pentru a detecta acest lucru este nevoie de seturi speciale de teste.

Putem lua ca exemplu selecția blocată în 0, în acest caz vom avea tabelul de adevăr următor:

Intrări			Ieșire
Sel	In0	In1	Out
0	0	X	0
0	1	X	1
1/0	X	0	0/X
1/0	X	1	1/X

Tabela 1.2: Tabel de adevăr pentru un mux 2x1 funcționând incorect

Pentru acest tabel am luat ca și convenție val1/val2 - valoarea așteptată/valoarea reală. Putem observa cum se modifică ieșirea în cazul în care selecția ar fi trebuit să fie 1.

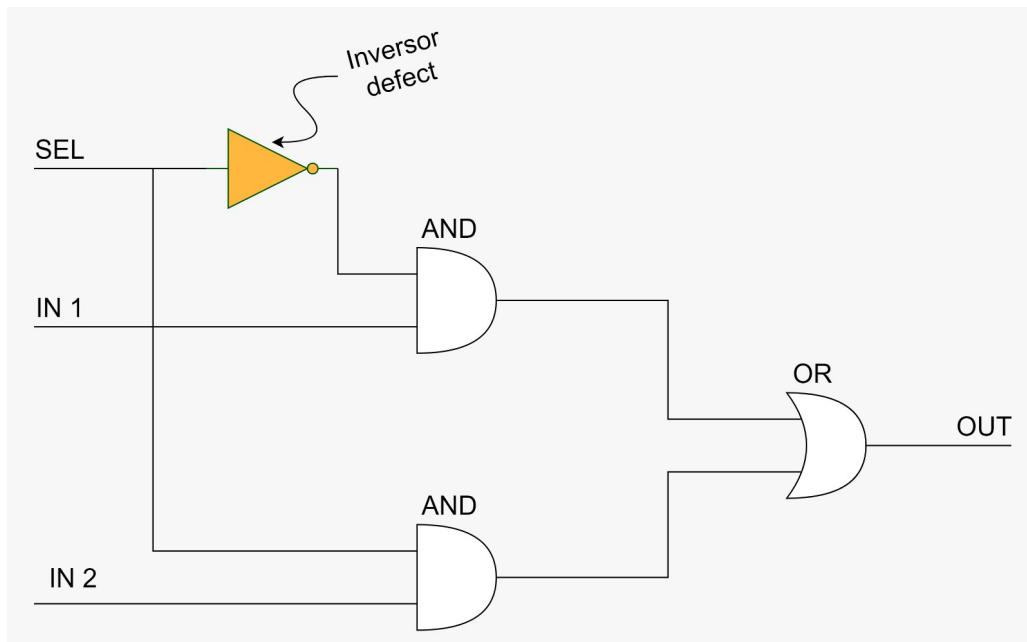


Figura 1.2: Reprezentare cu porți logice a unui mux 2x1 defect

Din această figură se poate observa că, făcând analiza porților logice componente ale multiplexorului, inversorul reprezintă partea care nu funcționează corespunzător, nepermițând selecției să ia valoarea inversă pe una dintre porțile AND. La fel de bine defectul putea să apară la una dintre celelalte porți (AND sau OR), efectul fiind altul la ieșire, și necesitând teste pentru a putea detecta acest lucru. Ducându-ne din ce în ce mai jos în circuit, următoarea figură reprezintă un model de inversor defectuos alături de tabelul de adevăr aferent acestuia. Atunci când pe intrare se aplică valoarea 1, ieșirea acestuia nu va fi 0 cum era de așteptat, ci Z (high impedance), reprezentând fix cazul de defect prezentat anterior, propagat pe selecție. [11]

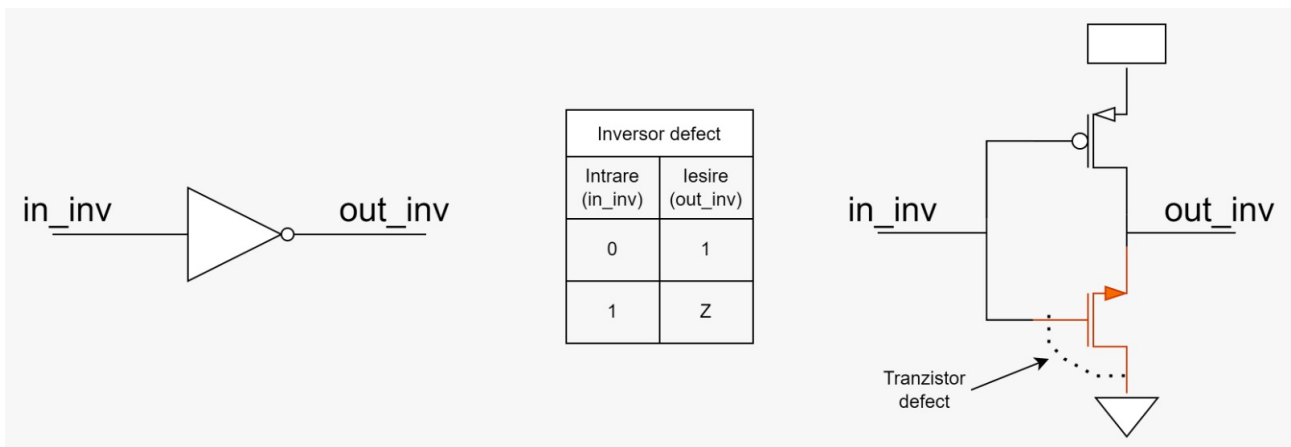


Figura 1.3: Vizualizarea sursei problemei pentru circuit, tranzistorul defect

1.3.1 Defecte funcționale

Defectele funcționale în circuitele digitale (chipuri) reprezintă probleme care afectează funcționarea corectă a circuitului, conducând la eșecuri sau la un comportament neașteptat. [11] De asemenea, un defect funcțional la nivel de poartă conduce la un tabel de adevăr diferit față de o poartă funcțională, de exemplu o poartă NOR care poate să se comporte ca un inversor sau ca un NAND la defecțiune. Ducându-ne la ce s-a discutat anterior, este evident că în cadrul multiplexorului, inversorul prezintă defecte funcționale care se propagă la nivel de întreg sistem.

1.3.1.1 Defecte de tranziție (delay faults)

Defectele de tranziție (de întârziere) în circuitele digitale sunt tipuri de defecte care apar atunci când starea unui element de circuit (un flip-flop, o poartă logică) nu se schimbă în mod corespunzător de la o valoare logică la alta (din 0 în 1 sau din 1 în 0) în timpul necesar, încălcând constrângerile de timp.[11] Aceste defecțiuni pot fi cauzate de mai mulți factori, inclusiv întârzieri excesive în circuite, probleme legate de calitatea semnalului sau caracteristici defectuoase ale dispozitivelor. În general, defectele de tranziție afectează performanța unui circuit digital și pot duce la funcționare incorectă.

Există două tipuri principale de defecte de tranziție în circuitele digitale:

1. Defecte de tranziție lentă la creștere (slow-to-rise):

- Acestea apar atunci când circuitul prezintă o întârziere semnificativă în schimbarea stării semnalului de la 0 la 1.

2. Defecte de tranziție lentă la cădere (slow-to-fall):

- Acestea apar atunci când circuitul prezintă o întârziere semnificativă în schimbarea stării semnalului de la 1 la 0.

3. Defecte de blocare în tranziție (Stuck-At Faults):

- Acestea sunt defecte în care o poartă logică rămâne permanent într-o stare logică, fie 0 (stuck-at-0) fie 1 (stuck-at-1), și nu poate trece la cealaltă stare.
- Pot fi clasificate de asemenea și ca defecte de tip structural.

Primnele două categorii se pot testa cu așa numitele teste ale timpului de răspuns (delay testing).[10] Acestea implică aplicarea unui set de vectori de test și măsurarea timpului de răspuns pentru a detecta defectele de tranziție lentă.

De asemenea, se mai aplica teste de viteză (At-Speed Testing) [10] unde dispozitivul este testat la vitezele sale normale de funcționare., folosind frecvența uzuală de ceas de sistem. Acestea se aplică atunci când circuitul funcționează corespunzător la viteze mici, dar nu la cele normale. Pentru ultimul tip de defecte se folosesc teste funcționale, despre care vom vorbi mai târziu.

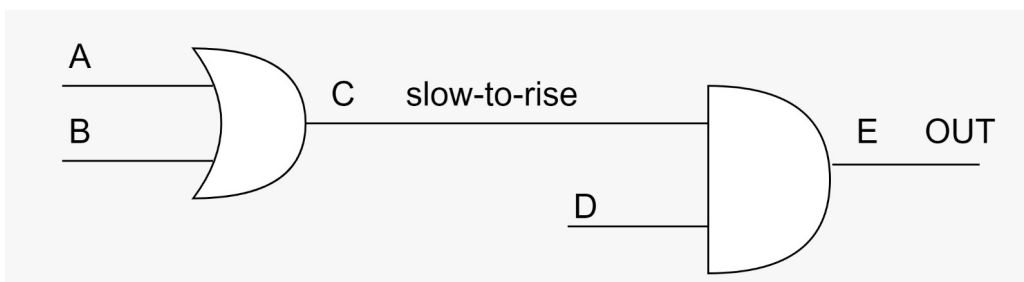


Figura 1.4: Exemplu de propagare întârziată a semnalului în circuit [3]

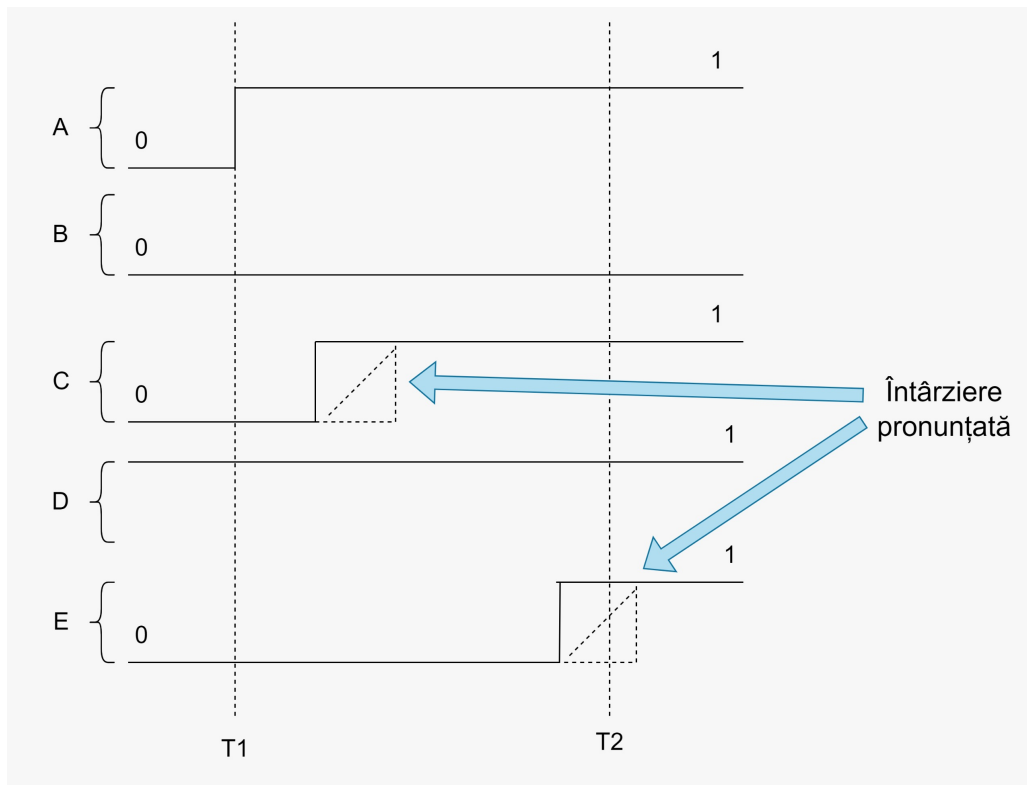


Figura 1.5: Exemplificare a semnalelor pentru a evidenția tranziția întârziată

Pentru figurile 1.4 și 1.5 de mai sus se ia în considerare că $T2-T1$ este o perioadă de ceas, ceea ce înseamnă că ieșirea circuitului din figura 1.4 rămâne 0 la schimbarea frontului de ceas și își schimbă valoarea abia după această trecere, deoarece întâzierea se propagă prin ieșirea porții OR (C) până la ieșirea porții AND (E). Datorită porților AND și OR, intrările B și D nu contează pe tot parcursul procesului deoarece sunt valori neutre.

1.3.2 Defecte structurale la nivelul porților

Defectele structurale în semiconductoare reprezintă defecte fizice în material sau în structura fizică a chipului, care pot afecta performanța și funcționarea acestuia. Aceste defecte pot apărea în timpul procesului de fabricare sau pot fi rezultatul unor factori externi, cum ar fi stresul termic sau mecanic.

Pe de altă parte, defecțiunile structurale la nivelul porților logice se referă la defecte care apar în elementele de bază ale circuitelor digitale, mai precis porțile AND, OR, NOT, etc. Acestea pot fi cauzate de defecțiuni în tranzistori sau în conexiunile dintre porți.

1.3.2.1 Defecte de tip stuck-at

Defectele de tip stuck-at se împart în 2 mari categorii:

1. Defecte de tip stuck-at-0 (SA0):

- O poartă logică sau un nod din circuit este blocat în starea logică 0.
- Poate apărea atunci când circuitul tinde sau este blocat la tensiuni de 0 (masă - ground)

2. Defecte de tip stuck-at-1 (SA1):

- O poartă logică sau un nod din circuit este blocat în starea logică 1.
- Poate apărea atunci când circuitul tinde sau este blocat la tensiuni mari.

Pentru a exemplifica acest aspect, să presupunem că luăm poarta SAU din figura următoare:

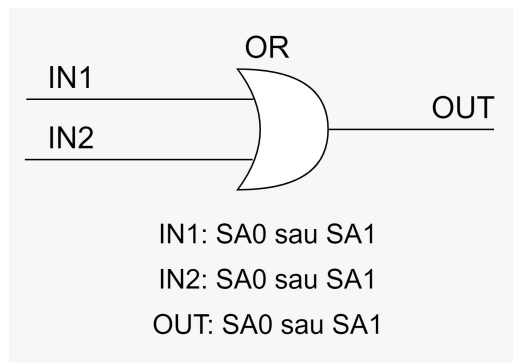


Figura 1.6: Exemplificare a defectelor de tip single stuck-at pe o poartă OR

După cum se poate observa din figură, pentru fiecare intrare și ieșire există câte 2 defecte de tip stuck-at posibile (SA0/SA1), ceea ce înseamnă că pe orice poartă cu k intrări și o ieșire vom avea $2(n+1)$ defecte posibile de tip stuck-at.

Deci în cazul nostru pentru $n=2$ intrări vom avea $2(2+1) = 6$ faulturi stuck-at în total.

Pentru acest tip de defecte se folosește cel mai des testarea funcțională, care utilizează modelul de defect "single stuck-at", cel mai folosit în simularea defectelor, datorită eficienței sale în detectarea multor tipuri comune de defecte. Modelul "stuck-at" descrie comportamentul terminalelor unui circuit care sunt blocate fie la tensiune înaltă (SA1), fie la tensiune joasă (SA0). Vom discuta despre pașii de testare pentru acest tip de teste într-un capitol ulterior, unde se va intra mai mult în detalii.

1.4 Simularea defectelor (faulturilor)

Noțiunea de simulare a defectelor este foarte importantă în fabricarea în masă a chipurilor deoarece are ca scop evaluarea posibilităților de integrare digitală a defectelor într-un chip folosind un model de defecte (fault model) despre care s-a discutat la capitolul 1.3. Această tehnică permite detectarea și diagnosticarea defectelor în circuitele digitale înainte de fabricarea lor, ceea ce reduce timpul și costurile de dezvoltare. Simularea faulturilor se utilizează pentru diferite tipuri de procese, printre care:

- Generarea vectorilor de test (ATPG)
 - Aceștia se aplică la intrare (pe inputuri) pe netlistul sintetizat pentru a detecta anumite tipuri de faulturi.
 - Generarea acestora astfel încât să detecteze tipul de defect corect se realizează cu anumite constrângeri despre care vom vorbi în capitolul ce urmează.
 - Evaluarea vectorilor (patternurilor) de test și a testabilității circuitelor.
- Aceasta se realizează cu ajutorul unor metrici de evaluare a testabilității, calității și fiabilității proiectării, printre care se numără [11] [12]:

– **fault coverage:**

- * Pentru un vector (pattern) de test fault coverage-ul este reprezentat de procentul defectelor detectate raportat la defectele totale ale vectorului de test respectiv, cele totale obținându-se după colaps. [11]

$$\text{fault coverage} = \frac{\text{număr de defecte detectate}}{\text{număr de defecte totale}}$$

– **test coverage:**

- * Reprezintă o măsură a calității testării și este dată de raportul dintre numărul de defecte detectate și numărul de defecte testabile.

$$\text{test coverage} = \frac{\text{număr de defecte detectate}}{\text{număr de defecte testabile}}$$

– **yield** (randament):

- * Reprezintă procentul de chipuri funcționale rezultate dintr-un lot de producție aferent.
- * Cu cât randamentul este mai mare, cu atât procesul de fabricare este mai fiabil și mai controlabil.

$$\text{yield} = \frac{\text{număr de chipuri funcționale din lot}}{\text{număr total de chipuri din lot}}$$

– **Defect rate** (rată de defecte):

- * Este reprezentată de raportul dintre numărul de dispozitive defecte și numărul total de dispozitive testate
- * Ca și unitate de măsură, se exprimă fie în procente (%), fie în părți per milion (ppm)

$$\text{defect rate} = \frac{\text{număr de dispozitive defecte}}{\text{număr total de dispozitive testate}}$$

* Aceste trei metrice se alfa în legătură una cu cealaltă prin următoare formulă: [13]:

$$\text{defect rate} = 1 - \text{yield}^{1 - \text{test coverage}}$$

- Localizarea precisă a defectelor în circuit
- Furnizarea de informații pentru cei care testează și pentru celelalte echipe.
 - Se realizează în principal prin loguri sau alte mijloace prin care aceștia pot să vadă exact cum a decurs simularea respectivă de testare.

Având în vedere ce s-a pus în evidență mai sus, elementele de care este nevoie pentru un mediu de testare și simulare a defectelor sunt următoarele:

- Netlist sintetizat și pregătit pentru inserarea defectelor
- Fișierul conținând vectorii de test
- Fișierul conținând lista de defecte

Ca urmare a acestui proces se produc fișiere de raporte (reports cu extensia .rpt) și fișiere de loguri care pot fi utilizare mai departe în testare.

1.5 Generarea automată a vectorilor de test (ATPG)

1.5.1 Generalități

Un vector de test reprezintă un set de valori de 1 și 0 aplicate pe toate intrările circuitului (dimensiunea lui trebuie să fie egală cu numărul de inputuri) cu scopul de a detecta defecte de un anumit tip. Acestea se generează cu un tool special, numit ATPG, acronim care vine de la Automatic Test Pattern Generation, tehnică esențială pentru asigurarea calității și fiabilității circuitelor integrate. În continuare vom discuta despre funcționarea acestei tehnici, avantaje/dezavantaje și vom explica cât mai în detaliu.

1.5.2 Mod de funcționare

În principal generarea automată a vectorilor de test constă în următoarele etape [10]:

1. Un netlist sintetizat:

Înainte de a genera vectori de test trebuie să avem un circuit bine stabilit pe care să aplicăm acești vectori generați, acest circuit fiind Netlistul sintetizat. Acest netlist descrie circuitul la un nivel abstractizat, alcătuit din porți logice interconectate.

2. Generarea propriu-zisă:

Un algoritm ATPG din cei existenți analizează tipul de defect care se dorește a fi detectat, precum și circuitul, ca mai apoi să genereze automat vectorii de test cu ajutorul unor constrângeri aplicate netlistului.

3. Simularea defectelor(opțional):

Ca o etapă de verificare, se pot simula vectorii de test generați automat într-un circuit de simulare (testbench) pentru a putea observa dacă vectorii de test funcționează corespunzător. Aceștia sunt aplicați la intrare, iar ieșirea rezultată este comparată cu ieșirea ideală pentru acel circuit fără defecte (care este de asemenea conținută în setul de test). Astfel se ia decizia dacă circuitul este fără defecte (fault-free) și dacă vectorul de test și-a atins scopul.

4. Evaluarea test-coverage-ului:

Se calculează o măsură de evaluare numită test-coverage pentru a evalua cât de eficienți au fost vectorii de test generați. Aceasta determină procentul din numărul total de defecte testabile care au fost detectate de acești vectori.

Ca și exemplu, putem lua figura 1.7 care conține un defect de tip stuck-at-1(SA1) la ieșirea primei porți AND.

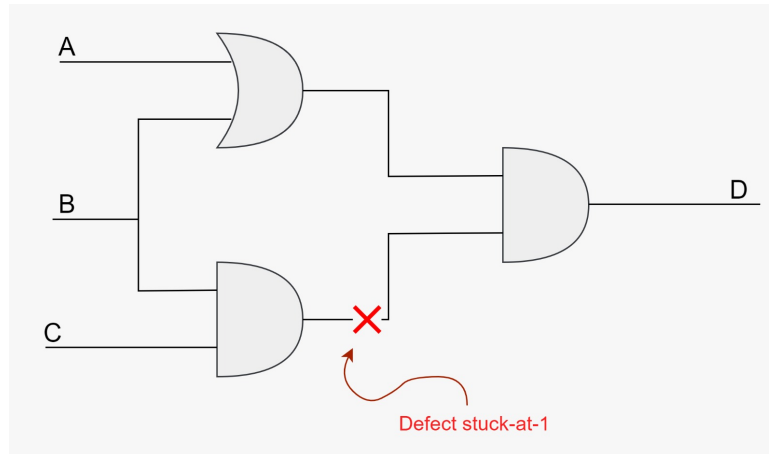


Figura 1.7: Exemplu de circuit cu 3 intrări, o ieșire și un defect stuck-at-1

Intrări (A,B,C)	Ieșire așteptată (D)	Ieșire reală (D_r)
000	0	0
001	0	0
010	0	1
011	1	1
100	0	1
101	0	1
110	0	1
111	1	1

Figura 1.8: Tabelul de adevăr care atestă prezența defectului din figura precedentă

Așa cum s-a vorbit anterior, fiecare poartă prezintă maxim 6 defecte posibile de tip stuck-at, aceștia fiind cel mai întâlnit tip de defect în industria circuitelor digitale. Ne rezumăm însă la a avea un singur defect de tip SA1 la ieșirea porții AND cu intrările B și C.

Având 3 intrări în acest circuit care pot lua valori de 0 sau 1, înseamnă că există 8 combinații posibile de intrări, adică 8 patternuri, și anume: 000, 001, 010, 011, 100, 101, 110 sau 111, așa cum se poate observa în tabelul 1.8. Pentru a putea detecta că acea poartă AND este defectă, va fi nevoie să generăm un vector de test astfel încât pe ieșirea porții să avem valoarea opusă celei în care este blocată poarta, pentru a vedea dacă se schimbă sau nu valoarea la ieșirea ei.

În acest caz având o poartă AND va trebui să avem la ieșirea acesteia valoarea 0, asta înseamnă că pe cel puțin una din intrări va trebui să avem valoarea 0, deci BC poate fi 00, 01 sau 10. În același timp, pentru a se propaga eroarea până la ieșire și a putea fi detectată, ultima poartă AND va trebui să propage ieșirea ultimei porți defecte fără a-i modifica valoarea, asta

însemnând că o valoare de intrare a porții AND de la ieșire va trebui să fie 1, valoarea neutră. Aceste aspecte ne conduc la faptul că din cei 8 vectori posibili pentru ABC doar 4 detectează defectul respectiv, și anume 100, 101, 010, 110, restul nemodificând valoarea defectă de la poarta AND cu probleme.

Acesta fiind un circuit foarte mic, patternurile de test favorabile sunt ușor de detectat, însă pentru un circuit cu milioane de porți logice este foarte dificil acest lucru. Aici intervine toolul de ATPG, care încearcă să genereze vectorii de test care pot să detecteze toate faulturile posibile de tip stuck-at, în caz contrar va clasifica defectele ca fiind netestabile.

1.5.3 Metode ATPG

Principalele două metode de generare a petternurilor de test sunt:

- metoda aleatoare
- metoda deterministă

În plus, tool-ul de ATPG poate simula defecte dintr-un pattern extern și să le pună pe acestea într-un pattern de test cu care să detecteze defecte.

1.5.3.1 Metoda aleatoare

Tool-ul de ATPG (Automatic Test Pattern Generation) folosește generarea de vectori de test aleatori într-un număr foarte mare pentru un circuit prestabilit de producător și dintre aceștia îi identifică și selectează doar pe cei care detectează defecte, salvându-i în continuare. [10]

Cu toate acestea, simularea defectelor utilizată în generarea de vectori de test aleatori nu poate înlocui generarea de vectori deterministă deoarece cea dintâi nu reușește să identifice defecte redundante sau acele defecte cu o probabilitate foarte mică de apariție. Însă, utilitatea acesteia constă în faptul că poate conlucra cu metoda deterministă deoarece se poate folosi la începutul generării vectorilor, urmată mai apoi de metoda deterministă pentru a spori performanța tool-ului.

1.5.3.2 Metoda deterministă

Această metoda are la bază principiul explicat în figura 1.7, și anume următoarele etape, în această ordine [10]:

1. Alegerea unui defect care se dorește a fi detectat.
2. Crearea unui pattern de testare pentru defectul respectiv.
3. Simularea patternului
4. Verificarea corespunzătoare pentru a avea siguranța că patternul a detectat cu succes defectul, comparând ieșirea ideală cu ieșirea reală.

Mai important decât acestea, cum s-a discutat pe tabelul 1.8, instrumentul de ATPG trebuie să aplice constrângeri inteligente și să ia decizii în așa fel încât defectul să se propage în continuare pe circuit până într-un punct de test în care acesta poate fi detectat, caz contrar riscă să se piardă, iar vectorul de test poate fi considerat invalid.

În caz în care se epuizează toate variantele de vectori și defectul nu este detectat, se realizează o analiză mai specială, iar apoi defectul este reclasificat. Această etapă este importantă

deoarece clasificarea defectelor prezintă un rol crucial în calcularea metricilor de performanță, cum ar fi fault coverage-ul.

Spre exemplu, când un defect este redundant, tool-ul îl clasifică ca și nedetectabil, însă în cazuri speciale acesta îl poate considera potențial detectabil și atunci fault coverage-ul nu mai este calculat corect.

1.5.3.3 Metoda de generare externă

Tool-ul de ATPG folosește metoda de generare de patternuri externe doar atunci când sursa ATPG-ului este un set de patternuri externe care există deja. [10] Mai apoi tool-ul se folosește de aceste patternuri externe pentru a vedea dacă se detectează defecte în circuit, după care pe cele mai eficiente le introduce într-un set intern de vectori de test, alegând pe cei care obțin acoperirea (coverage-ul) cea mai mare.

1.5.4 Avantaje și dezavantaje ATPG

Printre avantajele acestei tehnici, se numără [14]:

1. Automatizare ridicată:

- ATPG automatizează procesul de generare a vectorilor de test, reducând în mod semnificativ timpul de testare și mărinde performanțele.

2. Test coverage mai mare:

- Algoritmii ATPG generează vectori de test care pot viza o gamă largă de defecte, ceea ce conduce la o acoperire mai mare a defectelor decât celelalte metode.

3. Producție și lansare pe piață mai rapidă:

- Fiind un proces automatizat, acesta contribuie la accelerarea procesului de testare, și implicit la accelerarea fabricării pe piață a chipurilor.

Cât despre dezavantaje, există câteva, printre acestea fiind [14]:

1. **Cost de testare:** Instrumentele ATPG pot costa mai mult, mai ales pentru circuitele digitale mari și complexe.
2. **Complexitate:** Implementarea algoritmilor de ATPG poate fi complexă și necesită instrumente software specializate.
3. **Timp de rulare:** Generarea vectorilor de test pentru circuitele semnificativ de mari (milioane/miliarde de porți) poate conduce la un timp de rulare proporțional.

1.6 Tehnici de proiectare pentru testabilitate (DFT)

În circuitele digitale și semiconductoare, testabilitatea reprezintă un atribut de design și se referă la cât de ușor pot fi identificate și localizate defectele prin intermediul testării.[15]

La începuturile testării digitale, existau două procese separate: testarea și proiectarea. Conform metodologiilor de atunci, procesul de proiectare era efectuat prima dată, iar apoi la finalul acestuia se efectua procesul de testare propriu-zis.

Însa cu trecerea timpului, s-au dezvoltat tehnici prin care testarea poate fi îmbinată în fazele preliminare ale proiectării circuitului pe punct de fabricație, aceste tehnici numindu-se tehnici DFT (design-for-testing). [11]

Principalele aspecte cheie ale testabilității includ:

1. Observabilitate
2. Controlabilitate

Despre aceste 2 noțiuni vom vorbi în cele ce urmează. Pentru ca testabilitatea să poată fi maximă, cei care proiectează trebuie să utilizeze tehnici speciale de DFT în procesul de dezvoltare, despre care vom vorbi în acest subcapitol.

1.6.1 Tipuri de tehnici DFT

În esență tehnicile DFT pot fi împărțite în 2 mari categorii: tehnici AD-HOC și tehnici structurate.

1.6.1.1 Tehnici ad-hoc

Pe scurt, aceste tehnici se folosesc pentru a îmbunătăți testabilitatea unui circuit fără a-i modifica prea mult design-ul. Nu prezintă o structură sau reguli predefinite pentru utilizare, și se folosesc mai mult ca improvizații depinzând de tipul de circuit testat și de defectele pe care dorim să le detectăm.

Cateva tehnici ad-hoc pot fi următoarele [10]:

- Inserarea punctelor de testare/observație
- Reducerea logicii redundante sau asincrone
- Izolarea semnalelor de ceas de restul circuitului
- Împărțirea circuitelor complexe în circuite mai mici.

Acestea îmbunătățesc testabilitatea circuitului, însă folosind instrumentele Mentor Graphics DFT și tehnicile structurate pe care vom insista în cele ce urmează, putem obține rezultate mult mai bune per total.

1.6.1.2 Tehnici structurate

Tehnicile DFT structurate urmează metode și standarde bine definite pentru a sprijini testarea sistematică și cât mai automată a circuitelor. Aceste tehnici sunt implementate în timpul fazei de proiectare, având ca scop îmbunătățirea testabilității și controlabilității generale a circuitului. Cele mai întâlnite 3 tehnici structurate sunt [10]:

- Tehnica de proiectare SCAN, unde se introduce logică suplimentară în circuit pentru a facilita testarea mai rapidă.
- Tehnica Built-in Self-Test (BIST), care introduce o funcție de testare în dispozitivul însuși.
- Tehnica Boundary Scan (JTAG), care de asemenea crește testabilitatea prin adăugarea de circuite în chip, și mai ales pe frontieră pentru a ușura introducerea pinilor.

1.6.2 Testarea circuitelor combinaționale și secvențiale

Așa cum știm, circuitele foarte complexe conțin atât logică combinațională, cât și logică secvențială. Pentru fiecare dintre acestea se aplică principiul testabilității, și anume îmbunătățirea controlabilității și observabilității, fault coverage-ul, cât și ca generarea testelor să fie cât mai eficientă.

1.6.2.1 Circuite combinaționale

Un circuit combinațional reprezintă un tip de circuit digital în care rezultatul sau ieșirea este determinată doar de starea curentă a intrărilor [16]. Acest lucru înseamnă că stările și valorile anterioare ale intrărilor nu contează pentru rezultatul curent. Ca și exemplu poate fi multiplexorul de la figura 1.1, adunător etc. Acestea pot beneficia de tehnicile DFT, întrucât se poate introduce logică suplimentară în circuite pentru a avea o testabilitate mai mare, cu sacrificiul de a introduce arie suplimentară care afectează producția, însă nu în mod semnificativ.

1.6.2.2 Circuite secvențiale

Circuitele secvențiale reprezintă tipuri de circuite digitale în care starea actuală (cât și ieșirea) nu depinde doar de intrări, ci și de starea anterioară a sistemului. [17]

Pentru a se realiza acest lucru sunt introduse elemente de memorie, precum registre (latch-uri) sau flip-flop-uri. Ca și exemple concrete avem timerele, mașinile cu stări finite (FSM), countere (contoarele) etc. Acestea au în comun faptul că rețin starea anterioară în interior și prezintă semnal de ceas. [18]

Așa cum s-a discutat anterior, putem spune că testabilitatea circuitelor secvențiale poate fi mult mai complicată și costisitoare decât cea a circuitelor combinaționale din cauza faptului că starea internă a sistemului influențează rezultatele. În plus, există posibilitatea apariției defectelor în memorii, care pot fi mult mai dificil de detectat.

Cu toate acestea, folosindu-ne de tehnicile DFT, putem nu doar să facem circuitele secvențiale mai testabile, însă putem altera circuitul și să-l modificăm într-un model de circuit secvențial care poate funcționa într-o mare măsură ca un circuit combinațional pe care putem aplica tehnici DFT cu succes. [11]

Astfel, se pot genera vectori de test cu metode aleatoare sau deterministe care se pot aplica ulterior pe modelul "combinațional" al respectivului circuit secvențial.

După cum se poate observa în figura 1.9, avantajul modelului explicat anterior este conceptul de separație dintre logica combinațională și secvențială dintr-un circuit/chip în curs de testare, fie el cât de complex. Aceasta ne permite să tratăm registrele și elementele de memorie separat de restul circuitului pentru o testabilitate cât mai bună.

În modelul rezultat intrările și ieșirile primare sunt separate de restul circuitului, iar cele pseudo-primare (atât intrări cât și ieșiri) sunt folosite cu scop de testare. Ieșirile registrelor devin intrări pseudo-primare pentru partea combinațională, iar intrările devin ieșiri pseudo-primare, putând controla și propaga valori într-un mod mai ușor pe porțiunile "separate" de circuit.

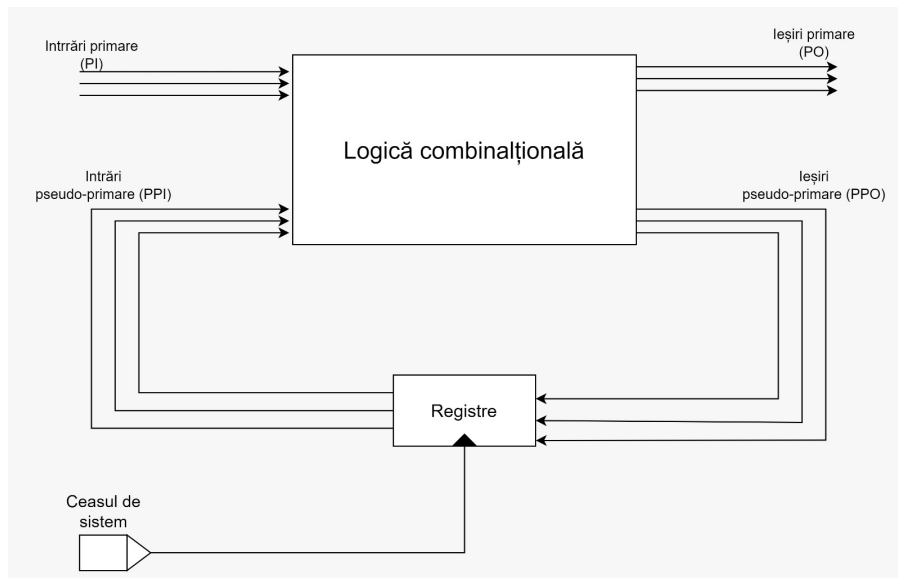


Figura 1.9: Exemplu de circuit secvențial pe care se aplică tehnici DFT

De asemenea, registrele de set, reset, ceas etc. nu sunt testate în acest tip de model și trebuie tratate în mod separat.

Putem exemplifica și mai mult logica secvențială din aceste tipuri de modele prin figura următoare, în care înlocuim registrele cu 4 bistabile de tip D:

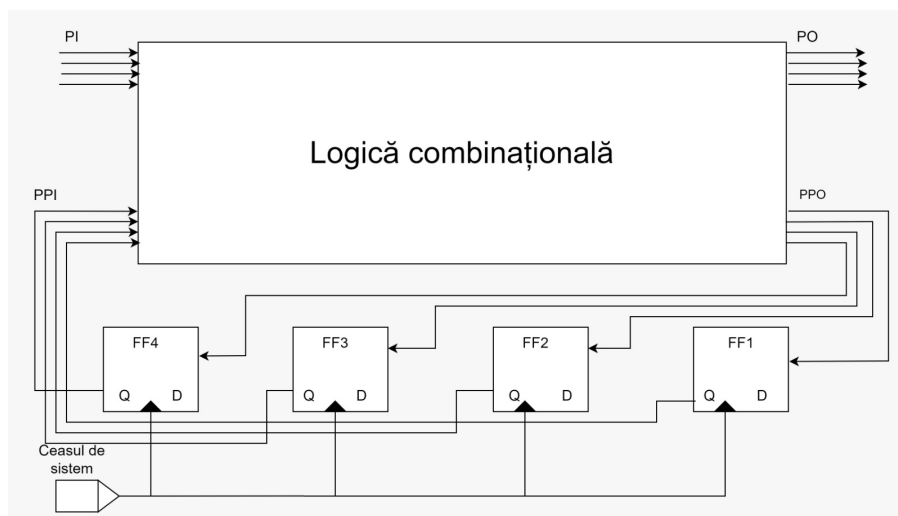


Figura 1.10: Exemplificare a logicii secvențiale și combinaționale folosind 4 bistabile

Vom vedea în capitolul următor cum, făcând inserția de elemente SCAN în circuit, valorile se propagă de la un flip-flop la altul cu ajutorul unor semnale și a unei logici suplimentare de testare.

1.6.3 Observabilitate și controlabilitate

Concepte foarte importante în testarea circuitelor, despre care am discutat în capitolele anterioare, observabilitatea și controlabilitatea trebuie să fie în același timp implicate pentru ca testabilitatea unor circuite foarte complexe să fie maximă.

1.6.3.1 Observabilitate

Observabilitatea reprezintă măsura în care stările interne ale circuitului pot fi observate la ieșirile primare ale acestuia. Un circuit este mai testabil dacă ieșirile reflectă cât mai clar stările interne, cât și defectele.

Pentru a putea îmbunătăți observabilitatea unui circuit, se pot folosi câteva tehnici, printre care [11]:

- **Adăugarea de pini de ieșire**

Prezintă avantajul de a propaga un defect provenit de la o anumită componentă direct la ieșirea circuitului principal, altfel se riscă pierderea acelui defect până acesta ajunge la ieșire, mai ales dacă componenta se află aproape de intrare. Rezultatul este reprezentat de mai puțini vectori de test generați.

- **Extragerea ieșirilor de la flip-flopii de stare către ieșirile primare ale circuitului**

Luând ca exemplu un automat Mealy, putem spune că un defect se poate propaga la o anumită stare și la un anumit ciclu de ceas, acesta disipându-se la următoarea stare dintr-un motiv sau altul. Ei bine, legând ieșirea acestor flip-flopi de stare la o ieșire primară putem captura aceste defecte.

- **Adăugarea de puncte de testare**

Se pot adăuga puncte de testare (test points) oriunde în circuit pentru a spori observabilitatea și a face măsurători directe ale semnalelor interne.

1.6.3.2 Controlabilitate

Controlabilitatea reprezintă capacitatea de a controla și seta stările interne ale circuitului prin intermediul intrărilor primare ale sale. Cu cât stările interne pot fi controlate mai ușor de la intrările primare, cu atât se poate spune că circuitul respectiv este mai testabil.

Valorificând circuitele secvențiale, se poate spune că intrările de flip-flopi pot fi considerate eligibile pentru a fi legate și controlate direct de intrările circuitului. De asemenea, controlul semnalelor de reset, ceas poate ajuta ca aceștia să fie aduși într-o stare dorită.

Față de observabilitate, unde doar trebuia să legăm ieșirea flip-flopilor la ieșirea sistemului, controlabilitatea este puțin mai greu de obținut, însă există metode prin care aceasta poate fi îmbunătățită. Să presupunem că avem logica X și logica Y în circuit, iar între ele trebuie să fie inserat hardware pentru a controla linia dintre acestea. Putem realiza următoarele [11]:

- **Inserare de 0:** După cum se poate observa din figura 1.11, putem controla linia de inserare de 0 direct de la intrarea circuitului, atunci când este 0 circuitul functionând normal, iar când este 1 se propagă valoarea 0 în logica Y.
- **Inserare de 1:** Analog după cazul anterior, exceptând faptul că putem propaga valoarea 1 în logica Y, nu 0.

Aceste 2 metode pot fi combinate pentru o mai bună controlabilitate.

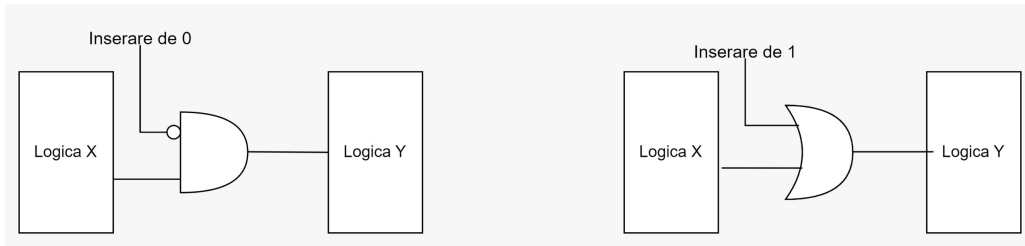


Figura 1.11: Metode pentru îmbunătățirea observabilității

1.7 Tehnologia SCAN

O altă tehnologie care intră în conceptul de Design-For-Testing (DFT) o reprezintă tehnologia SCAN. Aceasta se bazează pe modificarea logicii și circuitelor interne pentru creșterea testabilității.

În principal, scopul acestei tehnici, este transformarea unui circuit secvențial (sau secvențial + combinațional) greu de testat într-un circuit combinațional mult mai ușor de testat. [10] Acest lucru se poate realiza prin modificarea celulelor secvențiale și transformarea lor în celule de scan, iar mai apoi interconectarea acestora în lanțuri de scan pentru a putea propaga datele prin acestea cu ajutorul unor semnale de control (Scan Input și Scan Enable).

1.7.1 Transformarea flip-floilor normali în flip-flopi de Scan [1]

În oricare circuit secvențial, toți flip-flopii sunt transformați în flip-flopi de SCAN, mai puțin următoarele:

- cei pe care îi exclude proiectantul sau testerii, aceștia se vor numi celule non-scan
- cei care prezintă violări DRC (Design Rule Check)

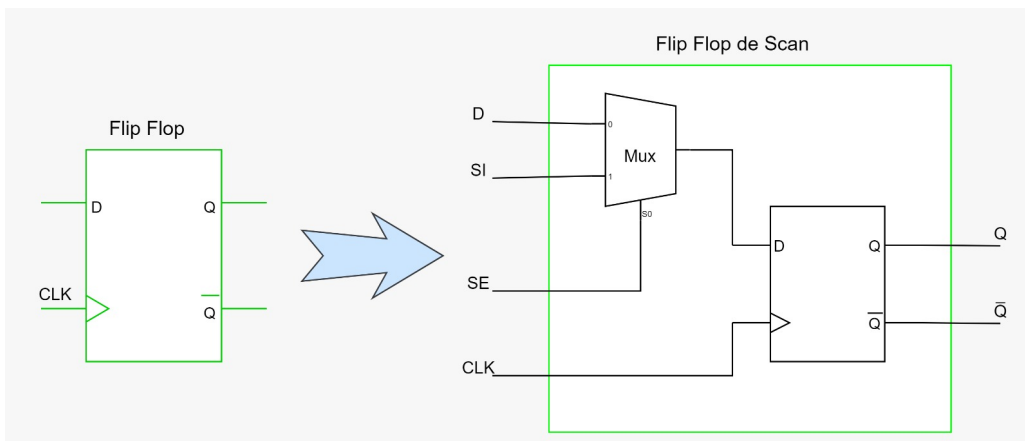


Figura 1.12: Modificarea unui flip-flop normal în flip-flop de Scan

După cum se poate vedea, în figura 1.12 este reprezentată transformarea unui flip-flop normal într-o celulă de Scan. Pentru aceasta transformare, unui flip-flop normal i se adaugă un multiplexor, iar apoi acestea sunt instanțiate într-un modul de top care reprezintă celula propriu-zisă de Scan.

Multiplexorul se folosește pentru a putea controla valorile care se propagă la intrarea celulei. Acesta are ca și intrări: D (intrarea propriu-zisă) și semnalul Scan Input (intrarea care este

adăugată în faza de testare), iar ca și selecție o reprezintă semnalul de Scan Enable, pe care îl folosim pentru a comuta între cele 2 intrări. Când $SE=0$ (sau numită fază de capturare), flip-flopul ia valoarea normală, iar când $SE=1$ (sau fază de shiftare) acesta ia valoarea pe care i-o dă testerul în mod personalizat, adică Scan Input. Restul, mai precis ieșirile și intrarea de ceas etc rămân identice.

1.7.2 Metodologia de proiectare Scan

Această metodologie spune că, pentru a observa și controla cât mai bine elementele de memorie dintr-un circuit trebuie ca acestea să fie transformate în elemente de scan care să fie interconectate de la intrare până la ieșire, această grupare numindu-se un lanț de scan (scan chain). [10]

Valorile sunt propagate la intrare (Scan Input) și deplasate mai departe secvențial prin componentele lanțului de scan, fiind mai apoi capturate la ieșirea de scan (Scan Output). Mai jos este un exemplu simplu de lanț de scan într-un circuit cu diferite elemente de memorie și elemente combinaționale.

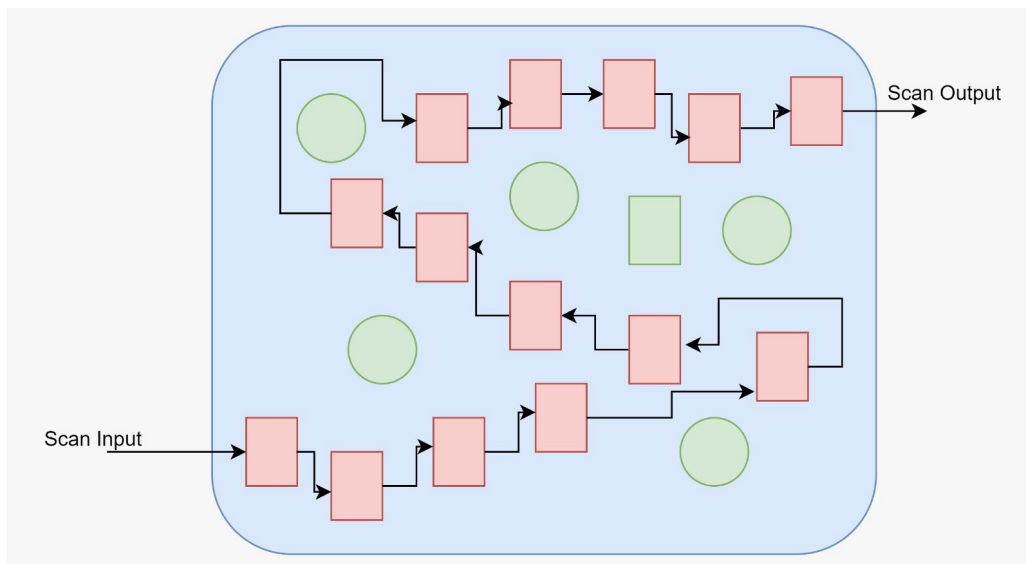


Figura 1.13: Exemplu de lanț de Scan

În acest exemplu celulele verzi reprezintă elementele combinaționale sau cele care au fost excluse de către tester, iar cele roșii reprezintă cele secvențiale, transformate în celule de scan și legate între ele prin lanțul de scan, care propagă valorile de la intrare la ieșirea de scan.

Ca și avantaje privind această tehnologie avem următoarele [10]:

- **Ușurința în folosire:** Cu ajutorul acestei metodologii, se reușește inserarea elementelor scan împreună cu rularea tool-ului de ATPG fără a avea nevoie de ajutor suplimentar.
- **Automatizare:** Proces în preponderență automat cu ajutorul instrumentelor software puse la dispoziție.
- **Eficiență:** Aceasta metodă prezintă atât eficiență ridicată, cât și beneficiul unui test coverage mai bun.
- **Calitate asigurată:** Având la dispoziție posibilitatea testării riguroase a fiecărei componente și flip-flop în parte, este asigurată calitatea maximă a produsului.

Cu alte cuvinte, reluând figura 1.10 și aplicând metodologia de SCAN, se poate obține următoarea structură:

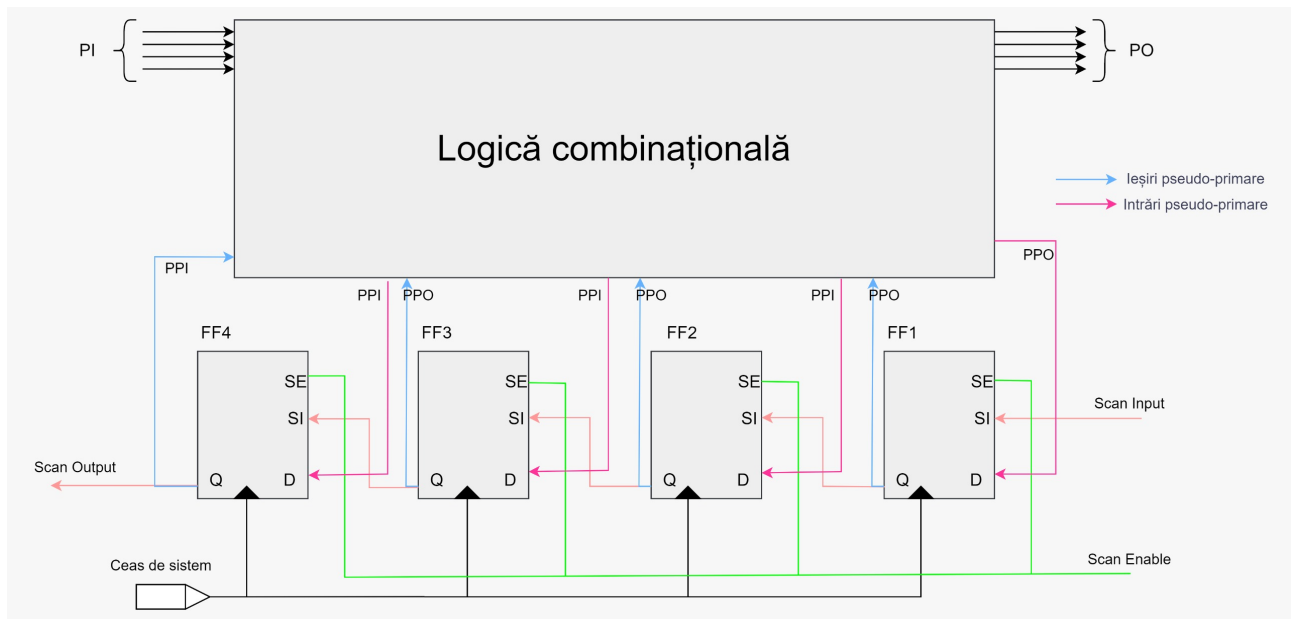


Figura 1.14: Exemplu de circuit testabil după inserarea logicii de SCAN

Făcând o analogie simplistă, în acest mod arată un circuit testabil după ce i se aplică inserția SCAN. Logica secvențială poate fi controlată din semnalele de Scan Input, Scan Output și Scan Enable, făcând procesul de testare și generare de vectori mult mai simplu și facil.

1.8 Embedded Deterministic Test

Embedded Deterministic Test (EDT) este o tehnologie avansată de testare utilizată pentru a reduce costurile și complexitatea testării circuitelor digitale complexe. Aceasta combină metodele de testare și de generare de vectori de test (ATPG) deterministice cu tehnici de compresie a vectorilor de test pentru a eficientiza procesul de testare și pentru a minimiza volumul de test necesar, două elemente esențiale.

În cazul procesului de ATPG normal, se vizează un anumit tip de defecte care se dorește a fi detectate și se determină valorile pentru intrări care ajută la depistarea și propagarea defectului. Însă pentru restul flip-floperilor care nu contribuie la detecția defectelor se generează valori aleatoare la intrare deoarece propagarea lor este irelevantă. Acești biți duc la memorie și timp de testare în plus inutil, ceea ce este un dezavantaj.

EDT-ul rezolvă această problemă, extrăgând doar acei biți utili pentru flip flopii vizati și formând un pattern de EDT [19]. Aceștia sunt transportați în prima componentă a EDT-ului, decompresorul, după care mai departe în scan chain-urile propriu-zise. Biții nespecificați la intrare sunt înlocuiți cu valori aleatoare în decompresor, dar care nu afectează restul circuitului. Acest lucru reduce numărul de cicli semnificativ, deci și test-time-ul.

1.8.1 Avantaje și caracteristici EDT

Următoarele informații se pot considera atât avantaje cât și caracteristici, așa că vor intra la aceeași categorie. Acestea sunt [20]:

1. Compresia vectorilor de test:

- EDT folosește tehnici avansate de compresie a vectorilor de test pentru a reduce numărul vectorilor necesari acoperirii tuturor scenariilor de testare.
- Reduce timpul de testare și volumul de date, ceea ce duce la costuri semnificativ mai reduse.

2. Capacități deterministe:

- Spre deosebire de metodele de generare aleatoare, EDT asigură că toate defectele detectabile sunt identificate printr-o secvență deterministă de vectori de test.
- Crește fiabilitatea și acuratețea testării, reducând riscul de defecte care pot apărea nedetectate.

3. Integrarea în proiectare:

- EDT este integrat în faza de proiectare (design) a circuitelor, permițând optimizarea testabilității de la început.
- Micșorează semnificativ necesitatea de modificări ulterioare și asigură că circuitul este testabil.

1.8.2 Structura EDT

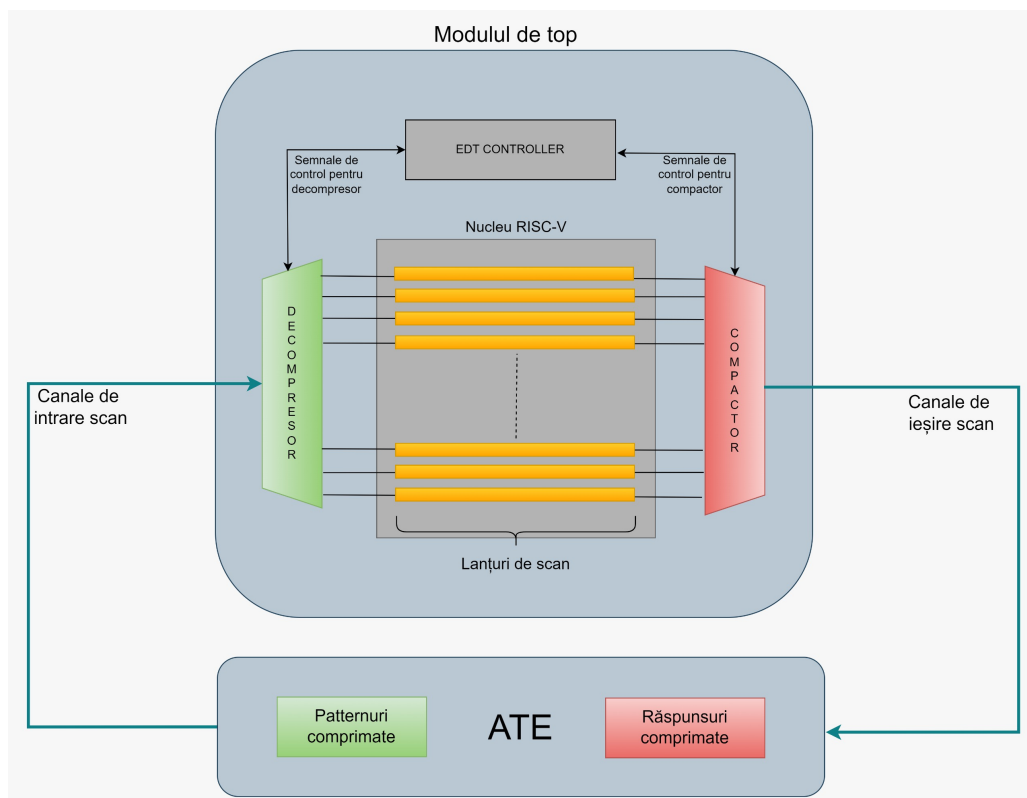


Figura 1.15: Structura EDT-ului conectat la un nucleu RISC-V

Modulul EDT pentru această lucrare este format din 3 componente: decompresor, compactor și controller. Acestea se regăsesc în figura 1.15.

După cum se poate observa, EDT-ul nu introduce logică suplimentară în circuitul principal, ci doar în afara acestuia, contribuind la canalele de SCAN. În acest exemplu există mai multe

lanțuri interne de scan în design, însă datorită decompresorului și compactorului, cât și a controllerului, din exterior se percep un număr de canale de scan mai mic decât numărul de scan chain-uri, ceea ce prezintă o diferență semnificativă.

1.8.2.1 Decompresor

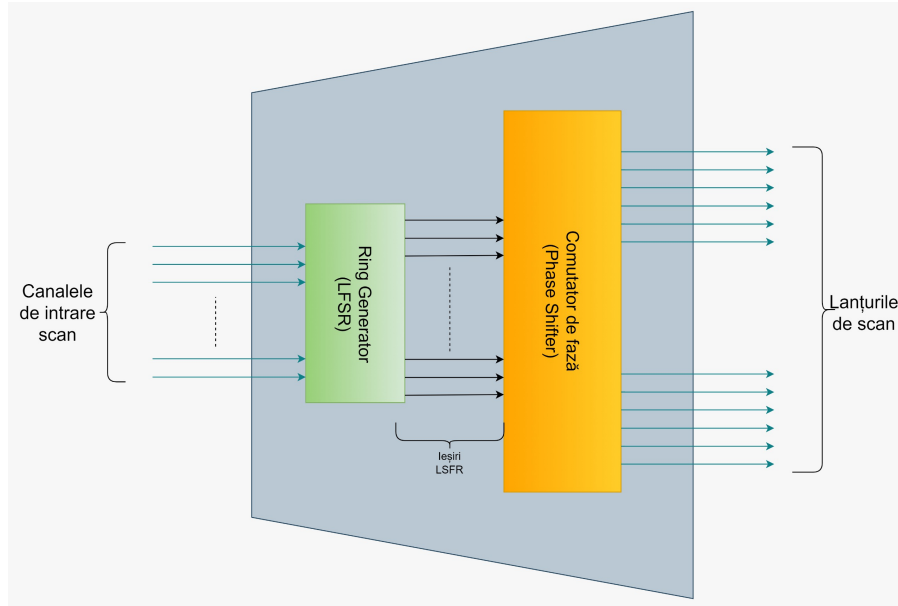


Figura 1.16: Structura unui decompresor

În figura 1.16 este reprezentată structura internă a unui decompresor din cadrul modulului de EDT. Acesta, după cum s-a menționat anterior, are rolul de a transforma patternul de EDT într-un pattern de ATPG de dimensiunea numărului lanțurilor de scan, pentru a putea deplasa corect datele în interiorul modulului.

Acesta este format din 2 componente cheie:

1. Un Ring Generator, mai exact un Ring LSFR (Linear Feedback Shift Register) cu intrări externe (canalele de scan).
 - Un circuit LSFR (Linear Feedback Shift Register) este un tip de registru de deplasare utilizat în special în aplicații de generare a secvențelor de numere pseudo-aleatoare. Acesta este compus dintr-o serie de biți care sunt deplasați într-un mod determinist sub acțiunea unei secvențe de comenzi. Cu ajutorul flip-flopilor săi interni și parametrilor folosiți, acesta poate calcula funcții polinomiale complicate, utile în multe aplicații practice. În figura 1.17 este reprezentată forma unui Ring LFSR (Ring Generator) pe 8 biți în care componentele sunt interconectate și care implementează ecuația polinomială

$$f(x) = x^8 + x^5 + x^2 + 1 \quad (1.1)$$
 - Avantajul major al intrărilor externe în acest tip de circuit este faptul că se pot introduce mai multe variabile deodată, deci probabilitatea de rezolvare a ecuației crește, prin urmare și test-coverage-ul va crește.
 - Acest Ring LSFR practic rezolvă o ecuație polinomială foarte complicată definită odată cu definirea EDT-ului, având ca variabile de intrare ale ecuației valorile canalelor de scan și valorile interne ale LFSR-ului (valorile de seed).

Astfel la ieșire vom avea rezultatele polinomului care reprezintă un vector de test modificat, format din biții vizați să detecteze tipul de defect specificat.

2. Un comutator de fază (Phase Shifter)

- Acesta face legătura dintre lanțurile de scan propriu-zise din circuit și ieșirile Ring LFSR-ului.
- De asemenea, realizează extinderea dimensiunii vectorului de la ieșirea Ring Generatorului până la lungimea unui vector de test normal ATPG, mai precis numărul de lanțuri de scan. Acest lucru se realizează cu ajutorul mai multor porți XOR.

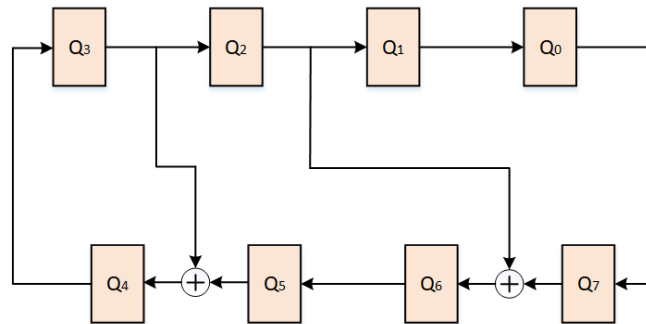


Figura 1.17: Un exemplu simplu de Ring LFSR care calculează o funcție polinomială

1.8.2.2 Compactor

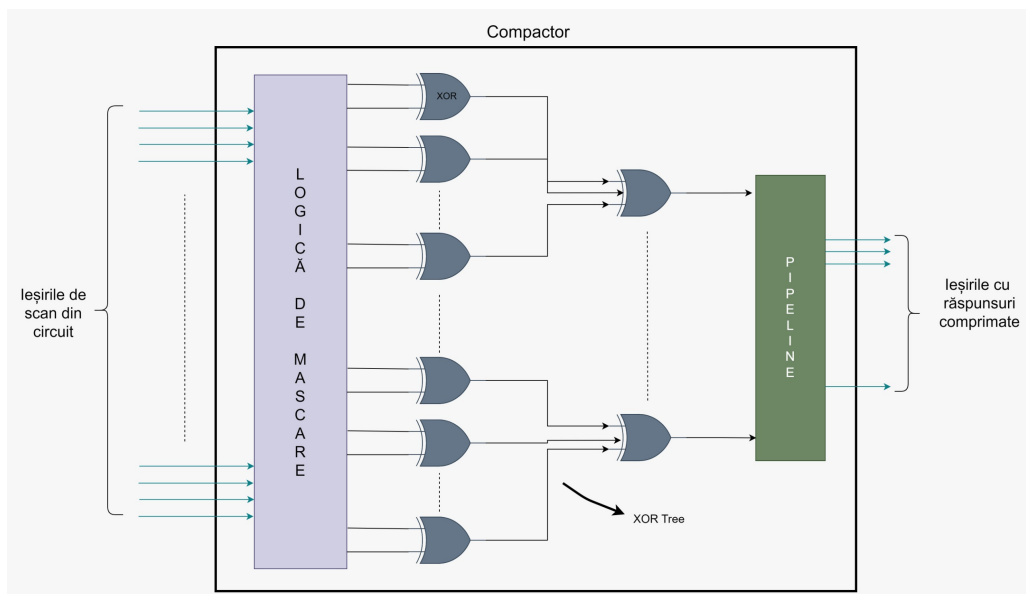


Figura 1.18: Structura unui compactor

Figura 1.18 reprezintă structura unui compactor dintr-un circuit EDT. Principalele roluri ale acestuia sunt reducerea datelor de răspuns și menținerea integrității informațiilor, acestea nefiind afectate. După cum putem observa din figură, compactorul, în parametrii săi de funcționare normală este format din următoarele elemente:

- **Logică de Mascare (Masking Logic):** Logica de mascare este utilizată pentru a selecta și "masca" anumite porțiuni din răspunsurile de la ieșire ale circuitului în timpul compresiei datelor.

- * Maschează datele inutile sau zgomotul din răspunsuri, permițând compactorului să se concentreze pe datele care ne interesează pentru detectarea defectelor.
 - * Permite evitarea coliziunilor de date atunci când răspunsurile multiple sunt comprimate simultan.
- **Arbore XOR (XOR Tree):** Aceasta reprezintă o structură logică utilizată pentru a combina și comprima datele de răspuns utilizând operații și porți XOR.
 - * Combină mai multe răspunsuri într-un singur răspuns comprimat, folosind operații XOR pentru a reduce volumul de calcul și de date.
 - * Permite mai multor lanțuri de scan să fie observate în același timp datorită combinărilor de operații XOR folosite.
 - **Registre de Pipeline:** Acestea sunt utilizate pentru a sincroniza datele de răspuns în timpul procesului de compactare.
 - * Ajută la păstrarea temporară a datelor de răspuns provenite din porțile XOR înainte de a fi procesate și transportate la ieșire.
 - * Permite un flux continuu de date, prevenind blocările și asigurând că toate datele sunt procesate în ordinea corespunzătoare.

1.8.2.3 Controller

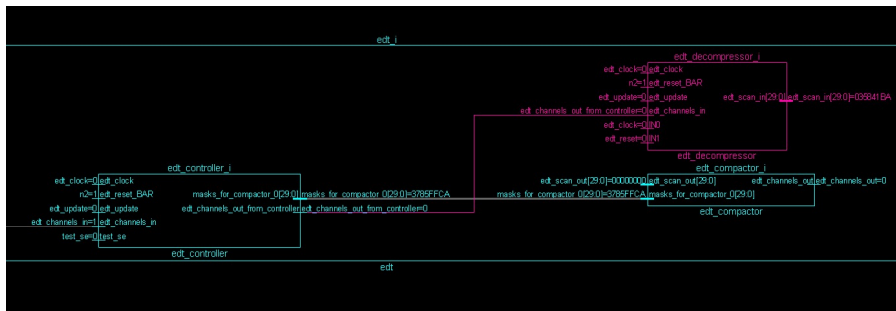


Figura 1.19: Controller+decompresor+compactor

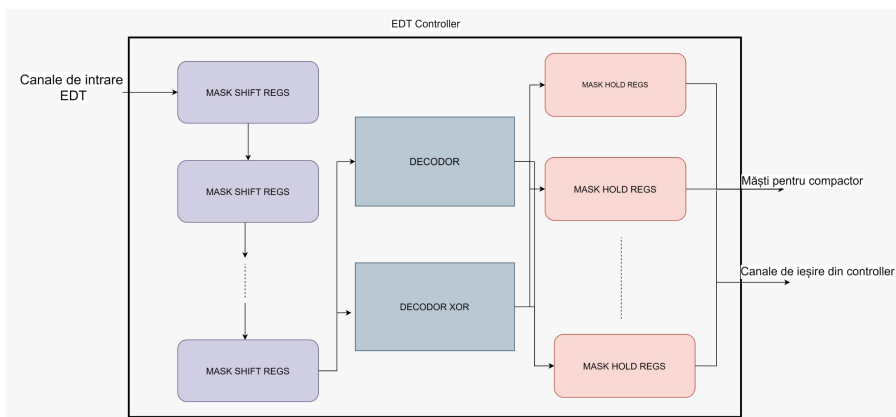


Figura 1.20: Structura unui controller EDT

Figura 1.19 reprezintă controllerul de EDT, decompresorul și compactorul, împreună cu conexiunile dintre acestea, iar figura 1.20 reprezintă structura internă a unui controller de EDT. Acesta este format din:

- Regiștrii de shiftare mascați (mask shift registers)
- Regiștrii de hold mascați (hold shift registers)
- Decodor
- Decodor XOR

Principalele funcții ale acestui modul sunt următoarele:

- Calculează măștile despre care s-a vorbit anterior pentru compactor întrucât atunci când se face deplasarea datelor în lanțurile de scan spre ieșire, se pot selecta doar anumite chain-uri să fie compactate, tool-ul făcând această mascare a chain-urilor în interiorul controllerului.
- În timpul deplasării datelor, acestea trec prin regiștrii de shiftare mascați, iar valorile sunt salvate temporar în regiștrii de hold mascați pentru diferite operații interne între biții fiecărui scan-chain (operații XOR).

Cu alte cuvinte, prin regiștrii de shiftare se deplasează date constant, după care se salvează în regiștrii de hold pentru următorul pattern, moment când se schimbă semnalul `edt_update`.

1.8.3 Mod de funcționare EDT

În principiu, EDT-ul funcționează în felul următor:

1. Mai întâi se generează vectorii de test corespunzători folosind ATPG și metode deterministe.
2. Se comprimă acești vectori de test prin algoritmi de compresie fără pierderea datelor.
3. Se "decomprează" vectorii înainte să fie aplicați pe chip-ul propriu-zis folosind decompresorul.
4. Se aplică pattern-urile de test și se deplasează prin lanțurile de scan.
5. Se capturează răspunsurile de la ieșire în registre de scan.
6. Folosind compactorul se face compresia răspunsurilor capturate la ieșire.

1.8.4 Tessent TestKompress

Tessent TestKompress este un instrument avansat de la Mentor Graphics, parte din Siemens EDA, care implementează tehnicile EDT discutate anterior pentru a optimiza procesul de testare a circuitelor digitale.

Cu alte cuvinte, Tessent TestKompress reprezintă tool-ul care generează logica de decompresor și compactor din cadrul EDT-ului la nivel RTL [19]. Acesta este utilizat pe scară largă pentru a genera, comprima și aplica vectori de test în mod eficient, reducând semnificativ volumul datelor de testare și timpul necesar.

1.8.4.1 Avantaje și caracteristici TestKompres

La fel ca și pentru EDT, putem enumera câteva avantaje și caracteristici în utilizarea TestKompres [21] [22]:

- **Generare de vectori de test compriși:**
 - TestKompres utilizează algoritmi avansați pentru a genera vectori de test care sunt compriși, reducând volumul de date necesar testării.
 - Reduce necesitatea unor echipamente de testare de capacități mari, realizând costuri reduse.
- **Reducerea timpului și volumului de testare:**
 - Realizând compresia vectorilor de test, se reduce de asemenea semnificativ timpul și volumul de testare.
- **Performanțe EDT**
 - TestKompres implementează logica EDT pentru a asigura că toate defectele sunt detectate în mod eficient cu un set minim de vectori de test.
- **Maximizarea fault coverage-ului**
- **Suport pentru diverse arhitecturi:**
 - Suportă o gamă destul de largă de arhitecturi și tipuri de circuite, inclusiv FPGA și ASIC.
- **Suport de defecte semnificativ**

1.8.4.2 ATPG vs EDT

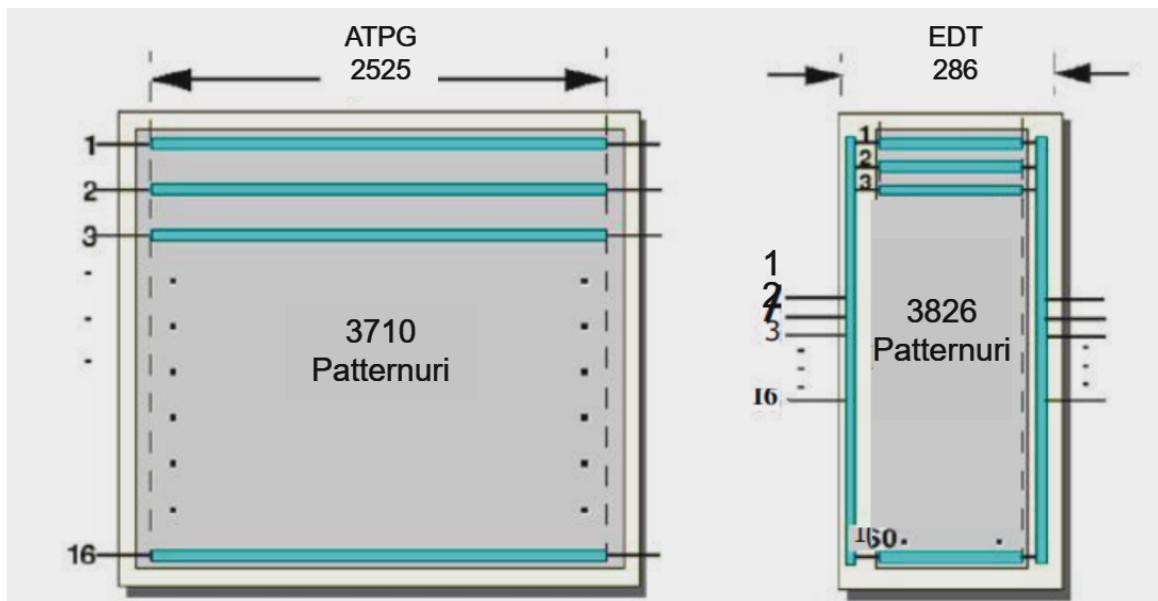


Figura 1.21: Comparatie între ATPG normal si ATPG integrat cu EDT

Făcând o comparație side-by-side între ATPG normal și ATPG cu EDT, putem stabili cu siguranță care este mai performant. Să luăm exemplul din figura 1.21 în care avem un proces

normal, fără compresii ATPG și avem 3710 vectori de test generați, iar lungimea unui lanț de scan este de 2525 (exemplul din figură este reprezentat de un circuit mai complex). Pe de altă parte, în privința EDT-ului avem 3826 de patternuri de test (din cauza introducerii de ciclii EDT), iar lungimea unui lanț de scan acum este 286 deoarece s-au introdus mai multe lanțuri de scan. Făcând raportul ciclilor totali necesari testării aceluiași circuit, obținem:

$$\text{raport} = \frac{\text{ciclii de testare ATPG}}{\text{ciclii de testare EDT}} = \frac{\text{nr patternuri ATPG} * \text{lungime lanț de Scan ATPG}}{\text{nr patternuri EDT} * \text{lungime lanț de Scan EDT}}$$

Rezultă:

$$\text{raport} = \frac{3710 * 2525}{3826 * 286} = 8.56$$

Cu alte cuvinte, folosirea EDT-ului în testarea și generarea de vectori prezintă o performanță mult mai ridicată (de aproximativ 8.56 de ori) față de folosirea tehnicii ATPG normale, reducând timpul de testare semnificativ.

Capitolul 2

Descrierea sistemului [2]

În continuare voi vorbi despre modulul pe care l-am utilizat în această lucrare de licență și de la care a pornit totul. Acestuia i s-a aplicat întreg flow-ul de DFT despre care am discutat în capitolele anterioare.

Chiar dacă nu s-au folosit caracteristicile sale structurale și funcționale la capacitate maximă (deoarece a fost nevoie doar de netlistul circuitului în Verilog), voi vorbi pe scurt despre fiecare în parte.

2.1 Arhitectura SoC bazată pe tehnologia RISC-V

Proiectul reprezintă un SoC (System-on-Chip) de test bazat pe tehnologie RISC-V, utilizând un nucleu RV32IM (care este capabil să booteze Linux).

SoC-ul include un set de periferice (GPIO, UART, I2C, Timer) și este destinat în principal pentru implementări pe FPGA, cum ar fi Digilent Artix-7 Arty (reprezentată în figura 2.1).

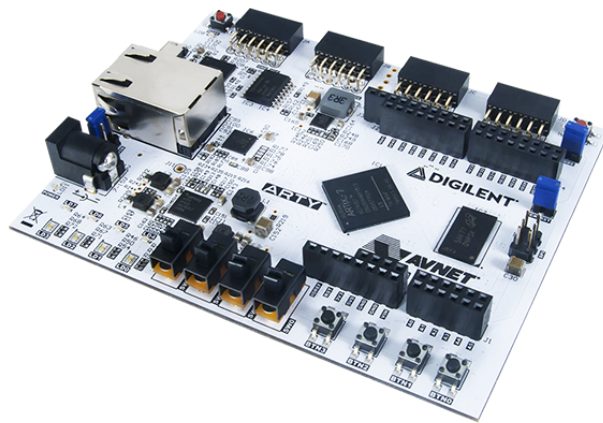


Figura 2.1: Plăcuța FPGA Digilent Artix-7 Arty

Implementarea arhitecturii RISC-V prezintă un set de instrucțiuni open-source proiectat pentru o gamă largă de sisteme de calcul. CPU-ul poate fi adaptat pentru nevoi diferite de performanță și eficiență. De asemenea, această arhitectură se remarcă prin simplitatea și flexibilitatea sa.

2.2 Module dedicate

2.2.1 Timer

Modulul de Timer este utilizat pentru generarea întreruperilor bazate pe timp. Acesta conține registre pentru setarea valorii curente (count), a valorii de comparat (compare) și a controlului timerului (control) și se află în fișierul `rtl/timer.v`.

Timerul poate fi configurat să genereze o întrerupere atunci când valoarea curentă ajunge la valoarea de comparat prin utilizarea semnalului `irq`, facilitând astfel sincronizarea la nivelul întregului sistem de lucru.

2.2.2 UART

Modulul UART (Universal Asynchronous Receiver/Transmitter) , regăsit în `rtl/uart.v`, este destinat comunicațiilor seriale duplex sau full-duplex. [23]

Acesta include registre pentru transmiterea (`tx_data`) și recepționarea (`rx_data`) datelor, controlul acestora (control), stare (status), precum și pentru configurarea ratei de baud, a parității și a bitului de stop. UART-ul facilitează comunicarea serială cu alte dispozitive externe, esențială pentru multe aplicații embedded. De asemenea, folosește un buffer (memorie tampon) de date FIFO (First-In-First-Out) pentru a gestiona transmiterea și primirea datelor.

2.2.3 SPI

În comparație cu modulul de UART, modulul de SPI (Serial Peripheral Interface), regăsit la calea `rtl/spi.v`, permite comunicația serială sincronă cu alte dispozitive.

Acesta are în componența sa registre pentru configurarea ceasului de SPI (CPOL, CPHA), a modului de funcționare (master/slave), transmiterea și recepționarea datelor (`tx_data`, `rx_data`), cât și starea SPI-ului (status). Acesta utilizează de asemenea un buffer de memorie de tip FIFO pentru manipularea datelor.

SPI este crucial pentru interfațarea cu diverse periferice precum senzori, memorii flash și alte microcontrolere.

2.2.4 GPIO

Modulul GPIO (General Purpose Input/Output) care se regăsește la `rtl/gpio.v`, oferă pinii configurați pentru intrări, respectiv ieșiri digitale.

Fiecare pin GPIO poate fi setat individual ca intrare sau ieșire, iar modulul include registre pentru setarea direcției (direction), citirea și scrierea valorilor (input, output), și configurarea întreruperilor pentru schimbările de stare ale pinilor.

2.2.5 AXI RAM

[24] Advanced eXtensible Interface (AXI) reprezintă un protocol de magistrală de comunicații on-chip și prezintă legături și ramificații cu protocolul AMBA (Advanced Microcontroller Bus Architecture).

AXI RAM (Advanced eXtensible Interface Random Access Memory) reprezintă interfața de memorie, utilizând protocoalele AXI4 și AXI4-Lite pentru a facilita accesul rapid la memoria RAM.

Aceasta permite nucleului RISC-V și perifericelor să acceseze memoria cat mai eficient, fiind esențială pentru performanța generală a sistemului. Interfața AXI asigură transferuri de date de mare viteză, crucială pentru operațiile care necesită lățime de bandă ridicată.

Aceste module explicate mai sus sunt integrate pentru a oferi un SoC funcțional, utilizabil în diverse aplicații și scenarii de testare într-un mediu FPGA.

2.3 Modulul de top și instanțierea SoC-ului

Modulul de top al SoC-ului este descris în fișierul `rtl/top.v`. El integrează toate componentele, cum ar fi nucleul RISC-V, perifericele (Timer, UART, SPI, GPIO) și memoria AXI RAM, într-o singură entitate.

Acesta gestionează semnalele globale, cum ar fi cele de reset și ceas, și instanțiază interfețele AXI pentru conectarea modulelor.

2.4 Logica de test și integrarea EDT

Logica de test, inclusiv integrarea EDT (Embedded Deterministic Test), este esențială pentru validarea și depanarea SoC-ului, iar EDT-ul permite testarea funcționalităților integrate în mod determinist, oferind un control precis asupra testelor efectuate.

Cu ajutorul modulului de EDT explicat în secțiunea 1.8, putem spori considerabil performanțele de testare ale modulului de SoC, aplicând conceptele teoretice în practică. Cele două module (SoC-ul și EDT-ul) sunt instanțiate într-un modul de TOP numit `SoC_Student_EDT` și integrate împreună pentru a facilita testarea deterministă și generarea de vectori de test folosind tehnici ATPG. Despre acest aspect se va vorbi mai mult în capitolul ce urmează.

Capitolul 3

Metodologie și implementare

În continuare, voi vorbi despre detecția defectelor de fabricație (de tip stuck-at - 1.3.2.1 și transition - 1.3.1.1) în circuitul digital explicat anterior la capitolul 2, unde se vor aplica tehnicile DFT și ATPG, se vor aplica vectori de test generați automat și se va evalua eficiența acestora.

În următoarea figură sunt reprezentați pașii care vor fi urmați în implementare pentru a obține un produs care să fie conform cerințelor clientului:

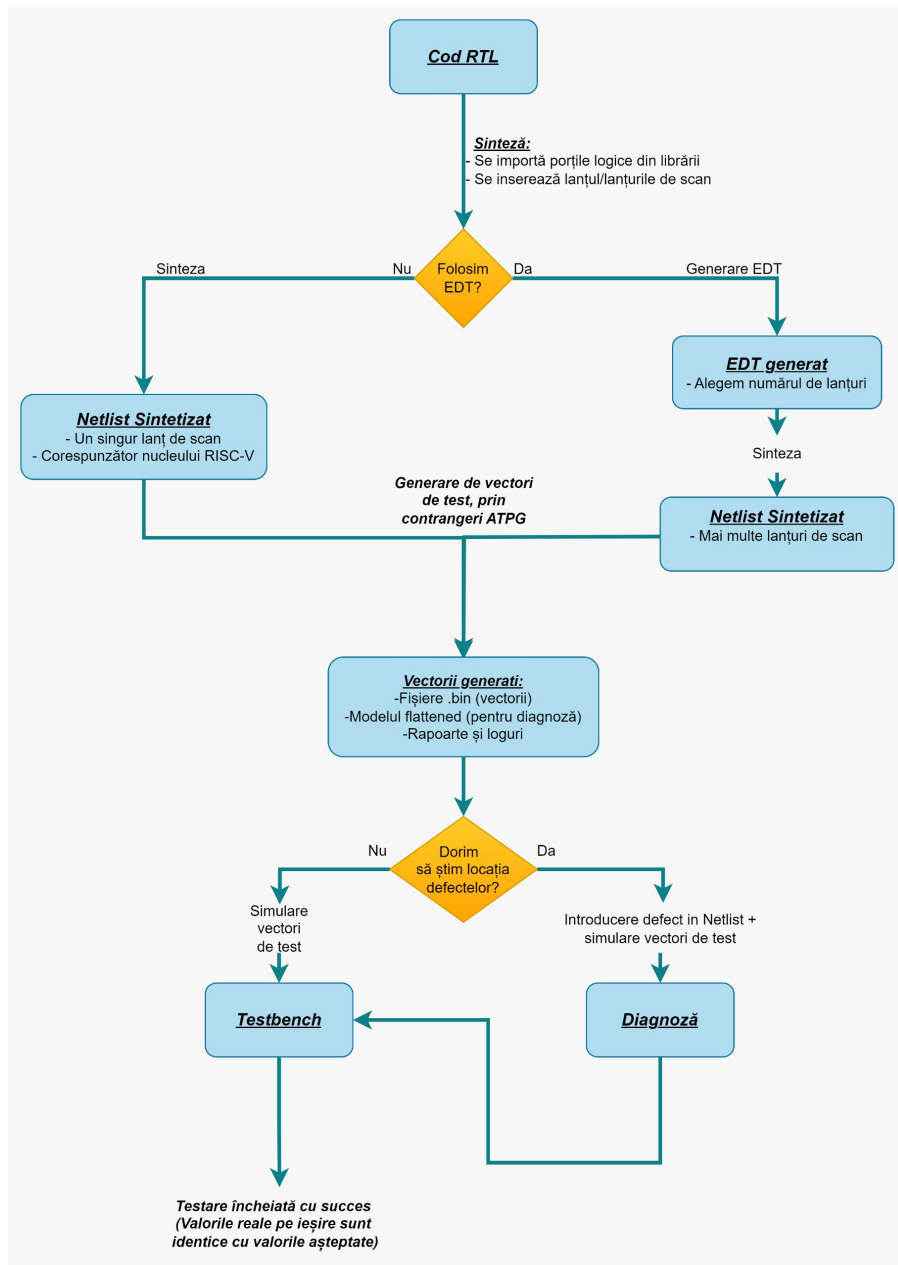


Figura 3.1: Procesul întreg de testare cu SCAN, ATPG și EDT

Mai mult decât atât, cu ajutorul modulului de EDT, vom testa mai multe cazuri posibile și vom vedea diferențele între acestea, cât și diferențe semnificative în performanță între cazurile cu și fără EDT.

Întâi de toate, pentru ca un chip să poată intra în producție, trebuie să existe o descriere a design-ului acestuia în limbaj HDL (Hardware Description Language) de la care se începe, după care urmează sinteza circuitului, inserarea logicii DFT, diagnoza (optional) și alte etape de testare pe care le vom discuta în cele ce urmează, la final având un produs gata de livrare pentru client.

Cum se poate vedea, se va porni de la un cod RTL (Verilog), căruia i se va aplica sinteza pentru a obține un Netlist în care este introdusă logica DFT (flip-flopii normali devin flip-flopi de Scan conectați în lanțuri de scan) și în care porțile logice sunt importate din librării specializate care permit vizualizarea întregului circuit în programele software aferente. În esență, acum netlistul specifică interconectarea componentelor logice la cel mai de jos nivel, cum ar fi porțile AND, OR și flip-flopuri de tip D.

Pe netlistul sintetizat la pasul anterior se vor genera automat vectori de test folosind tehnici și constrângeri ATPG, se vor aplica și simula acești vectori la intrarea testbench-ului corespunzător circuitului și se vor compara ieșirile rezultate cu cele așteptate pentru a avea un rezultat pozitiv sau negativ.

Generarea de vectori se va aplica atât pentru cazul cu un singur Scan Chain (ATPG normal), cât și pentru situația în care avem 30,40,50, respectiv 60 de scan chainuri, inserând EDT-ul (generat în prealabil) pentru a putea compresa datele de test și a avea lanțuri de lungime mult mai mică, datele fiind deplasate prin acestea în paralel, sporind performanțele.

Ultima etapă este una opțională și o reprezintă diagnoza, în care se va insera în mod intenționat un defect în circuit (Netlist) pentru a vedea dacă, simulând vectorii de test generați anterior pe noul netlist, defectul va fi sau nu depistat cu succes.

3.1 Generarea EDT-ului și crearea netlist-ului pentru modulul de top

Avantajul major al tehnologiei EDT (Embedded Deterministic Test) 1.8 este posibilitatea de a crește exponențial performanțele de generare de vectori de test prin adăugarea mai multor lanțuri de scan în circuit, contrar ATPG-ului normal în care putem avea un singur lanț de scan. Acest lucru ne permite deplasarea (shiftarea) datelor din patternurile de test trecute de decompresor (necomprimate) în paralel prin toate lanțurile, fiind apoi comprimate la ieșire în compactor, deplasate și interpretate de modulul de ATE. 1.8.4.2

În cele ce urmează se va discuta despre generarea și integrarea modulului de EDT cu un număr de lanțuri de scan dorit.

3.1.1 Configurări necesare

Înainte de a genera modulul de EDT corespunzător, va trebui să setăm anumite variabile și să se realizeze câteva configurări pentru a ajunge la rezultatul așteptat:

1. Se setează variabila ***number_of_chains*** cu numărul de lanțuri aferent (30,40,50,60) din scriptul bash numit `my_script_gen_edt` (regăsit în anexa de coduri)
2. Se modifică fișierul ***skeleton_input_file***, scheletul de la care se pornește întreaga generare a EDT-ului.
 - Se specifică pinul de intrare și ieșire pentru modul

- Se specifică parametrii semnalului/semnalelor de ceas (valoarea de început)
- Se specifică diferite date despre lanțurile de scan.

În cazul acestei lucrări există 4 configurări diferite: pentru 30,40,50,60 de lanțuri, scrise fiecare într-o linie de cod în mod separat. Voi explica o singură linie deoarece restul sunt sugestive. Să luăm spre exemplu **1 40 150 LE clk LE clk**:

- LE provine de la Leadine Edge și atestă faptul că semnalul de ceas, clk, este activ în 1 și inactiv în 0. Este trecut de 2 ori deoarece se vizează clock-ul de la prima celulă, respectiv ultima celulă din lanțurile de scan.
- Primele 3 cifre (1 40 150) reprezintă, în ordine, indicele primului lanț cu flip-flopii pe front pozitiv, indicele ultimului lanț și lungimea maximă admisă a unui lanț. Având în vedere că la cazul fără EDT, când există un singur lanț de scan, acesta are lungimea 5548 (3.5), dacă avem 40 de lanțuri logica ar spune că lungimea unui lanț ar fi în jur de 5548:40, adică 138. Însă întotdeauna se ia o marjă de eroare, setând lungimea maximă la un număr mai mare.

3. Se modifică fișierul `edt_ip.dofile` în funcție de câte lanțuri de scan dorim să introducem:

- Acesta ajută la crearea Netlistului pentru modulul de TOP și a conexiunilor compactorului, respectiv decompresorului.
- Se modifică parametrii lungimii minime, respectiv maxime a unui lanț de scan prin comanda ***set_edt_options -longest_chain_range min max***, unde în cazul de față pentru 40 de lanțuri, lungimea medie fiind 138, putem lua intervalul de la 110 la 160.
- Se setează conexiunile compactorului prin comanda ***set_compactor_connection -channel 1 -chains*** și se specifică conexiunea respectivă de la 1 la 40 prin ***chain_nr_conexiune*** (de exemplu `chain29`).
- Se modifică variabila `SCAN_CHAINS` din fișierul ***source/verilog/rtl/soc_student_edt.v*** (netlistul nesintetizat) pentru a putea pregăti numărul de conexiuni din modul ce urmează a fi conectate la EDT.

```

////////////////////////////////////
//Skeleton Design Input File
////////////////////////////////////
scan_chain_input  edt_si  bused
scan_chain_output edt_so  bused
begin_clocks
clk 0
end_clocks
begin_chains
//1 30 200 LE clk LE clk
1 40 150 LE clk LE clk »
//1 50 120 LE clk LE clk »
//1 60 100 LE clk LE clk
//1 70 90 LE clk LE clk
end_chains

```

```

1 40 150 LE clk LE clk
| | |
| | +-----> lungimea maxima a unui lant
| |
| | +-----> ultimul lant
|
+-----> primul lant cu flip-flopii pe front pozitiv

```

Figura 3.2: Detalii despre fișierul `skeleton_input_file` din cadrul EDT-ului

3.1.2 Etape pentru generare

Din moment ce s-au realizat cu succes configurările și setările preliminare, este timpul să se efectueze generarea EDT-ului și a netlistului cuprinzând EDT-ul și modulul RISC-V descris în capitolul 2. Pașii sunt următorii (script):

1. Pornind de la fișierul *skeleton_input_file* modificat corespunzător, folosind comanda *create_skeleton_design* și pasând ca parametru de intrare scheletul se returnează la ieșire 4 fișiere noi numite *dummy_design.v*, *dummy_design.atpglib*, *dummy_design.testproc* și *dummy_design.dofile*.

Acestea reprezintă fișiere intermediare care arată toate conexiunile realizate în design ținând seama de numărul lanțurilor de scan.

2. La pasul următor se execută fișierul *edt_ip.dofile*:

- Acesta citește librăria și netlistul generate la pasul anterior, după care apelează fișierul *dummy_design.dofile* pentru a seta constrângerile necesare pentru semnale și a forma legăturile pentru lanțurile de scan.
- Mai departe, prin comanda *set_edt_options -input_channels 1 -output_channels 1 -pipeline 0 -bypass_logic off -reset_signal asynchronous* se setează numărul de canale de intrare/ieșire, numărul de nivele de pipeline din EDT, logica de bypass și semnalul de reset asincron.
- Așa cum am văzut la subcapitolul anterior 3, se setează lungimile lanțurilor de scan, cât și conexiunile compactorului, decompresorului.
- Ultima comandă din fișier (*write_edt_files*) creează fișierele care implementează logica de EDT [25] în directorul gen/created. Prefixul -Verilog realizează scrierea logicii în format Verilog (.v), iar prefixul -nortl_prefix ajută la păstrarea numelui fișierului rezultat.

3. Ultimul pas este copierea fișierelor rezultate necesare utilizării ulterioare în proiect:

- Fișierul gen/created_edt.v se copiază în directorul sursă al proiectului (source/verilog/rtl/edt.v) și reprezintă netlistul pe care se va realiza procesul de sinteză în capitolul următor.
- Fișierul gen/created_edt.dofile definește numele de instanțe pentru decompresor, compactor, cât și pentru modulul care instanțiază decompresorul și compactorul.

De asemenea, se șterg restul constrângerilor de pini și semnale din fișier pentru a nu interfera mai târziu, păstrându-se doar cele legate de decompresor și compactor.

Acest fișier va fi folosit ulterior pentru generarea de vectori de test folosind ATPG, prin urmare este mutat în directorul corespunzător.

3.2 Sinteza circuitului

Etapa preliminară, înainte de generarea vectorilor de test, o reprezintă sinteza modulului de lucru, urmând ca la finalul acesteia să avem un netlist în care sunt introduse lanțul/lanțurile, împreună cu celulele de scan.

3.2.1 Cazul fără EDT (sinteză normală)

În acest caz, netlistul generat post-sinteză va conține un singur scan chain.

Procesul propriu-zis de sinteză se realizează cu un program numit Synopsys Design Compiler. Ca să funcționeze în mod corect, acesta trebuie să aibă în vedere numeroase fișiere de configurare, constrângeri pentru design, cât și librării specifice.

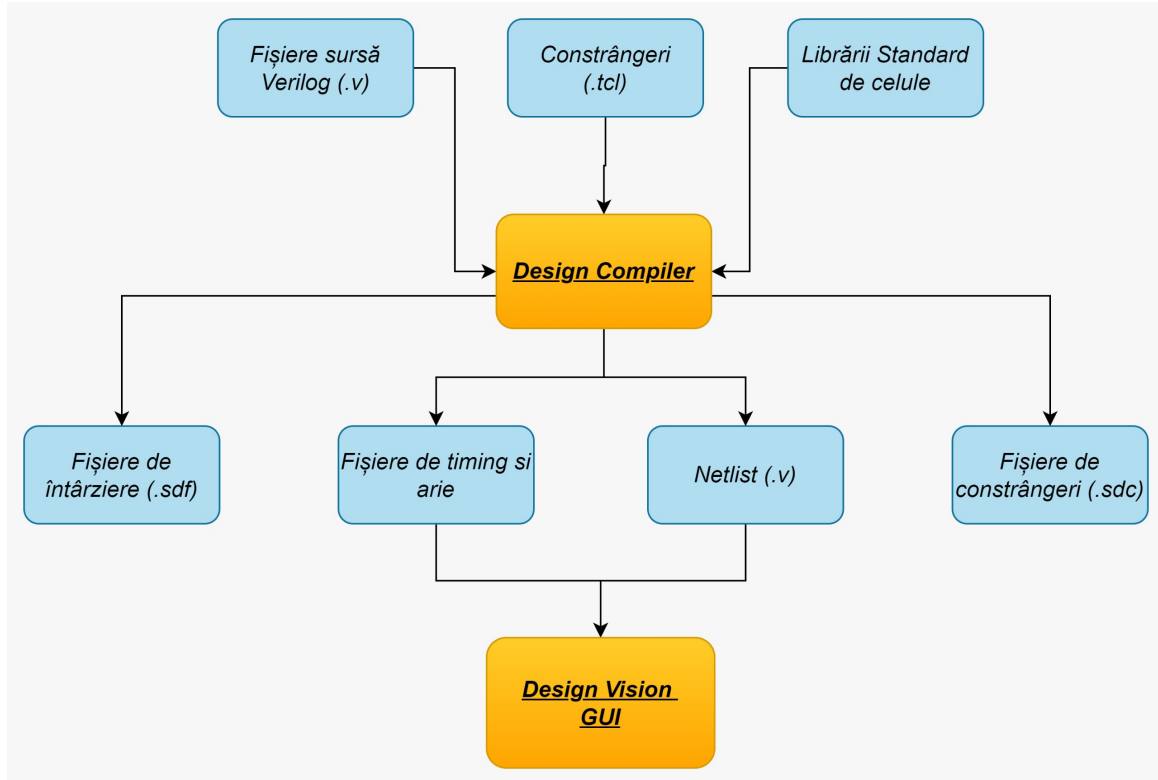


Figura 3.3: Legătura dintre fișierele de intrare și ieșire pentru programul Synopsys Design Compiler

Pentru ca flow-ul de sinteză să funcționeze corespunzător, directorul va trebui să conțină următoarele tipuri de fișiere:

- Librării/biblioteci
- Constrângeri
- Fișiere rtl
- fișiere Makefile:
 - Makefile.analyze
 - Makefile.dc
 - Makefile.elaborate
 - Makefile.libs
 - Makefile.setup
- scripturi de sinteză:
 - analyze.tcl

- analyze_libs.tcl
- compile.tcl
- elaborate.tcl
- fix.tcl
- insert_dft.tcl
- write_netlist.tcl
- dc_setup.tcl și dc_setup.local.tcl
- addons/soc_student.user_dft.tcl
- rezultate (results):
 - fișiere cu extensia .ddc (soc_student.compile.ddc, soc_student.elaborate.ddc, soc_student.insert_dft.tcl)
- rapoarte (reports):
 - fișiere cu extensia .rpt, de exemplu soc_student.fix.timing.rpt, dft_post.drc.rpt, dft_post.scanpath.rpt

Librăriile sunt plasate într-un director sau link-uite din alte directoare și sunt folosite de programul Design Compiler pentru a putea vizualiza rezultatul sub formă de porți logice interconectate.

Fișierele de tip RTL (Register-Transfer Language) se află într-un director format din fișierele sursă în cod rtl (.v) ale elementelor componente modului: axi_ram.v, soc.v, soc_student.v, gpio.v etc.

Constrângerile pe care i le dăm instrumentului de sinteză sunt tot sub formă de scripturi .tcl (Tool Command Language) și reprezintă constrângeri aplicate design-ului pentru a nu avea erori în procesul de sinteză, precum constrângeri de pini, voltaj sau de semnal de ceas (perioada, tranziția etc.).

Makefile-urile reprezintă fișiere speciale utilizate în principal pentru automatizarea procesului de compilare și gestionare a proiectelor. Ele sunt folosite de utilitarul make, care este un instrument de automatizare a construcției software-ului. Un makefile definește un set de reguli pentru compilarea și legarea unui program, specificând cum și când să recompileze părți ale unui program în funcție de modificările din fișierele sursă. [26]

Cu ajutorul makefile-urilor, utilizatorul poate compila doar fișierele care au fost modificate cel mai recent, ținând seama în același timp de fișierele dependente de cele actualizate. Acest lucru crește considerabil performanța comparând cazul în care utilizatorul ar compila manual fiecare fișier.

În sistemul de operare UNIX, un makefile conține trei tipuri de componente [27] [26]:

- Ținta (Target): Un fișier de ieșire pe care dorim să-l generăm, cum ar fi un fișier executabil.
- Dependente (Dependencies): Fișierele sursă sau alte fișiere intermediare necesare pentru a construi ținta, de exemplu versiuni mai noi pentru fișiere.
- reguli (comenzi): Comenzile de shell care trebuie executate pentru a genera ținta cu ajutorul dependențelor.

Luând în calcul conținutul fișierului `Makefile.dc`, putem da un exemplu concret pentru executarea unui `makefile`. Acest fișier conține țintele pentru toate scripturile din directorul `/scripts` (`analyze`, `analyze_libs`, `elaborate`, `explore`, `compile`, `insert_dft`, `fix`, `write_netlist`). Avantajul în acest caz este că utilizatorul poate executa orice țintă fără a fi îngrijorat de faptul că scripturile executate înaintea țintei respective sunt actualizate sau nu. Dacă sunt actualizate, programul Design Compiler își va da seama de acest lucru și va executa țintele de la prima modificată cel mai recent. De exemplu, dacă programatorul execută ținta `write_netlist` (ținta finală) însă scriptul `compile.tcl` prezintă modificări neexecutate în același timp, tool-ul va executa acea țintă prima oară, apoi toate țintele în ordinea corespunzătoare până la cea dorită. Acest lucru oferă o flexibilitate și o automatizare mult mai mare în execuția diferitelor părți din program.

De asemenea, este prezentă și sintaxa `.PHONY`. Aceasta este o directivă folosită pentru a marca anumite ținte ca fiind "false" (nu corespund unui fișier real), fiind utilă pentru a evita conflictele între țintele care au nume identice cu fișierele existente în sistemul de fișiere. Directiva `.PHONY` ajută la clarificarea `makefile`-ului, indicând explicit că anumite ținte sunt doar comenzi și nu fișiere ce trebuie generate, iar dacă se specifică un fișier al cărui nume nu există, `makefile`-ul va rula indiferent acea țintă.

Exact cum am referit ordinea scripturilor de sinteză, putem defini procesul ca urmând următorii pași:

1. **Analyze_libs:**

- Acest script este similar cu `analyze.tcl`, dar este folosit pentru a analiza librăriile necesare pentru proiect. Aceste librării pot conține componente standard necesare sintetizării codului RTL.

2. **Analyze:**

- Acest script este utilizat pentru analiza fișierelor sursă HDL. Analiza înseamnă că tool-ul de sinteză va verifica sintaxa și semantica codului HDL pentru a se asigura că nu există erori înainte de a continua cu procesul de sinteză.

3. **Elaborate:**

- Acest script gestionează procesul de elaborare, unde design-ul descris la nivel RTL (Register-Transfer Level) este transformat într-un format recunoscut de instrumentul Design Compiler, pregătit pentru sinteză.

4. **Compile:**

- Acest script este utilizat pentru a compila codul HDL analizat. Compilarea transformă descrierea comportamentală a designului într-o reprezentare structurală cu ajutorul celulelor din librăriile utilizate
- Permite optimizarea design-ului pornind de la constrângerile specificate.

5. **Insert_dft:**

- Scriptul `insert_dft` este folosit pentru inserarea unuia sau a mai multor lanțuri de scan în circuit, cât și pentru transformarea flip-flopilor în elemente de scan corespunzătoare.

6. **Fix:**

- Acest pas nu este unul obligatoriu, însă în esență reprezintă o recompilare a designului pentru a avea garanția că totul a fost compilat.

- De asemenea, prezintă reverificări de sintaxă.

7. Write_netlist:

- Acest script are ca scop generarea de fișiere necesare pentru utilizare ulterioară. De exemplu, se generează fișierul `soc_student.v` pe care îl vom folosi ca și netlist de intrare în fișierele de generare de vectori de test, și alte fișiere folositoare (`.rpt`, `.ddc`, `.sdc`).

3.2.1.1 Inserția logicii de SCAN

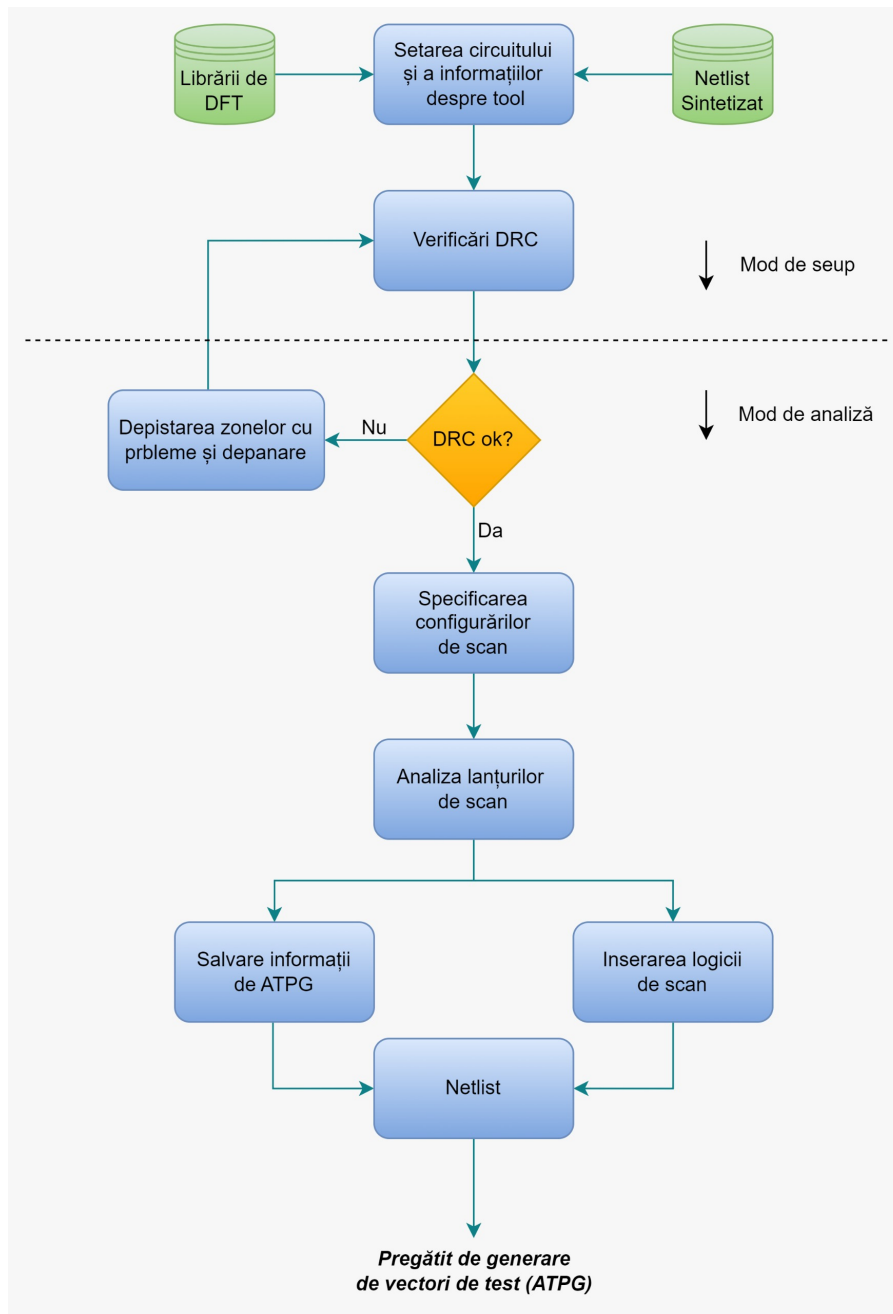


Figura 3.4: Diagrama pentru pașii de inserție a logicii scan în cadrul sintezei

În continuare vom vorbi despre pasul cel mai important din sinteza modulului nostru de SoC, și anume inserarea lanturilor de scan și transformarea flip-flopilor. Acestea se realizează la

pasul de `insert_dft`, ținând cont de constrângerile și rezultatele de la pașii anteriori. Se apelează fișierul `dc_setup.tcl`, fișier principal pentru procesul de DC (Design Compiler), care "sursează" la randul lui scriptul `dc_setup.local.tcl`, ceea ce înseamnă că respectivele comenzi din script pot fi rulate și compilate ca și cum ar exista în fișierul principal.

De asemenea, înainte de inserția elementelor DFT trebuie făcute anumite configurări pentru ca aceasta să fie realizată corespunzător, aflate în scriptul `/adons/soc_student.user_dft.tcl`:

- `set test_point_insertion "false"` și `set wrapper_insertion "false"` se folosesc pentru a nu se insera puncte de test în circuit și lanțuri wrapper (formate din componente aproape de frontiere conectate între ele), permițând inserarea pură de lanț/lanțuri de scan
- `set_dft_insertion_configuration -preserve_design_name true` permite păstrarea numelui designului inițial chiar și după executarea întregii sinteze, usurând astfel munca.
- `set SCAN_CHAINS 1` reprezintă variabila setată care manipulează numărul de lanțuri de scan care se doresc introduse în design, în acest caz fără EDT fiind 1.
- `set_scan_configuration -insert_terminal_lockup false` nu permite inserția latch-urilor pentru sincronizare la finalul lanțurilor de scan.
- `set_scan_configuration -clock_mixing no_mix` permite flexibilitatea lanțurilor de scan și separarea acestora.

Dacă shell-ul este în modul topografic (comenzile sunt date cu `-topo` la final), atunci în continuare se vor crea noi semnale și porturi pe care le vom apela în protocolul de test creat în curând și pe care le vom folosi în continuare în procesul de testare, cu următoarele comenzi:

- `create_port` creează un port dacă acesta nu era deja creat în netlistul inițial dat ca intrare în procesul de sinteză
 - Argumentul din continuare `"-direction in"` reprezintă faptul că vom crea portul ca și input în circuit, iar apoi îi vom seta locația cu ajutorul comenzii `set_port_coordinate x, y`.
 - Această comandă se folosește pentru a crea porturile folosite pentru scan: `scan_enable.i`, `scanmode.i` și `scan_enable_cg.i`.
- `set_dft_signal` se folosește pentru a configura parametrii anumitor semnale de interes.
 - Argumentul `-view existing_dft` se folosește pentru semnale care deja au conexiunea legată în design, dar trebuie percepute de tool în modul configurat de noi.
De exemplu, comanda `"set_dft_signal -view existing_dft -type ScanClock -timing 45 55 -port clk.i"` asignează portului `clk.i` din netlist deja existent semnalul de ceas ScanClock în care frontul pozitiv să se afle la 45ns distanță de începutul perioadei de ceas, iar frontul negativ la distanța de 55ns.
Alte semnale configurate la acest moment sunt semnalul de Reset asignat portului `rst_n.i` cu valoare activă în 0, semnalele ScanEnable și TestMode asignate porturilor deja create anterior (`scan_enable.i`, `scanmode.i` și `scan_enable_cg.i`) cu valoare activă în 1
 - Argumentul `-view spec` se folosește atunci când dorim ca semnalul respectiv să fie conectat în design la portul respectiv prin comanda `insert_dft`.
Spre exemplu, semnalele ScanEnable și TestMode sunt create cu această comandă, folosind argumentul `-usage scan` și `-usage clock_gating`.

În același timp, având un singur scan chain, este nevoie să se creeze o singură intrare și o singură ieșire de scan, anume semnalele ScanDataIn și ScanDataOut, asigurate porturilor test_sil și test_sol.

- Tot aici se creează protocolul de test prin comanda create_test_protocol

Având aceste configurări la dispoziție, în fișierul insert_dft.tcl se sursează scriptul dc_setup.tcl și se introduce lanțul sau lanțurile de scan aferente. De asemenea se testează regulile DRC (Design Rule Check) și se scriu diferite rapoarte (.rpt). Deocamdată ne ajută doar raportul soc_student.dft_post.scanpath.rpt care ne arată exact informațiile despre lanțul de scan inserat în circuit, așa cum putem vedea în figura următoare:

```
=====
AS BUILT BY insert_dft
=====
```

Scan_path	Len	ScanDataIn	ScanDataOut	ScanEnable	MasterClock	SlaveClock
I 1	5548	test_sil	test_sol	scan_enable_i	clk_i	-

```
=====
TEST MODE: Internal_scan
VIEW      : Existing DFT
=====
```

```
=====
AS SPECIFIED BY USER
=====
```

```
=====
AS BUILT BY insert_dft
=====
```

Scan_path	Cell #	Instance_name	Clocks
I 1	0	u_axi_ram/read_addr_reg_reg[2]	clk_i
	1	u_axi_ram/read_addr_reg_reg[3]	
	2	u_axi_ram/read_addr_reg_reg[4]	
	3	u_axi_ram/read_addr_reg_reg[5]	
	4	u_axi_ram/read_addr_reg_reg[6]	
	5	u_axi_ram/read_addr_reg_reg[7]	
	6	u_axi_ram/read_addr_reg_reg[8]	
	7	u_axi_ram/read_addr_reg_reg[9]	
	8	u_axi_ram/read_addr_reg_reg[10]	
	9	u_axi_ram/read_addr_reg_reg[11]	
	10	u_axi_ram/read_addr_reg_reg[12]	
	11	u_axi_ram/read_addr_reg_reg[13]	
	12	u_axi_ram/read_addr_reg_reg[14]	
	13	u_axi_ram/read_addr_reg_reg[15]	
	14	u_axi_ram/read_addr_reg_reg[16]	
	15	u_axi_ram/read_addr_reg_reg[17]	
	16	u_axi_ram/read_addr_reg_reg[18]	
	17	u_axi_ram/read_addr_reg_reg[19]	
	18	u_axi_ram/read_addr_reg_reg[20]	
	19	u_axi_ram/read_addr_reg_reg[21]	
	20	u_axi_ram/read_addr_reg_reg[22]	
	21	u_axi_ram/read_addr_reg_reg[23]	

Figura 3.5: Raportul privind lungimea, intrarea, ieșirea și elementele lanțului de scan inserat

3.2.2 Cazul cu EDT (mai multe lanțuri de scan)

Situația discutată anterior a fost cea în care design-ul este în starea inițială, fără modificări din partea EDT-ului. De data aceasta sinteza și inserția DFT se realizează pe modulul de top format din instanțierea nucleului original și a modulului de EDT într-un modul de top.

```
set_dft_connect EX_EDT_CG -type clock_gating_control -source scan_enable_cg_i -exclude [get_cells -hier -filter clock_gating_logic==true&&scan_element==false]
remove_cell [get_cells -hierarchical -filter full_name==scan_chains_inv_i*]

for {set i 0} { $i < $SCAN_CHAINS } {incr i} {
    set_dft_signal -view spec -type ScanDataIn -hookup_pin edt_i/edt_scan_in[$i]
    set_dft_signal -view spec -type ScanDataOut -hookup_pin edt_i/edt_scan_out[$i]
}
}
```

Figura 3.6: Modificări în cod pentru inserția Scan cu EDT

Configurările pentru permiterea înlanțuirilor de scan sunt în mare parte aceleași în comparație cu cazul fără EDT, cu mici excepții:

- Stabilirea unui nou semnal de clock-gate
 - Așa cum se poate vedea în figura 3.6 se definește și stabilește conexiunea unui nou semnal de testare în design cu comanda `set_dft_connect`
 - Argumentul `-type clock_gating_control` specifică tipul de conexiune, în acest caz reprezentând control pentru un clock-gate.
 - `-source scan_enable_cg_i` specifică sursa conexiunii, în acest caz este semnalul `scan_enable_cg_i`
 - `-exclude [get_cells -hier -filter clock_gating_logic==true && scan_element==false]` realizează excluderea celulelor care nu sunt elemente de scan, dar care prezintă logica internă de clock-gate.
- Comanda `remove_cell [get_cells -hierarchical -filter full_name= scan_chains_inv_i*]`
 - Aceasta elimină anumite celule specifice din ierarhia designului, ale căror nume se potrivește cu un anumit tipar.
- În continuare avem o instrucțiune "for" așa cum se regăsește în figura 3.6, în care există 2 vectori: `edt_scan_in` și `edt_scan_out` care reprezintă canalele de scan pentru EDT.
 - Dimensiunea acestora va fi dată de numărul de lanțuri de scan inserate în design, cu ajutorul variabilei `SCAN_CHAINS`
 - Argumentul `-hookup_pin` stabilește conexiunea între semnalele definite (`ScanDataIn` și `ScanDataOut`) și pinul specific de intrare/ieșire, folosindu-ne de elementele indexate din vectori.
 - Porturile create (pinii) sunt setați apoi ca porturi de intrare sau ieșire, prin "direction in/out"
- În acest exemplu am setat variabila `SCAN_CHAINS` ca fiind 70, ceea ce înseamnă că nu va mai fi un singur lanț de 5548 de elemente, ci 70 de lanțuri, fiecare a câte 80 sau 79 de celule, în total reieșind în jur de 5548, așa cum se vede în figura 3.7.

Scan_path	Len	ScanDataIn	ScanDataOut	ScanEnable	MasterClock	SlaveClock
I 1	80	edt_i/edt_scan_in[0]	edt_i/edt_scan_out[0]	scan_enable_i	clk_i	-
I 2	80	edt_i/edt_scan_in[1]	edt_i/edt_scan_out[1]	scan_enable_i	clk_i	-
I 3	80	edt_i/edt_scan_in[2]	edt_i/edt_scan_out[2]	scan_enable_i	clk_i	-
I 4	80	edt_i/edt_scan_in[3]	edt_i/edt_scan_out[3]	scan_enable_i	clk_i	-
I 5	80	edt_i/edt_scan_in[4]	edt_i/edt_scan_out[4]	scan_enable_i	clk_i	-
I 6	80	edt_i/edt_scan_in[5]	edt_i/edt_scan_out[5]	scan_enable_i	clk_i	-
I 7	80	edt_i/edt_scan_in[6]	edt_i/edt_scan_out[6]	scan_enable_i	clk_i	-
I 8	80	edt_i/edt_scan_in[7]	edt_i/edt_scan_out[7]	scan_enable_i	clk_i	-
I 9	80	edt_i/edt_scan_in[8]	edt_i/edt_scan_out[8]	scan_enable_i	clk_i	-
I 10	80	edt_i/edt_scan_in[9]	edt_i/edt_scan_out[9]	scan_enable_i	clk_i	-
I 11	80	edt_i/edt_scan_in[10]	edt_i/edt_scan_out[10]	scan_enable_i	clk_i	-
I 12	80	edt_i/edt_scan_in[11]	edt_i/edt_scan_out[11]	scan_enable_i	clk_i	-
I 13	80	edt_i/edt_scan_in[12]	edt_i/edt_scan_out[12]	scan_enable_i	clk_i	-
I 14	80	edt_i/edt_scan_in[13]	edt_i/edt_scan_out[13]	scan_enable_i	clk_i	-
I 15	80	edt_i/edt_scan_in[14]	edt_i/edt_scan_out[14]	scan_enable_i	clk_i	-
I 16	80	edt_i/edt_scan_in[15]	edt_i/edt_scan_out[15]	scan_enable_i	clk_i	-
I 17	80	edt_i/edt_scan_in[16]	edt_i/edt_scan_out[16]	scan_enable_i	clk_i	-
I 18	80	edt_i/edt_scan_in[17]	edt_i/edt_scan_out[17]	scan_enable_i	clk_i	-
I 19	79	edt_i/edt_scan_in[18]	edt_i/edt_scan_out[18]	scan_enable_i	clk_i	-
I 20	79	edt_i/edt_scan_in[19]	edt_i/edt_scan_out[19]	scan_enable_i	clk_i	-
I 21	79	edt_i/edt_scan_in[20]	edt_i/edt_scan_out[20]	scan_enable_i	clk_i	-
I 22	79	edt_i/edt_scan_in[21]	edt_i/edt_scan_out[21]	scan_enable_i	clk_i	-
I 23	79	edt_i/edt_scan_in[22]	edt_i/edt_scan_out[22]	scan_enable_i	clk_i	-
I 24	79	edt_i/edt_scan_in[23]	edt_i/edt_scan_out[23]	scan_enable_i	clk_i	-
I 25	79	edt_i/edt_scan_in[24]	edt_i/edt_scan_out[24]	scan_enable_i	clk_i	-
I 26	79	edt_i/edt_scan_in[25]	edt_i/edt_scan_out[25]	scan_enable_i	clk_i	-
I 27	79	edt_i/edt_scan_in[26]	edt_i/edt_scan_out[26]	scan_enable_i	clk_i	-

Figura 3.7: Vizualizare a câtorva din cele 70 de lanțuri de scan de dimensiune 80 sau 79

3.3 Generarea vectorilor de test pentru defecte de tip STUCK-AT și TRANSITION

În acest capitol se va discuta despre generarea automată a vectorilor de test pe netlistul creat la punctul anterior, folosind tehnici ATPG și fișiere de tip .dofile. Procesul propriu-zis este reprezentat în figura următoare:

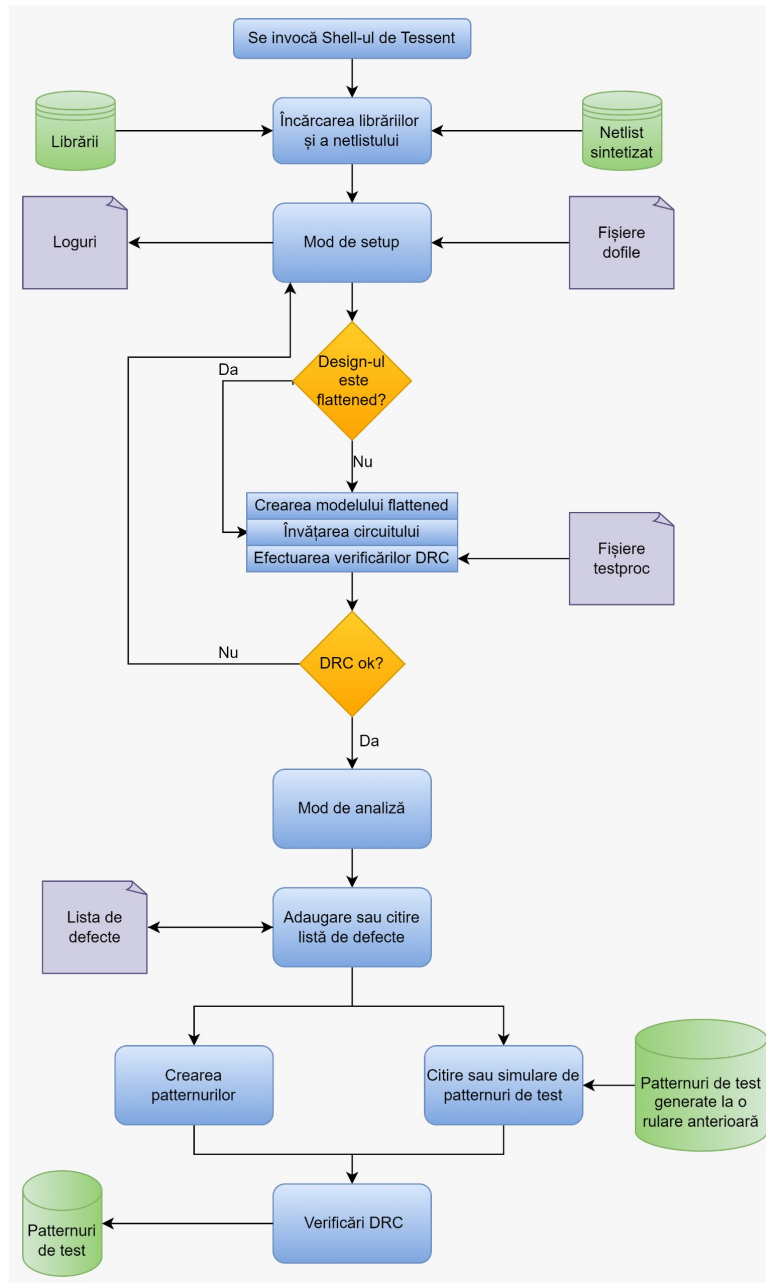


Figura 3.8: Descrierea procesului de ATPG

Un fișier dofile este un tip de fișier utilizat în cadrul unor medii de dezvoltare software sau de simulare pentru a stoca o serie de comenzi scriptate sau instrucțiuni care pot fi executate în mod automat. Acestea se pot interapela unul în cadrul celuilalt pentru o mai bună structurare a fișierelor și pentru a avea codul cât mai curat.

De asemenea, ne vom folosi și de fișiere de tip .testproc, utilizate în industria electronică și a semiconducătorilor pentru a defini și controla proceduri de testare specificate pentru un anumit design de circuit integrat sau chip. Acestea prezintă configurări de semnale și proceduri

specifice pentru a detecta anumite tipuri de defecte.

Pentru început tool-ul va trebui să încarce librăriile și netlistul sintetizat necesar generării vectorilor de test. În continuare se intră în modul de setup în care se aplică fișierele de tip dofile circuitului și se generează logurile aferente. Se verifică dacă design-ul este sau nu "flattened". Dacă nu este, se creează un model "flattened" pentru design-ul nostru, în caz contrar se poate începe învățarea circuitului.

La pasul următor se efectuează verificările de design cu ajutorul procedurilor de test pentru a vedea dacă acesta este pregătit pentru generarea vectorilor de test. Dacă acesta nu este pregătit se revine la pasul de setup, altfel se poate trece la pasul de analiză. Acest pas reprezintă generarea și simularea propriu-zisă de vectori de test, unde se dă o lista de defecte stuck-at sau transition (optional), se aplică ultimele constrângeri design-ului (defectele respective introduse în design) și se verifică integritatea patternurilor. Dacă aceștia detectează cu succes în simulare defectele vizate, sunt salvați pentru utilizare ulterioară.

3.3.1 Cazul fără EDT (ATPG normal)

Figura următoare reprezintă ierarhia și interapelarea fișierelor dofile și testproc pentru generarea de vectori de test vizând defecte de tip stuck-at și transition.

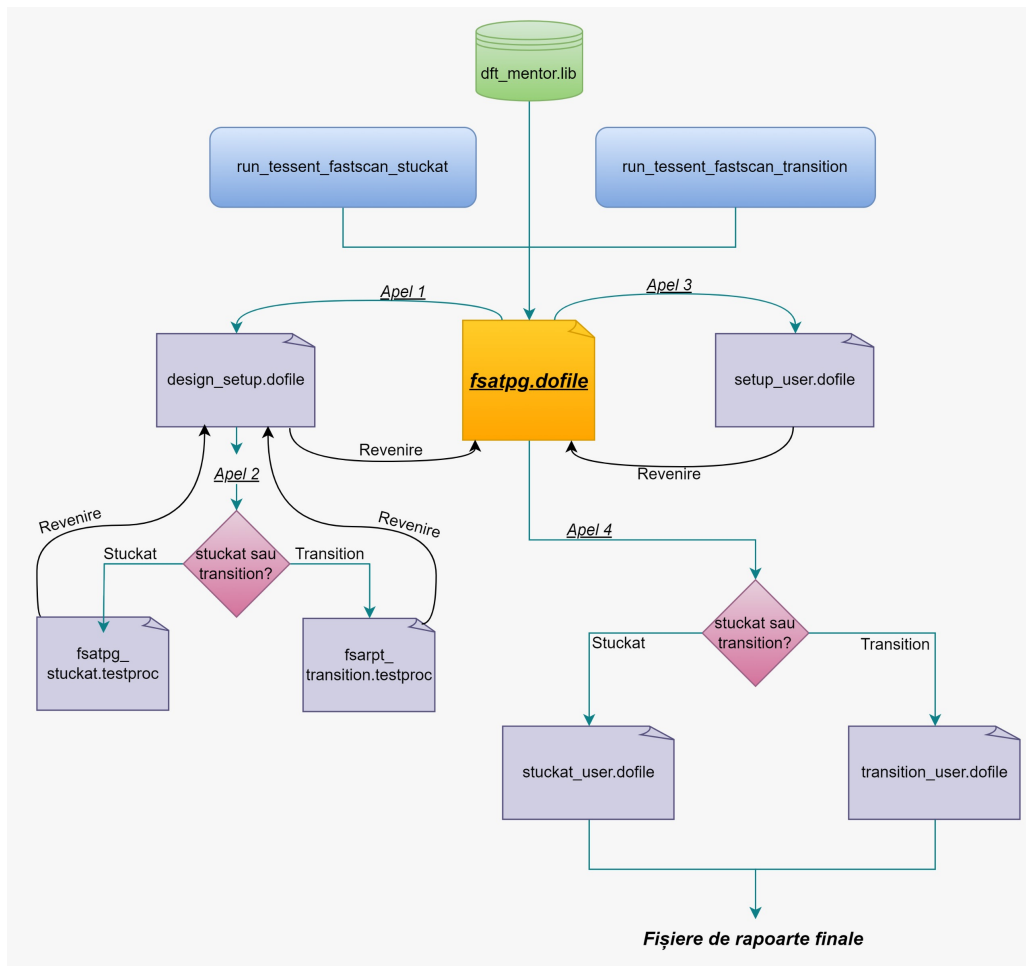


Figura 3.9: Ierarhizarea și interapelarea fișierelor pentru ATPG

3.3.1.1 Generarea defectelor de tip Stuck-at

Pentru defectele de tip stuck-at, se realizează flow-ul următor

1. Prima oară se rulează în terminal fișierul executabil `run_tessent_fastscan_stuckat`

- Se setează două variabile globale (`TESENT_EXECUTE_STUCKAT`, respectiv `TESENT_EXECUTE_TRANSITION`) care vor fi apelate ulterior în fișierele următoare pentru a ne da seama ce tip de defecte dorim să detectăm
Comanda `setenv` se folosește pentru a seta variabile globale (de environment), cea de `stuckat` va fi setată cu 1, iar cea `transition` cu 0
- Se încarcă în sistem modulul `dft-mentor` necesar rulării fișierelor de tip `.dofile` în Shell-ul de Tessent
- Comanda `batch tessent -shell -dofile ./fsatpg.dofile -logfile logs/fsatpg_run_stuckat.log replace` este o comandă specifică tool-ului de Tessent care permite executarea dofile-urilor.

Se apelează fișierul principal `fsatpg.dofile`, reținând informațiile de rulare într-un log.

2. Fișierul ***fsatpg.dofile*** reprezintă fișierul principal pentru generarea vectorilor de test, restul fișierelor fiind apelate de acesta, urmând ca după terminarea lor să se revină la execuția acestuia până la final.

- Comenzile `set_TOPMODULE soc_student` și `set OUTPUTFILE $TOPMODULE` setează variabilele necesare pentru design.
- `set_context patterns -scan` specifică în ce context vor fi utilizate comenzile următoare: dacă EDT-ul este folosit, se va utiliza `TestKompress.1.8.4` [25]
- `read_cell_library` și `read_verilog` se folosesc pentru încărcarea fișierului cu librăriile aferente modului și încărcarea netlistului sintetizat la pasul anterior.
- `set_current_design` specifică designul utilizat pentru tot ce se va apela în continuare, în cazul nostru acesta este `soc_student`.
- Prin comanda `"dofile setup_user.dofile` se invocă respectivul fișier.

3. `setup_user.dofile`:

- Acesta stabilește anumite setări globale de environment pentru configurarea procesului.
- Urmează setarea variabilelor `ADVANCED_POWER_REPORT`, `HIGH_EFFORT_ATPG`, `EXECUTE_STUCKAT`, `EXECUTE_TRANSITION` pe OFF (valoare 0) pentru a porni cu o rulare cât mai curată, iar dacă este nevoie de acestea, ele vor fi setate 1 mai târziu.
- Se verifică existența variabilelor globale setate anterior și, în funcție de ce defecte dorim să testăm, una din ele devine 1 (ON)
- Comanda `set_internal_fault off` permite neluarea în considerare a defectelor interne pentru fiecare celulă.
- `set_contention_check on -atpg` înseamnă că, în contextul ATPG, se vor verifica conflictele de date trimise pe magistrale comune.
- ***set_power_metrics -shift on -capture on -composite_shift_power on*** permite calcularea puterii disipate și a activității pentru operațiile de captură și shiftare de date (parcursarea secvențială a datelor în lanțurile de scan).
- După acestea se revine înapoi la fișierul `fsatpg.dofile`

4. În continuare, în cadrul execuției `fsatpg.dofile` se apelează fișierul `design_setup.dofile`:

- Acesta desemnează setări pentru modelul nostru comportamental, cum ar fi setarea de semnale (clock și reset) și de constrangeri pentru semnale (scanmode.i sa fie constrâns în 1 la început).
 - În funcție de tipul de defect pe care dorim să fie detectat, se definește grupul de scan corespunzător apelat de fișierul testproc.
 - Se definește apoi lanțul de scan cu intrarea test_si1, ieșirea test_so1 și parametrii, respectiv procedurile definite în fișierul testproc.
5. Se revine în dofile-ul principal, în care se verifică DFT DRC-urile (Regulile de Design) și, dacă totul funcționează corespunzător și nu există erori, se intră în modul de analiză prin comanda ***set_system_mode analysis***
 6. Valorificând tipul de defect dorit a fi detectat, se generează diferite rapoarte pentru regulile de Design, lanțurile și celulele de scan, semnalele de ceas și contrângerile de pini. Acestea se salvează fie în directorul ***/reports/drc/drc_stuckat***, fie în ***/reports/drc/-drc_transition*** (în cazul nostru în directorul pentru stuckat).
 7. Ultimul pas înainte de final și generarea rapoartelor finale de ATPG este apelarea fișierului ***stuckat_user.dofile/transition_user.dofile***, unde se realizează setări despre modul de generare al patternurilor.

Vorbim în continuare despre cele mai importante fișiere din flow-ul de ATPG, și anume ***fsatpg_stuckat.testproc*** și ***stuckat_user.dofile***.

Fișierul ***fsatpg_stuckat.testproc*** are rolul de a seta parametrii pentru semnale și pentru procedurile utilizate în lanțul de scan. La început se specifică unitatea de timp de baza (1ns) și strobe_window-ul de 0 (întârzierea). Se creează 4 "timeplateuri" (toate sunt identice în acest caz, dar cu nume diferit pentru a face separația procedurilor). Un timeplate reprezintă un template pentru unitatea de timp (o perioadă) folosită pentru datele din scan chain (time + template = timeplate). Sintaxa este următoarea:

```
timeplate defaultTP =
force_pi          0;
measure_po        35;
pulse clk_i       45 15;
pulse rst_n_i     45 15;
period           100;
end;
```

Prin urmare, acest timeplate definește următoarele:

- ***force_pi 0*** forțează valoarea 0 pe toate intrările primare ale circuitului.
- ***measure_po*** măsoară valoarea de pe toate ieșirile primare ale circuitului.
- ***pulse clk_i*** și ***pulse rst_n_i 45 15*** setează valorile de timing ale semnalelor de ceas și reset.
 - Prima valoare (45) reprezintă numărul de ns de la începutul perioadei unde semnalul trece din valoarea 0 în 1. 3.10
 - A doua valoare (15) reprezintă durata în ns a frontului pozitiv al semnalului, până acesta trece înapoi în 0. 3.10

- Perioada unui puls de scan de 100ns. 3.10

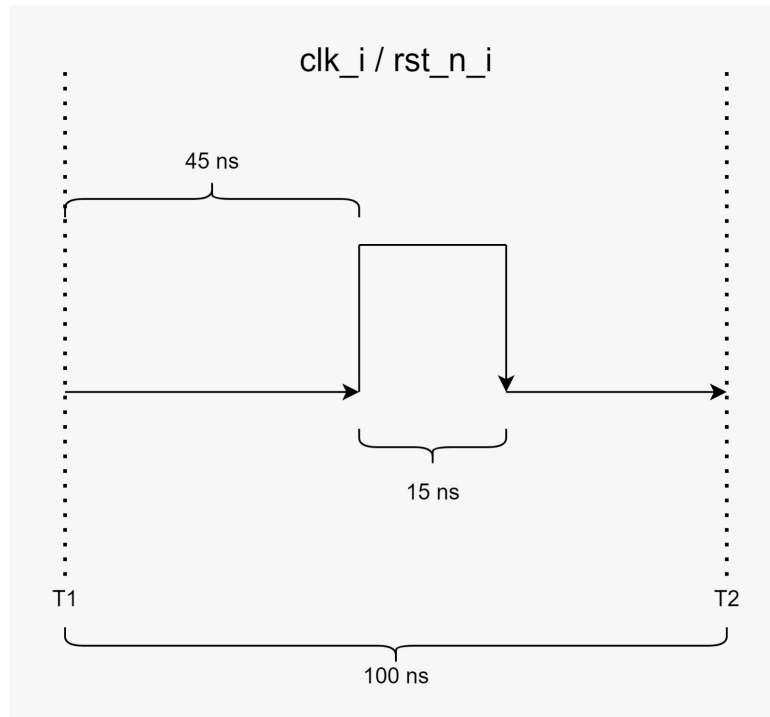


Figura 3.10: Reprezentare a configurării semnalului de ceas în timeplate

Pentru a ușura puțin procesul, putem defini o subprocedură în cadrul unei proceduri, pentru a evita repetiția aceluiași linii de cod de mai multe ori în cadrul aceleiași proceduri (analog funcțiilor dintr-o clasă care sunt apelate și în alte clase de mai multe ori). Sintaxa este următoarea:

```
procedure sub_procedure scan_clk_pulses =
timeplate setupTP;
cycle =
pulse clk_i;
end;
end;
```

Aceasta se folosește de timeplate-ul definit de noi anterior pentru a forma o subprocedură în care se obține un puls de ceas (necesar shiftării datelor). În continuare se definesc procedurile de **test_setup**, **shift** și **load_unload**.

Procedura de **test_setup** se folosește pentru a testa integritatea semnalelor de clock și reset, forțând câteva pulsuri înainte de intrarea în modul de test (procedură opțională).

Procedura de **shift** se folosește pentru a defini operația de deplasare (shiftare) a datelor în interiorul lanțului de scan cu o poziție.

Aceasta prezintă următoarea sintaxă:

```

procedure shift =
timeplate shiftTP;
scan_group group1 ;
cycle =
force_sci;
force rst_n_i 1;
force scan_enable_i 1;
measure_sco;
pulse clk_i;
end;
end;

```

Fiind procedură de shiftare, se folosește de timeplate-ul de shiftare pentru configurările de timing ale semnalelor și de grupul de scan definit în fișierul anterior. Ca și mod de funcționare al ciclului de shiftare a datelor în procedură, acesta este următorul:

- ***force_sci*** forțează valori pe intrarea de scan din circuit.
- ***force rst_n_i 1*** forțează valoarea de reset în 1, ca să nu existe un reset neașteptat pe parcursul deplasării datelor, iar acestea să se piardă.
- ***force scan_enable_i 1*** forțează valoarea de pe scan_enable în 1 în timpul deplasării datelor. Acesta este semnalul care ne arată că ne aflăm în modul de shift sau în modul de capture.
- ***measure_sco***; măsoară valoarea de pe ieșirea de scan.
- ***pulse clk_i***; realizează un puls de ceas pentru a se trece la următoarea poziție în lanț.

Următoarea procedură este cea de ***load_unload***, cu definiția:

```

scan_group group1;
timeplate loadUnloadTP;
cycle =
force rst_n_i 1;
force scan_enable_cg_i 1 ;
force scan_enable_i 1 ;
end ;
apply shift 3408;
end;

```

Aceasta definește operația de descărcare de date la ieșire prin tot lanțul de scan, iar semnalele de ***scan_enable***, ***scan_enable_cg_i*** și ***rst_n_i*** trebuie forțate în 1 deoarece datele sunt shiftate prin procedura de shiftare până când ultima valoarea este capturată la ieșire. În cadrul comenzii ***apply shift 3408*** nu contează ce număr folosim atât timp cât întreg lanțul de scan este descărcat la ieșire.

În restul fișierului testproc avem definite 3 tipuri de proceduri, numite named capture procedures (NCP). Acestea sunt cele mai importante și determină modul în care capturăm datele pentru a avea test coverage cât mai mare.

Ele se clasifică în:

- Proceduri pentru patternuri elementare:

- Procedura de captură ***capt_scan_reset_n***: Se forțează semnalul de ceas în 0 și se realizează un puls de reset, în timp ce se forțează valori pe intrarea primară și se masoară valoarea de pe ieșirea primară.
 - Procedura de captură ***capt_basic_clk***: Se forțează semnalul de reset în 1 și se realizează un puls de ceas, în timp ce se forțează valori pe intrarea primară și se masoară valoarea de pe ieșirea primară.
- Proceduri secvențiale:
 - Procedura de captură ***capt_seq_2clk***: Se realizează 2 cicluri secvențiale în care se pulsează semnalul de ceas, exact ca în procedura ***capt_basic_clk***: doar că se forțează valoarea pe intrări la începutul primului ciclu și se capturează valoarea la ieșire la sfârșitul ultimului ciclu.
Chiar dacă nu pare relevant, în așa fel putem genera alți vectori de test, reușind astfel să mai depistăm defecte și să avem coverage cât mai mare.
 - Procedura de captură ***capt_seq_3clk***: Analog versiunii cu 2 cicluri de ceas doar că aici sunt 3 cicluri secvențiale.
 - Procedura de captură ***capt_seq_4clk***: Analog versiunii cu 2 cicluri de ceas doar că aici sunt 4 cicluri secvențiale.
 - Proceduri multiple-load (se încarcă date la ieșire de mai multe ori, fiind opționale):
 - Procedura de captură ***capt_mul_2pat_1clk***: Procedura în care se folosesc 2 cicluri: primul la viteză normală, iar restul la viteză scăzută (folosind cycle slow) pentru a fi siguri că vom cuprinde defectul. În toate ciclurile se forțează valori pe intrări, însă numai pe ultimul se masoară valori pe ieșire.
 - Procedura de captură ***capt_mul_3pat_1clk***: La fel ca și procedura ***capt_mul_2pat_1clk***, doar că avem 3 cicluri: primul normal, iar următoarele 2 slow. 3.11
 - Procedura de captură ***capt_mul_4pat_1clk***: La fel ca și procedura ***capt_mul_2pat_1clk***, doar că avem 3 cicluri: primul normal, iar următoarele 3 slow.
 - Procedura de captură ***capt_mul_2pat_2clk***: La fel ca și procedura ***capt_mul_2pat_1clk***, doar că avem 2 perechi de cicluri, fiecare măsurând valoarea de pe intrare prima oară.

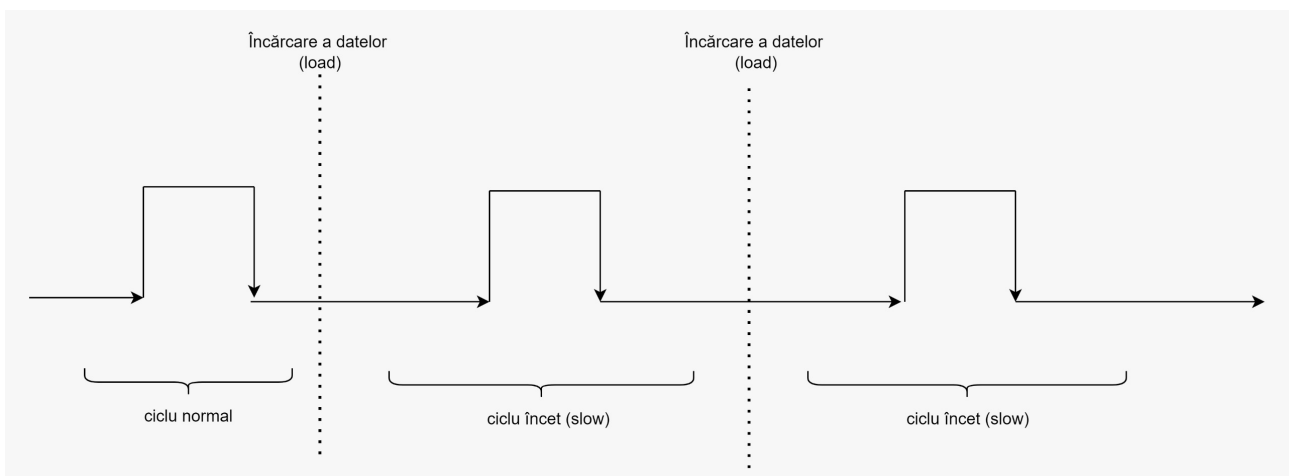


Figura 3.11: Reprezentarea ciclurilor datelor în procedura ***capt_mul_3pat_1clk***

În fișierul ***stuckat_user.dofile*** se execută procedurile definite în ***fsatpg_stuckat.testproc*** și se creează patternurile, fiind cel mai important fișier din ierarhia prezentată. Comanda de la

început ***set_abort_limit 300 100*** reprezintă o măsură de siguranță pentru generarea vectorilor de test în contextul timpului de execuție. Primul număr, 300, reprezintă limita superioară de încercări a unui vector de test generat pentru a detecta un anumit defect, iar 100 reprezintă limita inferioară. Astfel nu se intră în ciclu infinit, iar dacă respectivul vector de test trece de limita superioară acesta este eliminat deoarece nu a detectat defectul impus în numărul de încercări alocat.

Comanda următoare, ***set_pattern_type -sequential 4 -multiple_load on*** pregătește câteva setări pentru generarea de vectori:

- ***set_pattern_type*** determină tipul de patternuri care sunt folosite de către instrumentul de ATPG în timpul simulării [25]
- Argumentul ***-sequential 4*** reprezintă adâncimea la care poate să se ducă tool-ul în afara lanțului de scan (considerând elementele non-scan din design) pentru a putea detecta defecte [25], în cazul acesta 4 celule.
- Argumentul ***-multiple_load on*** permite crearea de patternuri cu procedurile de multiple_load create anterior.

În continuare vom apela procedurile configurate și explicate anterior pentru a vedea câți vectori de test sunt generați cu acestea. Sintaxa folosită este următoarea:

```
set_capture_proc off -all
set_capture_proc on nume_procedura
reset_au_faults
create_patterns -no_auto
report_statistics
```

Set_capture_proc off -all și ***set_capture_proc on nume_procedura*** sunt folosite pentru a dezactiva toate procedurile posibile active și a activa doar procedura de care avem nevoie, deoarece dorim să vedem întâi cum detectează vectori de test fiecare procedură în parte.

Linia de cod ***reset_au_faults*** resetează anumite categorii de defecte pe care utilitarul le consideră netestabile (ATPG Untestable) și le reclasifică, în speranța că le va putea detecta ca posibil testabile, mărinnd astfel test coverage-ul. [25]

Ultimele 2 rânduri din secvența de cod desemnă procedurii creează patternurile de test în mod automat, cu viteza de lucru maximă posibilă și timpul cel mai scurt posibil, la final generând rapoarte statistice pentru a vedea rezultatele. Se rulează această secvență pentru fiecare procedură în parte, însă putem rula și pentru mai multe proceduri deodată, acest lucru nefiind atât de eficient deoarece s-ar pierde mai mult timp de execuție în comparație cu vectorii de test câștigați.

În continuare, prin comenzile de ***compress_patterns*** și ***order_pattern*** se produce o compresie statică și o rearanjare a seturilor de vectori de test, realizându-se o optimizare.

La final, se generează rapoartele specifice pentru generarea reușită de vectori, se salvează modelul flattened al design-ului pentru utilizare ulterioară (etapa de diagnoză) și se salvează pattern-urile generate în fișiere .bin și .WGL.gz.

```
// Simulation performed for #gates = 63870 #faults = 157978
// system mode = analysis pattern source = internal patterns
// -----
// #patterns test #faults #faults # eff. # test process RE/AU/AAB
// simulated coverage in list detected patterns patterns CPU time
//
// Test generation with capture procedure "capt_basic_clk"
// deterministic ATPG invoked with abort limit = 100
// ---
// 105 83.57% 30980 124007 64 76 30.70 sec 0/2991/73
// 77.32 sec 0/2991/74
// 169 90.59% 16476 14504 64 140 78.28 sec
// 103.84 sec 0/2991/82
// 233 92.97% 11570 4906 64 204 104.28 sec
// 135.81 sec 0/2991/87
// 297 94.99% 7383 4187 64 268 136.39 sec
// 160.17 sec 0/2991/93
// 361 96.29% 4702 2681 62 330 160.41 sec
// 173.93 sec 0/2991/97
// 425 97.11% 3018 1684 64 394 174.22 sec
// 186.91 sec 0/2991/104
// 489 97.44% 2338 680 62 456 187.12 sec
// 195.80 sec 0/2991/104
// 553 97.67% 1863 475 63 519 195.96 sec
// deterministic ATPG invoked with abort limit = 100
// ---
// 617 97.90% 1389 474 60 579 203.87 sec 0/2991/17*
// 204.01 sec
// 681 98.03% 1117 272 64 643 208.09 sec 0/2991/18
// 214.54 sec 0/2991/18
// 745 98.12% 928 189 60 703 214.69 sec
// 215.91 sec 0/2991/18
// 809 98.19% 789 139 63 766 216.05 sec
// 216.54 sec 0/2991/18
// 873 98.24% 681 108 60 826 216.69 sec
// 217.61 sec 0/2991/18
// 927 98.27% 614 67 53 879 217.74 sec
```

Figura 3.12: Generare de vectori de test cu procedura ***capt_basic_clk*** în timp real

În figura 3.12 putem observa cum se generează vectorii de test prin procedura ***capt_basic_clk***. Putem observa faptul că numărul total de porți logice din circuit este 63870, iar numărul total de defecte din toate categoriile este 157978. De asemenea se observă cum primii vectori de test reușesc să detecteze cele mai multe defecte, test-coverage-ul crescând din ce în ce mai mult (ce este peste 95% reprezintă un circuit testat corespunzător).

3.3.1.2 Generarea defectelor de tip Transition

Pentru acest tip de defecte procesul de generare rămâne identic, diferențele majore fiind schimbarea aproape totală a procedurilor din fișierul .testproc și a câtorva configurări, deoarece defectele de tip transition 1.3.1.1 sunt mai greu de detectat decât cele de tip stuck-at 1.3.2.1, fiind nevoie de proceduri diferite.

Diferențele sunt următoarele:

1. În procedura de ***test_setup*** la fiecare ciclu se forțează valoarea resetului să fie în 1 (inactiv), deoarece atunci când dorim să generăm patternuri pentru defecte de tip transition, ce nu se dorește este să se genereze vreun reset.
În momentul în care se generează resetul, chiar dacă defectul este detectat, acesta nu se mai propagă până la ieșirea de test a circuitului pentru a putea fi reperat și atunci defectul se clasifică drept nedetectat, iar patternul de test se consideră nefuncțional.
2. Procedurile ***capt_scan_reset_n*** și ***capt_basic_clk*** sunt eliminate complet din același motiv explicat anterior.
3. Ca și proceduri multiple-load s-a folosit una singură (s-a observat ca adăugarea mai multor proceduri îngreunează prea mult simularea, depistarea de vectori de test noi fiind infimă), și anume ***capture_mul_1slow_2fast***.

Aceasta prezintă un ciclu slow în care se forțează valori pe intrările primare, urmat de 2 cicluri rapide de tip load (de încărcare a valorilor) pentru a fi siguri că putem depista defectul. Ultimul ciclu capturează datele pe ieșirile primare, iar toate ciclurile forțează reset-ul în 1 pentru a nu interveni. Ele pot fi observate mai bine în figura 3.13.

4. Ca și timeplate-uri, acestea sunt cele standard folosite și pentru generarea de vectori în cadrul defectelor stuck-at.

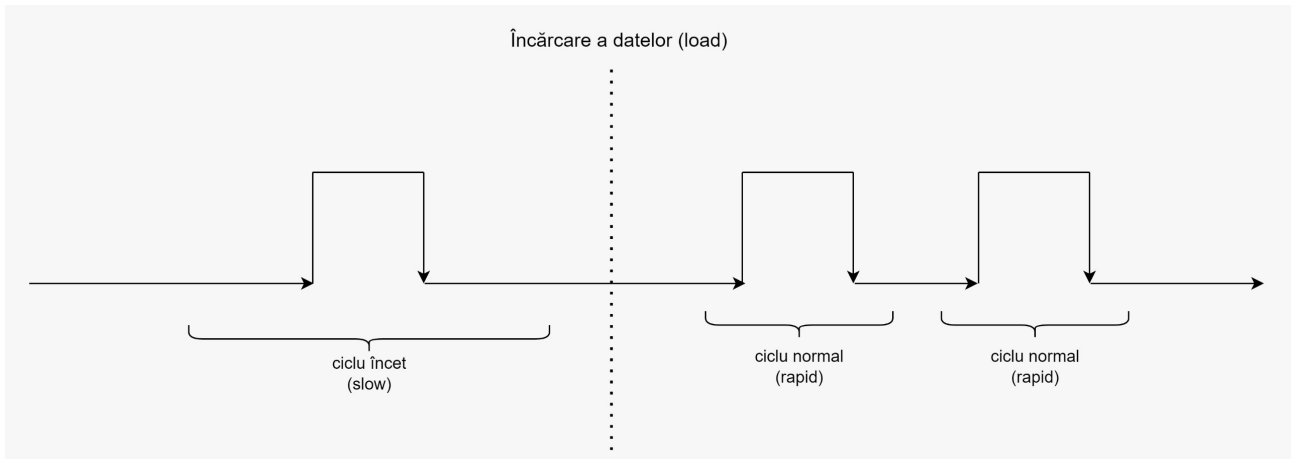


Figura 3.13: Reprezentare prin semnale a ciclurilor din procedura *capture_mul_1slow_2fast*

Lucrurile stau aproape identic și în fișierul final, *transition_user.dofile*, cu excepția următoarelor configurări pentru defecte de tranziție:

1. ***Set_fault_type transition -no_shift_launch*** îi comunică tool-ului de ATPG modelul de defecte necesar pentru vectorii de test
2. Comanda ***delete_faults -ASynchronous_controls*** șterge toate defectele asiguate semnalelor asincrone (reset, set) ale elementelor de stare și RAM deoarece nu este nevoie de acestea pentru detectarea defectelor de tip transition, semnalul de reset acționând doar la începutul simulării.

3.3.2 Cazul cu EDT (variante cu 30, 40, 50, 60 de lanțuri de SCAN)

Procesul de generare de vectori pentru cazul în care avem și EDT-ul integrat este în proporție de 90% același în comparație cu generarea de vectori cu ATPG fără EDT, cu excepția câtorva diferențe semnificative, și anume configurarea unor setări suplimentare pentru tool-ul de ATPG cu EDT.

Următoarea figură reprezintă flow-ul și structura de fișiere în cadrul acestui proces, preponderent asemănătoare cu cazul fără EDT, dar cu mici adăugiri.

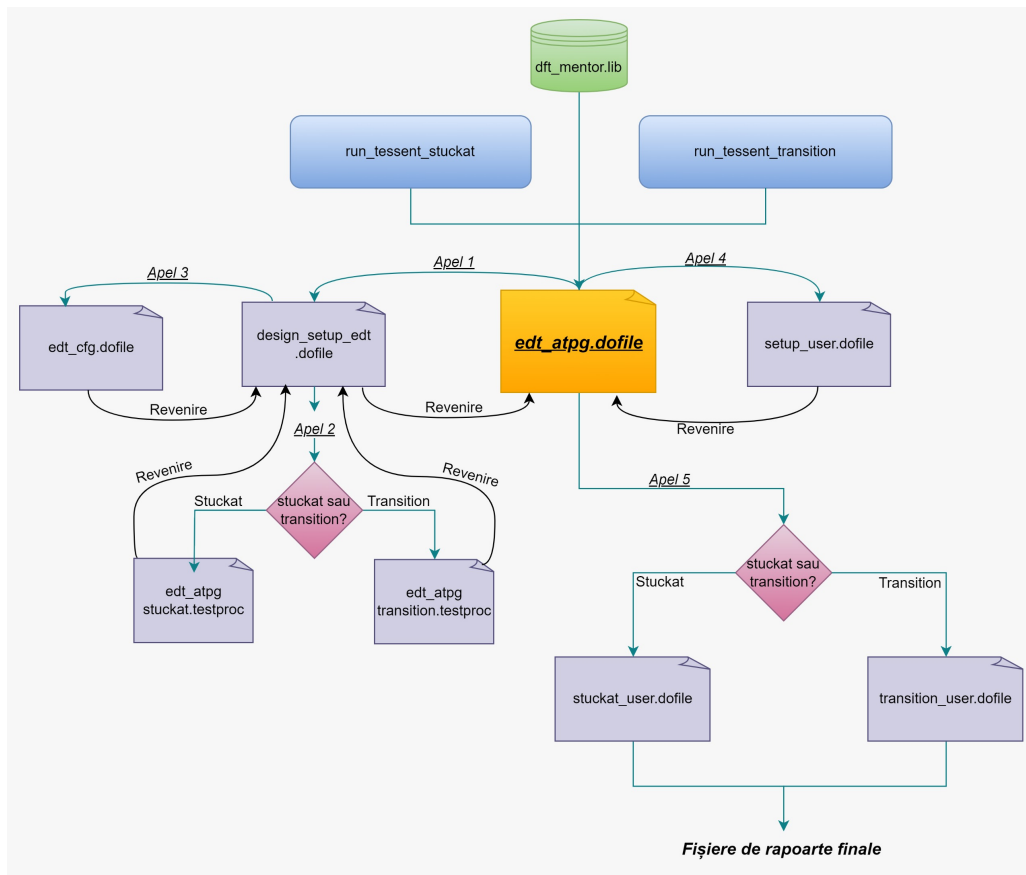


Figura 3.14: Ierarhizarea și interapelarea fișierelor pentru cazul ATPG folosind EDT

În cadrul acestor fișiere, principalele diferențe dintre cazul cu și fără EDT sunt următoarele:

- În cadrul fișierului **design_setup_edt.dofile** se introduc noi semnale specifice introducerii EDT-ului, cât și setarea diferită a lanțurilor de scan deoarece există mai multe în design acum.
 - Se introduc noi semnale de control și se setează pinii și constrângerile de valori pentru acestea, cum ar fi `edt_update`, `edt_clock`, `edt_reset`, `edt_channels_in` și `edt_channels_out`. Semnalele `edt_update`, `edt_clock` și `edt_reset` se utilizează pentru a avea mai mult control intern asupra modului de EDT, ținând cont că acesta este un model separat integrat în modulul de top care trebuie controlat separat față de nucleul RISC-V. Acestea li se configurează pini dedicați în modulul de top pentru a le genera din partea externă.
 - Semnalele `edt_channels_in` și `edt_channels_out` reprezintă canalele de intrare și ieșire de scan prin care circulă vectorii de test compresăți. Chiar dacă este scris la plural (channels), canalul de intrare și ieșire este unul singur în acest caz, logica de compresie fiind explicată în capitolul 1.8.
 - În funcție de tipul de defect detectat, este apelat `.testproc`-ul respectiv conținând procedurile de test care sunt exact aceleași în comparație cu generarea de vectori fără EDT deoarece se dorește o comparație între ele, aceasta fiind irelevantă dacă folosim alte proceduri de test pentru cazul EDT.
 - Se setează variabila `NUMBER_OF_CHAINS` la câte lanțuri de scan am generat în subcapitolul precedent de generare de EDT 3.1, după care se formează legăturile fiecărui lanț cu pinii de intrare și de ieșire ai top-ului, salvându-se ca un lanț intern.

- Ultima oară se setează pinii pentru semnalele despre care s-a reamintit și se apelează fișierul `edt_cfg.dofile`
- Dofile-ul `edt_cfg` este același ca și cel generat odată cu generarea EDT-ului, etapă explicată anterior 3, lăsând doar conexiunile pentru decompresor, compactor și registrul de mascare (parte din compactor, format dintr-un "xor tree").
- În cadrul fișierelor `stuckat_user.dofile` și `transition_user.dofile` se mai adaugă o comandă de configurare, și anume: `set_checkpoint_options on -pattern.fi checkpoint_stuckat.bin -pattern.format binary -replace -period 720`
 - Pe scurt, aceasta salvează în fiecare zi ultimul pattern generat în format binar.
 - `-period 720` s-a ales ca și număr total de minute, adică 60 minute * 12 ore.
- Ultima diferență este faptul că în fișierul `stuckat_user.dofile` se adaugă următoarea comandă: `add_input_constraints -slow_pad -all`
 - `add_input_constraints` desemnează forțarea unei valori anume pe toate intrările primare. [25]
 - Argumentul `-slow_pad`, împreună cu argumentul `-all` realizează aplicarea contrângerilor specificate pe pinii de intrări primare care sunt capabile să ia valoarea Z (impedanță ridicată).

Cum s-a afirmat și înainte, în urma acestor simulări de generare de vectori de test, în interiorul fișierelor se generează anumite loguri sau rapoarte care prezintă stările curente sau aspecte de interes din circuit, respectiv din cadrul vectorilor de test. Cele mai importante rapoarte și loguri sunt:

- loguri privind execuția generării patternurilor
- rapoarte privind lanțurile și celulele de scan
- rapoarte privind test coverage-ul și patternurile de test

Mai jos în figura 3.15 este reprezentat un raport pentru cele 30 de lanțuri de scan în cazul cu EDT.

```
chain = chain1 group = group1 input = '/edt_i/edt_decompressor_i/u34/Z' output = '/u_soc_student/u_soc/u_uart/rx_shift_reg_q_reg[7] /Q' length = 185
chain = chain2 group = group1 input = '/edt_i/edt_decompressor_i/u57/Z' output = '/u_soc_student/u_soc/u_uart/rxd_q_reg/Q' length = 185
chain = chain3 group = group1 input = '/edt_i/edt_decompressor_i/u68/Z' output = '/u_soc_student/u_soc/u_uart/txd_q_reg/Q' length = 185
chain = chain4 group = group1 input = '/edt_i/edt_decompressor_i/u68/Z' output = '/u_soc_student/u_soc/u_spi/spi_clk_q_reg/Q' length = 185
chain = chain5 group = group1 input = '/edt_i/edt_decompressor_i/u90/Z' output = '/u_soc_student/u_soc/u_spi/spi_mosi_q_reg/Q' length = 185
chain = chain6 group = group1 input = '/edt_i/edt_decompressor_i/u38/Z' output = '/u_soc_student/u_soc/u_spi/spi_ssr_value_q_reg/Q' length = 185
chain = chain7 group = group1 input = '/edt_i/edt_decompressor_i/u73/Z' output = '/u_soc_student/u_soc/u_uart/wr_data_q_reg[7] /Q' length = 185
chain = chain8 group = group1 input = '/edt_i/edt_decompressor_i/u80/Z' output = '/u_soc_student/u_soc/u_uart/wr_data_q_reg[6] /Q' length = 185
chain = chain9 group = group1 input = '/edt_i/edt_decompressor_i/u42/Z' output = '/u_soc_student/u_soc/u_timer/wr_data_q_reg[6] /Q' length = 185
chain = chain10 group = group1 input = '/edt_i/edt_decompressor_i/u64/Z' output = '/u_soc_student/u_soc/u_timer/wr_data_q_reg[9] /Q' length = 185
chain = chain11 group = group1 input = '/edt_i/edt_decompressor_i/u54/Z' output = '/u_soc_student/u_soc/u_timer/wr_data_q_reg[10] /Q' length = 185
chain = chain12 group = group1 input = '/edt_i/edt_decompressor_i/u83/Z' output = '/u_soc_student/u_soc/u_timer/wr_data_q_reg[13] /Q' length = 185
chain = chain13 group = group1 input = '/edt_i/edt_decompressor_i/u76/Z' output = '/u_soc_student/u_soc/u_timer/wr_data_q_reg[17] /Q' length = 185
chain = chain14 group = group1 input = '/edt_i/edt_decompressor_i/u45/Z' output = '/u_soc_student/u_soc/u_gpio/gpio_direction_output_q_reg[23] /Q' length = 185
chain = chain15 group = group1 input = '/edt_i/edt_decompressor_i/u49/Z' output = '/u_soc_student/u_soc/u_gpio/gpio_direction_output_q_reg[22] /Q' length = 185
chain = chain16 group = group1 input = '/edt_i/edt_decompressor_i/u93/Z' output = '/u_soc_student/u_soc/u_gpio/gpio_direction_output_q_reg[20] /Q' length = 185
chain = chain17 group = group1 input = '/edt_i/edt_decompressor_i/u58/Z' output = '/u_soc_student/u_soc/u_gpio/gpio_direction_output_q_reg[31] /Q' length = 185
chain = chain18 group = group1 input = '/edt_i/edt_decompressor_i/u74/Z' output = '/u_soc_student/u_soc/u_gpio/gpio_direction_output_q_reg[27] /Q' length = 185
chain = chain19 group = group1 input = '/edt_i/edt_decompressor_i/u81/Z' output = '/u_soc_student/u_soc/u_gpio/output_q_reg[23] /Q' length = 185
chain = chain20 group = group1 input = '/edt_i/edt_decompressor_i/u61/Z' output = '/u_soc_student/u_soc/u_gpio/output_q_reg[31] /Q' length = 185
chain = chain21 group = group1 input = '/edt_i/edt_decompressor_i/u86/Z' output = '/u_soc_student/u_soc/u_gpio/output_q_reg[28] /Q' length = 185
chain = chain22 group = group1 input = '/edt_i/edt_decompressor_i/u69/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[18]Xreg_bit2b
chain = chain23 group = group1 input = '/edt_i/edt_decompressor_i/u65/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[13]Xreg_bit2b
chain = chain24 group = group1 input = '/edt_i/edt_decompressor_i/u87/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[31]Xreg_bit1b
chain = chain25 group = group1 input = '/edt_i/edt_decompressor_i/u91/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[31]Xreg_bit2b
chain = chain26 group = group1 input = '/edt_i/edt_decompressor_i/u55/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[31]Xreg_bit2b
chain = chain27 group = group1 input = '/edt_i/edt_decompressor_i/u77/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[31]Xreg_bit2b
chain = chain28 group = group1 input = '/edt_i/edt_decompressor_i/u51/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[30]Xreg_bit2b
chain = chain29 group = group1 input = '/edt_i/edt_decompressor_i/u84/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[30]Xreg_bit2b
chain = chain30 group = group1 input = '/edt_i/edt_decompressor_i/u44/Z' output = '/u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEX_u_reg/reg_loop2[30]Xreg_bit2b
```

Figura 3.15: Lungimile, intrările și ieșirile celor 30 de lanțuri de scan, cazul EDT

3.4 Simularea vectorilor de test cu ajutorul testbench-ului

Acum că s-a realizat sinteza circuitului pe netlistul sintetizat și s-au generat vectori de test pentru a putea detecta defecte de tip stuck-at, respectiv transition, putem simula acești vectori folosindu-ne de un testbench pentru a-i putea vizualiza.

Un testbench reprezintă un mod de a valida și verifica funcționalitatea unui design de circuit integrat. Acesta constă într-un set de stimuli care sunt aplicați designului pentru a observa comportamentul acestuia și a verifica dacă rezultatele obținute sunt conforme cu așteptările. Un testbench poate fi implementat manual sau folosind unelte de simulare specializate. Este o parte importantă a procesului de dezvoltare a sistemelor electronice și software pentru a asigura că acestea îndeplinesc cerințele specificate. [28]

De asemenea, cu ajutorul testbench-ului putem urmări semnalele de interes în timpul simulării și putem depana situațiile problematice în cazul în care acestea apar. Testbench-ul fiind scris în System Verilog [28], există control deplin asupra modului său de funcționare, putând adăuga semnale de depanare pentru a vedea cu exactitate la ce ciclu de ceas se află eroarea.

3.4.1 Pași pentru simularea patternurilor

Simularea vectorilor de test se va realiza tot în fișierul de generare de vectori de test aferent numărului de lanțuri după care dorim să realizăm simularea, deci nu mai este nevoie de regenerarea acestora deoarece avem deja încărcate rezultatele în fișierele rezultate la capitolul 3.3.2.

Cu toate acestea există câțiva pași de urmat pentru a simula cu succes vectorii deja generați, aceștia fiind următorii:

1. Se intră în Shell-ul de comanda Tessent prin comanda ***ifxsf -eh_dispatch -eh_queue batch "tessent -shell"***. Nu mai este nevoie să specificăm vreun fișier dofile deoarece avem nevoie doar să intrăm în shell pentru a putea da comenzile următoare necesare simulării.
2. Fiind în modul de setup, se specifică tool-ului că este aplicat ATPG-ul standard și se încarcă modelul flattened generat anterior prin comanda ***read_flat_model results/soc_student_stuckat.flat.gz***
3. Se resetează lista de modele de defecte pentru a porni cu o simulare curată.
4. Aflându-ne în modul de analiză, putem citi și simula patternurile:
 - Comanda ***read_patterns results/soc_student.bin*** citește vectorii de test generați.
 - Comanda ***simulate_patterns -store_patterns all*** realizează simularea prin aplicarea sursei de patternuri specificate [25]. De asemenea, argumentul `-store_patterns all` stochează toate patternurile simulate deoarece este nevoie de acestea pentru testbench.
5. Următorul pas și cel mai important îl reprezintă scrierea patternurilor prin comanda ***write_patterns results/\$OUTPUTFILE_30_stuckat_results.v***
 - Aceasta specifică locația și formatul (.v) vectorilor de test simulați.
 - Prefixul `-Verilog`, nu doar că specifică formatul Verilog ca și ieșire, dar face ca tool-ul să creeze în mod automat fișierele necesare pentru testbench.
 - Prefixul `-Serial` determină simularea serială a vectorilor de test în testbench și parcurgerea acestora în mod secvențial în interiorul lanțurilor de scan.

```

ANALYSIS> simulate_patterns -store_patterns all
// -----
// Simulation performed for #gates = 64700 #faults = 166025
// Run fault simulation and store all patterns. pattern source = 1089 external patterns
// -----
// #patterns test #faults #faults # eff. # test process
// simulated coverage in list detected patterns patterns CPU time
// 64 81.93% 35423 130602 64 64 4.33 sec
// 128 89.57% 19437 15986 64 128 4.68 sec
// 192 92.89% 12516 6921 64 192 4.90 sec
// 256 94.79% 8609 3907 64 256 5.10 sec
// 320 95.84% 6430 2179 64 320 5.22 sec
// 384 96.59% 4887 1543 64 384 5.33 sec
// 448 97.11% 3790 1097 64 448 5.44 sec
// 512 97.47% 3042 748 64 512 5.54 sec
// 576 97.74% 2474 568 64 576 5.64 sec
// 640 97.97% 1986 488 64 640 5.73 sec
// 704 98.14% 1624 362 64 704 5.84 sec
// 768 98.28% 1339 285 64 768 5.94 sec
// 832 98.38% 1118 221 64 832 6.03 sec
// 896 98.48% 926 192 64 896 6.12 sec
// 960 98.53% 813 113 64 960 6.21 sec
// 1024 98.57% 718 95 64 1024 6.30 sec
// 1088 98.62% 628 90 64 1088 6.39 sec
// 1089 98.62% 627 1 1 1089 6.43 sec
// 2301 faults were identified as detected by implication.
ANALYSIS>

```

Figura 3.16: Logul pentru simularea vectorilor de test în cazul cu 30 de lanțuri de scan

În continuare, fișierele de testbench generate sunt copiate în directorul de simulare pentru a vizualiza formele de undă ale semnalelor de interes.

3.4.2 Vizualizarea vectorilor de test pe forma de undă

O formă de undă în cadrul circuitelor digitale reprezintă evoluția semnalului electric în funcție de timp. Aceasta poate fi reprezentată grafic și arată cum nivelul semnalului se schimbă de-a lungul timpului.

Pentru a genera forma de undă, se utilizează un program numit SimVision care preia fișierele necesare acestuia, le apelează și generează o simulare în timp real cu valorile semnalelor specificate de noi din cadrul testbench-ului.

Pentru executarea acestui program, avem nevoie de următoarele fișiere ca și date de intrare:

- O listă ordonată care să conțină calea și numele librăriilor tuturor componentelor utilizate (porți logice, bistabile etc.), a celulelor de memorie integrate în modulul de SoC, a netlistului sintetizat, cât și a testbench-ului.
- Fișierele necesare testbench-ului copiate de la punctul anterior.
- (Optional) Un fișier de configurare internă a tool-ului numit **restore.tcl** care restaurează setări folosite la simulări anterioare (semnale vizualizate pe forma de undă, unități de timp etc.)

După ce au fost efectuate aceste configurări cu succes, se pornește simularea SimVision prin comanda **irun -top soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl -f ordered_list_normal.txt -gui -access +c -timescale 1ns/1ps -s -sv -notimezeroasrtmsg -input "restore.tcl"**. "Irun" specifică faptul că se va utiliza software-ul SimVision, argumentul -top specifică numele modulului de top din cadrul testbench-ului, iar argumentul -input "restore.tcl" restaurează setările specificate pentru tool. Restul desemnează unități de timp, limbaje folosite pentru fișierele de intrare și alte specificații. În următoarea figură se poate observa o simulare SimVision efectuată cu succes.



Figura 3.17: Vizualizarea formelor de undă pentru simularea patternurilor pentru 30 de lanțuri

Aceasta reprezintă o simulare efectuată pentru vectorii de test generați cu ATPG folosind EDT cu 30 de lanțuri de scan. După cum se poate observa din figură, am ales anumite semnale de interes pe care le-am categorizat.

Există 1090 de patternuri generate, edt_update generează câte un puls de fiecare dată când se simulează alt pattern de test, resetul se generează doar la începutul simulării, scanmode_i desemnează faptul că modul de scan este utilizat, iar semnalul scan_enable_i funcționează exact așa cum este explicat în capitolul 1.7.1, doar că semnalele de scan_input și scan_output sunt înlocuite de edt_channels_in și edt_channels_out.

De asemenea, prin acest tool nu doar că se poate vizualiza orice semnal din cadrul sistemului de lucru preponderent simulării, dar există și posibilitatea de vizualizare a modulelor interne acestuia în orice ramură dorim. De exemplu, dacă dorim să vizualizăm o celulă de scan, se caută în ierarhia modulului un flip-flop ce conține semnalele aferente de scan și se folosește programul Schematic Tracer pentru vizualizarea internă a acestuia.

Mai jos avem o reprezentare vizuală a 3 elemente dintr-un lanț de scan aflat în circuit. După cum se poate observa, fiecare din acestea prezintă semnale de test_input și test_enable, iar ieșirea fiecăruia este conectată la intrarea de scan a următorului, deplasând datele în interiorul lanțului.

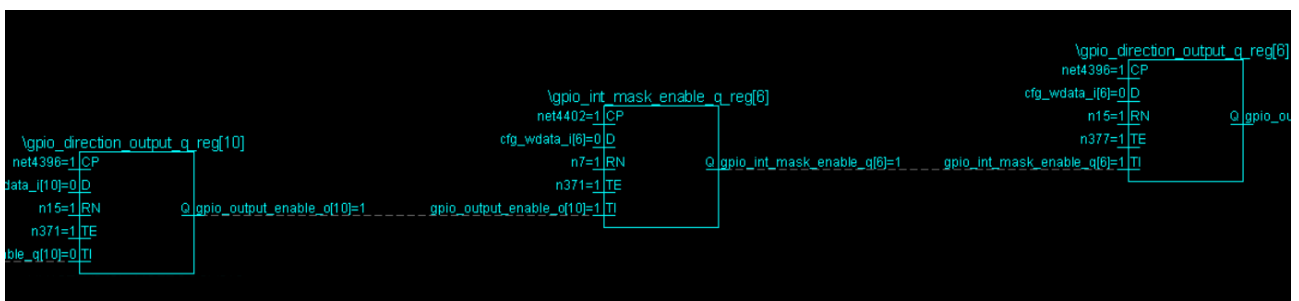


Figura 3.18: Exemple de celule conectate în scan chain (Schematic)

Următoarele 2 figuri (3.19, 3.20) reprezintă o privire mai detaliată asupra semnalelor de interes. Figura 3.19 arată modul în care se modifică semnalele de scan pe durata unei perioade de ceas așa cum se vede în SimVision, iar figura 3.20 arată deplasarea datelor într-un lanț de scan, prin prisma formelor de undă.

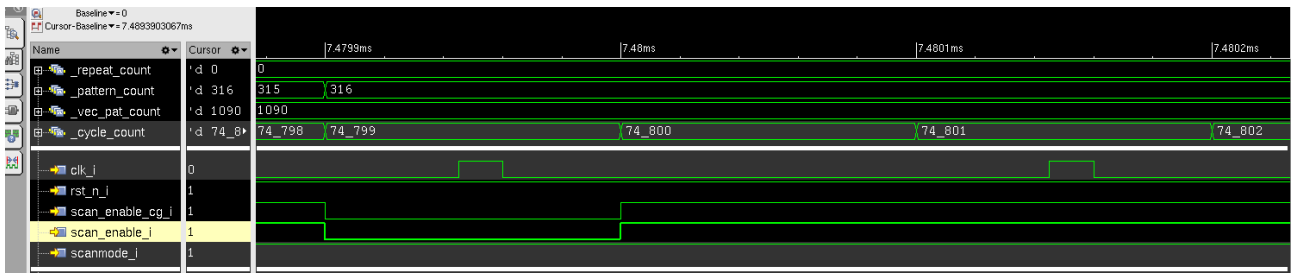


Figura 3.19: Privire mai detaliată asupra semnalelor de scan în cadrul unui pattern

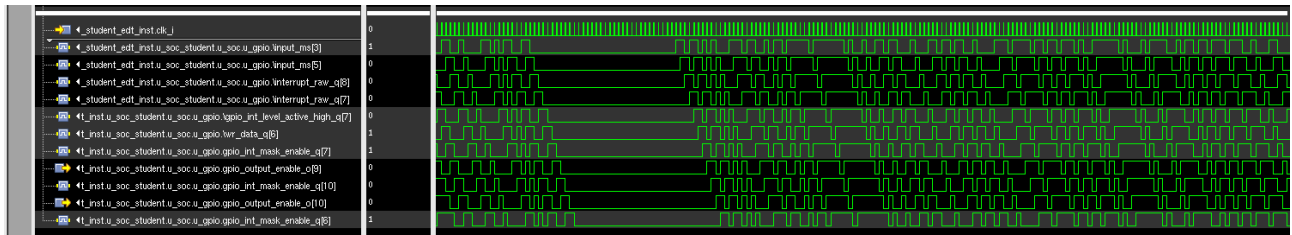


Figura 3.20: Exemplu de simulare pentru a evidenția deplasarea datelor în lanțul de scan

3.5 Etapa de diagnoză

Diagnoza pe un netlist în circuite integrate digitale este un proces complex care presupune identificarea erorilor sau defectelor într-un circuit digital, pe baza descrierii sale logice și a descrierilor suplimentare necesare.

În acest capitol se explică și se evidențiază faptul că, utilizând instrumentul de diagnoză și extrăgând anumite date rezultate din simulări, se poate afla cu ușurință și cu precizie de poartă logică (într-un circuit cu sute de mii sau milioane de porți logice) unde se află defectul care se dorește a fi detectat.

Pașii pentru a realiza acest proces sunt următorii:

1. Pentru început, ca să fie posibilă detectarea unui defect acesta trebuie să existe în circuitul inițial (netlist).

Pentru generarea vectorilor de test sunt introduse constrângeri care reprezintă defectele propriu-zise, însă acum se va introduce un defect intenționat în Netlist pentru a ști cu exactitate de la început unde este defectul, astfel dându-ne seama dacă procesul de diagnoză este eficient sau nu.

3. Următorul pas este extragerea și prelucrarea logului cu informațiile simulării, transformându-l într-un format care poate fi recunoscut de instrumentul de diagnoză.

- Se copiază logul sub numele de `irun.log` în directorul aferent pentru generarea vectorilor de test (în cazul nostru `testpattern/edt/edt_30_chains`) deoarece va fi nevoie de utilizarea Shell-ului de Tessent.
- Se utilizează comanda de bash ***pcregrep -M "Expected response.*\n.*\n.*\n" irun.log >final.txt***. Aceasta extrage doar perechile de 3 rânduri care încep cu string-ul "Expected response".
- Se folosește funcția "search and replace" pentru a selecta din aceste informații doar numărul pattern-ului și numărul ciclului de deplasare al valorilor în lanțul de scan (doar pentru patternurile care detectează acel defect).

Acest lucru se realizează prin regex-ul: `+pattern\s+(\d+).\s+\n.+ \n.+shift cycle:\s+(\d+)` și se înlocuiesc informațiile sub forma `\1 edt_channels_out \2` (indicele patternului + `edt_channels_out` + indicele ciclului din lanțurile respective). În continuare logul `final.txt` ar trebui să arate în felul următor:

```
final.txt
7 edt_channels_out 25
7 edt_channels_out 75
10 edt_channels_out 35
10 edt_channels_out 37
10 edt_channels_out 38
10 edt_channels_out 39
10 edt_channels_out 41
10 edt_channels_out 42
10 edt_channels_out 52
10 edt_channels_out 53
10 edt_channels_out 57
10 edt_channels_out 58
10 edt_channels_out 59
10 edt_channels_out 60
10 edt_channels_out 62
10 edt_channels_out 64
10 edt_channels_out 96
10 edt_channels_out 104
10 edt_channels_out 105
10 edt_channels_out 109
```

Figura 3.23: Fișierul final.txt după prelucrare

4. Acum că fișierul `final.txt` este pregătit, se va efectua procesul propriu-zis de diagnoză, regăsit în fișierul ***edt_diag.dofile***:

- Comanda *set_context patterns -scan_diagnosis* comunică Shell-ului de Tessent că urmează să se efectueze alte comenzi pentru diagnoza circuitului.
- Se citește modelul flattened (*read_flat_model_results/soc_student_dft_30_stuckat_all.flat.gz*)
- Prin comanda *set_diagnosis* se configurează diferiți parametrii pentru proces: argumentul *-max_suspects 50* setează posibilitatea de depistare a maxim 50 de suspecti posibili (porți logice defecte), iar argumentul *internal_failing_cells on* specifică ce patternuri detectează defectul introdus, respectiv scan chainul și celula din scan chain.
- *set_contention_check off* este utilizat pentru a împiedica tool-ul să facă verificări de conflicte pe parcursul simulării.

- Se citește fișierul .bin conținând patternurile de test.
- La final, având toți parametrii, se efectuează diagnoza prin comanda `diagnoze_failures` care ia ca argument fișierul cu defectiuni în formatul specificat de tool.

```

symptom=1 #suspects=2 #explained_patterns=64
7 10 33 55 57 59 60 68 80 94
115 116 130 135 150 152 159 160 177 213
223 231 233 262 284 288 291 316 320 328
330 336 340 341 345 366 376 386 465 483
495 499 501 523 535 574 576 615 642 649
689 690 706 710 728 743 767 788 796 927
989 1018 1050 1055
suspect score fail_match pass_mismatch type value pin_pathname cell_name net_pathname
-----
1 100 64 0 STUCK 0 /u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEXu_reg/u60/Z
/n89
2 100 64 0 EQ1 0 /u_soc_student/u_core/u_core/u_issue/u_regfile/REGFILE_XILINX_SINGLEXu_reg/u60/A
/n85
-----

Internal failing cells profiling
Total failing patterns=64, Total external compressed bits=176
Failing Pattern 7 has 2 compressed failing bits
chain=chain14, cell=25, confidence=100.00%
chain=chain14, cell=75, confidence=100.00%
Failing Pattern 10 has 25 compressed failing bits
chain=chain2, cell=34, confidence=86.21%
chain=chain2, cell=35, confidence=86.21%
chain=chain2, cell=37, confidence=86.21%
chain=chain2, cell=38, confidence=86.21%
chain=chain2, cell=39, confidence=86.21%
chain=chain2, cell=40, confidence=86.21%
chain=chain2, cell=41, confidence=86.21%
chain=chain2, cell=57, confidence=86.21%
chain=chain2, cell=58, confidence=86.21%
chain=chain3, cell=34, confidence=86.21%
chain=chain3, cell=40, confidence=86.21%
chain=chain3, cell=42, confidence=86.21%
chain=chain3, cell=52, confidence=86.21%
chain=chain3, cell=53, confidence=86.21%
chain=chain3, cell=59, confidence=86.21%
chain=chain3, cell=60, confidence=86.21%
chain=chain3, cell=62, confidence=86.21%
chain=chain3, cell=64, confidence=86.21%
chain=chain10, cell=96, confidence=86.21%

```

Figura 3.24: Rezultatul procesului de diagnoză

Duă cum se poate vedea din figura 3.24, la final se generează un vector de posibili suspecti (porți logice detectate defecte) ordonați descrescător în funcție de un indice de încredere, care arată cât consideră tool-ul că respectiva poartă este cea defectă. De asemenea, în această simulare patternul cu numărul 7 reușește să detecteze suspectul cu un indice de încredere de 100%, urmat de patternul 10 cu un indice de încredere de 86%, specificându-se în același timp tipul de defect detectat.

Se poate spune că procesul de diagnoză a funcționat cu succes, ajungând astfel exact la defectul introdus de noi inițial 1.

Capitolul 4

Rezultate și discuții (ATPG)

4.1 Criterii de performanță ale sistemului pentru diferite variante de test (non-EDT vs EDT-30/40/50/60 chains)

În acest capitol se vor sintetiza toate informațiile discutate la capitolele anterioare, atât despre sinteză cât și pe partea de generare de vectori de test pentru toate cazurile, și se vor analiza și discuta rezultatele obținute, făcându-se o comparație obiectivă. Următoarea figură prezintă toate rezultatele comprimate într-un tabel cu metrici de performanță.

ATPG										
Final	Stuckat					Transition				
	FastScan	EDT 30 chains	EDT 40 chains	EDT 50 chains	EDT 60 chains	FastScan	EDT 30 chains	EDT 40 chains	EDT 50 chains	EDT 60 chains
Patternuri totale	883	1122	1253	1398	1575	1609	2492	3158	3485	3805
Ciclii de shiftare	5548	185	139	112	93	5548	185	139	111	93
Offset de EDT	0	26	26	26	26	0	26	26	27	26
Ciclii totali de shiftare (per pattern)	5548	211	165	138	119	5548	211	165	138	119
Ciclii totali de shiftare	4898884	236742	206745	192924	187425	8926732	525812	521070	480930	452795
test time (@100MHz) [ms]	48.989	2.367	2.067	1.929	1.874	89.267	5.258	5.211	4.809	4.528
test coverage (%)	99.78	99.74	99.73	99.66	99.57	98.76	98.71	98.66	98.58	98.44
Arie (μm^2)	41526	42068	42117	42220	42295	41526	42068	42117	42220	42295

Figura 4.1: Tabelul cu metrici de evaluare pentru generarea patternurilor de test

Rezultatele s-au obținut pe urma simulărilor de generare de vectori de test pentru ATPG normal, respectiv pentru ATPG folosindu-se EDT cu 30,40,50 sau 60 de lanțuri de scan, aceste simulări realizându-se pentru a detecta defecte de tip stuck-at și transition.

În continuare se va lua pe rând fiecare metrică și se va explica utilitatea acesteia și rezultatele ei.

4.1.1 Patternuri de test

După cum se poate observa, în cazul ATPG-ului normal s-au generat 883 de vectori de test (acest număr nu se poate anticipa), în timp ce, integrând modulul de EDT, crește considerabil numărul de vectori de test generați. Acest lucru se întâmplă din următoarele cauze:

- EDT-ul, așa cum se specifică și în denumirea acestuia, se concentrează pe metode deterministice de generare de vectori de test care vizează defecte și modele de defecte specifice. Acestea sunt mai eficiente decât metodele aleatoare, dar tind să genereze un număr mai mare de vectori de test.
- EDT-ul este conceput să detecteze defecte greu de detectat cu metode de ATPG tradiționale, realizând acest lucru prin adăugarea de logică de test suplimentară în circuit pentru a crește observabilitatea și controlabilitatea nodurilor interne. Acest lucru necesită mai multe patternuri de test pentru un test-coverage cât mai mare.

- Diferența în numărul de lanțuri de scan.

În cazul fără EDT există un singur lanț de scan, în timp ce, adăugând modulul de EDT, se pot configura mai multe lanțuri de scan prin care datele se deplasează în paralel, ajungând la mai multe patternuri de test.

De asemenea, numărul de vectori de test se mărește și în cazul în care se crește numărul lanțurilor de scan pentru EDT. Cu cât numărul acestora este mai mare, cu atât scade gradual lungimea fiecărui lanț de scan și numărul de flip-flopi prin care circulă datele este distribuit mai multor lanțuri, ceea ce produce efectul descris.

Cât despre tipul de defect, există câteva motive principale de ce este nevoie de un număr mult mai mare de vectori de test pentru a detecta defecte de tip transition decât cele de tip stuck-at:

- **Propagarea defectului:** Defectele de tranziție au nevoie ca vectorul de test respectiv să poată propaga efectul până într-un punct observabil din circuit fără a se pierde între timp, în comparație cu defectele stuck-at care doar blochează intrarea sau ieșirea unei porți la o valoare fixă.
- **Considerente de sincronizare:** Defectele transition sunt mult mai sensibile la sincronizarea tranziției semnalelor de la o valoare în alta, ceea ce conduce la mai mulți vectori de test pentru a putea capta timpii necesari detectării defectului.

4.1.2 Ciclii de shiftare

Un ciclu de shiftare reprezintă o deplasare a datelor cu o poziție în lanțul de scan corespunzător. Cum în cazul ATPG normal este prezent un singur lanț de scan, numărul ciclilor este egal cu dimensiunea lanțului respectiv deoarece datele circulă prin toți flip-flopii de scan înainte să fie măsurați la ieșire.

În restul cazurilor (EDT), numărul total de celule de scan se împarte la numărul scan chain-urilor și se obține numărul de ciclii de ceas pentru un lanț de scan, datele circulând în paralel. Exemplu pentru EDT cu 30 de lanțuri de scan:

$$\text{ciclii shiftare} = \frac{\text{număr total de celule de scan}}{\text{număr lanțuri de scan}} = \frac{5548}{30} = 185$$

Tipul de defect nu afectează acest număr deoarece logica de testare din interiorul modulului principal nu se modifică.

4.1.3 EDT Offset

Din perspectiva ATE-ului, design-ul prezintă un singur lanț de scan care circulă prin intrarea `edt_channels_in` înainte să intre în decompresor.

Acest aspect se întâmplă doar în cazul EDT-ului, iar fiecare pattern necesită un număr de ciclii de shiftare adiționali care se adună la numărul de patternuri din fiecare lanț calculat la punctul anterior.

Acești ciclii adiționali provin din modul de funcționare și structura EDT-ului, mai exact din [22]:

- ciclii de initializare
- biții de mascare (dacă se folosește o tehnologie specializată numită Xpress)

- biți de low-power (când se folosește un decompresor de putere scăzută)
- biții de pipeline definiți de utilizator

Numărul de cicluri de inițializare se poate calcula cu ajutorul următoarei formule [22] :

$$\text{număr cicluri de inițializare} = \lceil \frac{\text{dimensiunea decompresorului}}{\text{numărul de canale}} \rceil$$

, unde fracția se rotunjește la valoarea sa superioară. De exemplu, dacă avem un design în care decompresorul are dimensiunea de 32 de biți și există 6 canale de scan, iar numărul de celule de scan din fiecare lanț este 870, atunci este nevoie de 6 cicluri de inițializare. Fiecare lanț având 870 de celule și fiecare vector comprimat având nevoie de 6 cicluri de inițializare, rezultă că este nevoie de 876 de deplasări per pattern (nefolosind ceilalți biți).

Acest număr se adună în continuare celorlalte elemente din lista de mai sus, rezultând astfel numărul total de cicluri de shiftare suplimentari:

cicluri suplimentari = cicluri de inițializare + biți de mascare + biți de low-power + biți de pipeline

În cazul nostru, din rapoartele de simulare reiese că există 26 de cicluri suplimentari, care se vor aduna la lungimea lanțului de scan, rezultând numărul total de cicluri de shiftare.

4.1.4 Cicli totali de shiftare (per pattern)

Așa cum s-a specificat anterior, numărul de cicluri totali de shiftare reprezintă suma dintre numărul de cicluri propriu-zis și offset-ul de EDT, în cazul ATPG-ului obișnuit acesta este 0.

4.1.5 Cicli totali de shiftare

Folosind numărul de cicluri de shift per pattern și numărul total de patternuri de test, putem calcula numărul total de cicluri de shift pentru întreg modulul cu ajutorul următoarei formule:

$$\text{cicluri totali de shift} = \text{cicluri de shift per pattern} \times \text{număr de patternuri}$$

Cu ajutorul acestei metrici se poate determina dimensiunea memoriei necesare pentru efectuarea simulărilor de generare de patternuri.

4.1.6 Timp de testare

Cum fiecare element din lanțurile de scan reprezintă flip-flipi de scan care sunt celule secvențiale de memorie, acestea funcționează în mod sincron cu ceasul sistemului. Prin urmare, folosindu-ne de frecvența sistemului putem calcula timpul necesar deplasării datelor prin lanțurile de scan folosind toate patternurile generate. Acesta se calculează cu formula:

$$\text{timp de testare} = \text{număr total de cicluri} \times \frac{1}{\text{frecvența ceasului de sistem}}$$

În cazul de față frecvența ceasului de sistem este 100MHz, ceea ce înseamnă că o perioadă de ceas are 0.01 μ s, adică 10 ns. De exemplu, pentru cazul cu 30 lanțuri de scan, avem 236.742 cicluri totali, ceea ce înseamnă că timpul total de testare este 2.367.420 ns, aproximativ 2,36 secunde.

De asemenea, având în vedere că datele circulă în paralel prin lanțuri, timpul de testare este mult mai mic în cazul utilizării EDT-ului în comparație cu ATPG normal (20 ori mai mic).

4.1.7 Test Coverage

În contextul design-for-testing (DFT), noțiunea de "test-coverage" se referă la măsura în care un set de teste aplicate unui sistem sau circuit electronic acoperă sau verifică toate părțile și funcționalitățile prevăzute. Acoperirea testelor este o metrică importantă a calității procesului de testare și al nivelului de încredere pe care ne putem baza ca să știm dacă produsul nu mai prezintă defecte după ce a fost testat.

După cum se poate observa din tabel, cu cât numărul de lanțuri de test este mai mare, cu atât test-coverage-ul este mai mic (putem spune că sunt mărimi invers proporționale), deoarece circuitul devine mai puțin controlabil din punct de vedere al testării. Cu alte cuvinte, un raport de compresie mai mare duce la mărirea compresiei pentru fiecare pattern de test, dar de asemenea crește și posibilitatea generării de patternuri care nu pot fi comprimate și care duc în final la un test-coverage mai mic.

4.1.8 Arie

Deși nu este o metrică atât de importantă pentru testare precum test-coverage-ul, am vrut să evidențiez 2 aspecte:

- Logica EDT adaugă o porțiune foarte mică de hardware suplimentar în circuit.

În acest caz având un modul cu doar 64000 de porți logice, logica suplimentară introdusă este de aproximativ 1%, însă pentru un chip în curs de testare care prezintă milioane de porți, logica introdusă pentru EDT ajunge undeva la 0,35% din arie, ceea ce este o adiție infimă în comparație cu timpul de test câștigat.

- Cu cât creștem numărul de lanțuri de scan, cu atât crește aria folosită, însă foarte puțin în comparație cu aria totală, coverage-ul scăzând și el deoarece circuitul este din ce în ce mai greu de testat.

Capitolul 5

Concluzii

5.1 Direcții viitoare de cercetare și dezvoltare

Cu toate ca această lucrare reprezintă un pas important în implementarea și testarea circuitelor digitale pentru detectarea defectelor de fabricație folosind vectori de test generați automat, există multiple direcții viitoare de cercetare și dezvoltare care pot îmbunătăți și extinde rezultatele obținute. Acestea includ următoarele, dar nu se limitează la acestea:

- **Optimizarea performanței sistemului de testare:** Un aspect esențial care poate fi explorat este optimizarea performanței sistemului de testare. În mod specific, se pot studia metode avansate de reducere a timpului de testare fără a compromite acuratețea și acoperirea testelor. Algoritmi noi și tehnici de optimizare a scan chain-urilor și a vectorilor de test pot contribui semnificativ la îmbunătățirea eficienței procesului de testare.
- **Integrarea noilor tehnologii de testare:** Pe măsură ce tehnologia avansează, noi metode și instrumente de testare devin disponibile. Integrarea acestor tehnologii, precum tehnici avansate de Built-In Self-Test (BIST) sau utilizarea inteligenței artificiale pentru generarea și analiza vectorilor de test, poate îmbunătăți semnificativ capacitatea de detectare a defectelor și eficiența procesului de testare.
- **Extinderea scalabilității:** Un alt domeniu de cercetare este extinderea scalabilității sistemului de testare pentru circuite mai complexe și mari. Explorarea modului în care tehnologiile actuale de SCAN și EDT pot fi adaptate și extinse pentru system-on-chips (SoC) de dimensiuni mai mari și cu complexitate sporită poate duce la îmbunătățiri importante în detectarea defectelor.
- **Dezvoltarea de metode de testare pentru noi tipuri de defecte:** În timp ce lucrarea de față se concentrează pe detectarea defectelor de tip STUCKAT și TRANSITION, există și alte tipuri de defecte care pot afecta performanța circuitelor digitale. Cercetarea și dezvoltarea unor metode de testare pentru detectarea defectelor de tip delay, crosstalk și defecte intermitente poate asigura o acoperire mai completă și o fiabilitate crescută a circuitelor testate.
- **Implementarea și validarea în cazuri practice:** O direcție importantă de dezvoltare este aplicarea și validarea metodologiilor propuse în contexte industriale reale. Colaborarea cu industria semiconductorilor pentru implementarea soluțiilor dezvoltate în cadrul unor proiecte pilot poate oferi feedback valoros și poate ajuta la rafinarea tehnicilor de testare.
- **Automatizarea procesului de diagnoză:** Deși lucrarea a abordat deja etapa de diagnoză, există loc pentru automatizarea completă a acestui proces. Dezvoltarea unor instrumente de diagnoză automată, care pot identifica rapid și precis locația defectelor în circuit, ar contribui semnificativ la reducerea timpului de depanare și la îmbunătățirea eficienței generale a procesului de testare.

- **Studii comparative asupra metodelor de testare și posibilitatea combinării acestora:** Realizarea unor studii comparative între diferitele tehnologii și metode de testare disponibile poate oferi o înțelegere mai profundă a avantajelor și dezavantajelor fiecărei abordări. Aceste studii pot ghida alegerea metodologiilor optime pentru diferite tipuri de circuite și aplicații.

De asemenea, se poate aborda în viitor o posibilă combinare a acestor metode pentru o automatizare și o eficientizare mai puternică, extrăgând punctele forte din fiecare metodă de testare.

Deci, deși s-au atins obiective importante în domeniul detectării defectelor de fabricație în circuitele digitale, există multiple direcții viitoare de cercetare și dezvoltare care pot contribui la avansarea cunoștințelor și tehnologiilor în acest domeniu. Explorarea acestor direcții poate duce la sisteme de testare mai eficiente, fiabile și scalabile, asigurând astfel calitatea și performanța superioară a circuitelor digitale.

5.2 Contribuții principale și concluzii

Lucrarea de față, intitulată "Detectia defectelor de fabricație într-un circuit digital cu ajutorul vectorilor de test generați automat", a avut ca scop implementarea unui set de teste pentru un circuit digital, pentru depistarea defectelor de fabricație (DFT). Circuitul digital, descris în limbaj Verilog, a constat dintr-un SoC bazat pe tehnologie RISC-V, cu module dedicate de Timer, UART, SPI și periferice GPIO. În acest capitol se vor sintetiza concluziile principale obținute în urma cercetării și implementării efectuate.

În cadrul acestei lucrări am utilizat tehnologii complexe de testare precum tehnologia SCAN (introducerea lanțurilor de scan în circuit + transformarea flip-flopilor în celule de scan), ATPG (generarea automata a vectorilor de test cu ajutorul constrângerilor), DFT (concept care reunește noțiunile de SCAN, ATPG, EDT) și EDT (compresia vectorilor de test pentru îmbunătățirea performanțelor). Fiecare din aceste tehnologii a prezentat un rol esențial în îmbunătățirea detectării defectelor și optimizarea procesului de testare.

Introducerea logicii de scan în circuit, generarea vectorilor (patternurilor) de test, compresia acestora, procesul de simulare și diagnoză, pentru defecte de tip STUCKAT și TRANSITION au reprezentat exemple și aspecte esențiale în asigurarea acurateții și eficienței testării.

Unul dintre principalele obiective majore ale lucrării a fost verificarea performanțelor sistemului, în special timpul de testare și test-coverage-ul utilizând diverse structuri de test folosind generarea automată de vectori de test. În acest context, s-au analizat performanțele testării modulului atât în cazul utilizării unui singur lanț de scan, cât și în cazul utilizării multiplelor lanțuri de scan cu ajutorul tehnologiei EDT.

Rezultatele au demonstrat că utilizarea EDT a redus semnificativ timpul de testare, păstrând în același timp test-coverage-ul aproape de valoarea rezultată testării cu un singur lanț de scan, iar testele realizate cu 30, 40, 50, respectiv 60 de lanțuri de scan au arătat că este nevoie de atenție atunci când se alege acest număr, păstrând raportul dintre timpul de testare și test coverage în balanță.

Procesul de sinteză a inclus nu doar interpretarea și importarea librăriilor pentru celulele de bază ale circuitului (porți logice, bistabile etc), cât și inserarea logicii de scan în circuit. Acesta s-a realizat prin scripturi bine definite și cu ajutorul makefile-urilor pentru a păstra ordinea și automatizarea în execuția scripturilor.

S-a efectuat analiza codului sursă HDL și al librăriilor (pentru verificarea erorilor de sintaxă și semantică din codul realizat), elaborarea design-ului (transformarea acestuia într-un format inteligibil pentru tool-ul Design Compiler), compilarea design-ului (transformarea design-ului într-o reprezentare structurală), inserția logicii de scan, cât și generarea netlist-ului sintetizat gata pentru etapa de ATPG, pe care am discutat-o anterior, în care am generat vectorii de test pentru structuri de test cu și fără EDT cu ajutorul tool-ului Tessent - Mentor Graphics.

Simularea patternurilor de test folosind SimVision a permis vizualizarea semnalelor de scan (scan_enable, scan_mode, cât și edt_channels_in, edt_channels_out, edt_reset și edt_update), și a flip-flopilor de scan interconectate în lanțuri de scan (propagarea datelor în mod secvențial prin acestea).

Acest pas a fost crucial pentru a asigura că toți vectorii de test generați detectează toate defectele vizate în mod corect și într-un timp cât mai scurt.

În etapa de diagnoză, introducerea unui defect în mod intenționat în circuit a permis testarea capacității sistemului de a identifica exact poarta logică unde se regăsește defectul. Aceasta s-a depistat cu succes cu o precizie și un indice de încredere de 100%.

Această etapă a demonstrat că metodologia de testare utilizată este robustă și capabilă să localizeze cu acuratețe defectele din circuit, confirmând astfel validitatea și eficiența abordării propuse.

Lucrarea a demonstrat că utilizarea vectorilor de test generați automat și a tehnologiilor avansate de testare, precum EDT, poate îmbunătăți semnificativ detectarea defectelor de fabricație în circuitele digitale. Implementarea și testarea efectuate au arătat că abordarea propusă nu doar că reduce timpul de testare, dar și poate păstra test-coverage-ul la o valoare acceptabilă în același timp. În concluzie, metodologia aplicată în această lucrare oferă o soluție eficientă și scalabilă pentru detectarea defectelor în circuitele digitale complexe, fiind relevantă și aplicabilă în industria semiconductoarelor.

Bibliografie

- [1] Scan and ATPG Site kernel description. <https://vlsitutorials.com/dft-scan-and-atpg>. Accessed: 2010-09-30.
- [2] RISC-V Repo kernel description. https://github.com/ultraembedded/riscv_soc. Accessed: 2010-09-30.
- [3] Bo Yao. *Transition Faults and Transition Path Delay Faults: Test Generation, Path Selection, and Built-In Generation of Functional Broadside Tests*. 1st edition, 2013.
- [4] Niraj K Jha and Sandeep Gupta. *Testing of digital systems*. Cambridge University Press, 2003.
- [5] Faults in Digital Systems kernel description. <https://www.slideshare.net/slideshow/fa-31428491/31428491>. Accessed: 2010-09-30.
- [6] Testing vs Verification TestingBrain kernel description. https://www.testingbrain.com/tutorials/testing-vs-verification.html#google_vignette. Accessed: 2010-09-30.
- [7] Testing vs Verification Electronics kernel description. <https://electronics.stackexchange.com/questions/137483/>. Accessed: 2010-09-30.
- [8] Testing vs Verification DesignNews kernel description. <https://www.designnews.com/design-software/what-is-the-difference-between-test-and-verification->. Accessed: 2010-09-30.
- [9] Testing vs Verification Anysilicon kernel description. <https://anysilicon.com/verification-validation-testing-asic-soc-designs-differences/>. Accessed: 2010-09-30.
- [10] Mentor Graphics Corporation. *Tessent Scan and ATPG User's Manual*. Mentor, 2nd edition, 2017.
- [11] Zainalabedin Navabi. *Digital System Test and Testable Design: Using HDL Models and Architectures*. 1st edition, 2019.
- [12] Laung-Terng Wang, Cheng-Wen Wu, and Xiaoqing Wen. *VLSI test principles and architectures: design for testability*. Elsevier, 2006.
- [13] Dft slideshare. <https://www.slideshare.net/slideshow/design-for-testability-65535130/65535130>. Accessed: 2010-09-30.
- [14] Fulvio Corno, Matteo Sonza Reorda, and Giovanni Squillero. High-level observability for effective high-level atpg. In *Proceedings 18th IEEE VLSI Test Symposium*, pages 411–416. IEEE, 2000.
- [15] Hideo Fujiwara. *Logic testing and design for testability*. MIT press Cambridge, MA, 1985.

- [16] Sharad Malik. Analysis of cyclic combinational circuits. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 13(7):950–956, 1994.
- [17] Circuite secventiale, notite de curs 6 CID kernel description. https://archive.curs.upb.ro/2021/pluginfile.php/496758/mod_resource/content/0/Slides_Curs_6.pdf. Accessed: 2010-09-30.
- [18] George H Mealy. A method for synthesizing sequential circuits. *The Bell System Technical Journal*, 34(5):1045–1079, 1955.
- [19] Edt - vlsi tutorials. <https://vlsitutorials.com/test-compression-edt/>. Accessed: 2010-09-30.
- [20] Janusz Rajski, Jerzy Tyszer, Mark Kassab, and Nilanjan Mukherjee. Embedded deterministic test. *IEEE transactions on computer-aided design of integrated circuits and systems*, 23(5):776–792, 2004.
- [21] Siemens EDA. *TessentTM TestKompress Student Workbook*. Siemens, 2nd edition, 2022.
- [22] Mentor Graphics Corporation. *Tessent TestKompress User’s Manual*. Siemens, 2nd edition, 2015.
- [23] Yongcheng Wang and Kefei Song. A new approach to realize uart. In *Proceedings of 2011 international conference on Electronic & Mechanical Engineering and information technology*, volume 5, pages 2749–2752. IEEE, 2011.
- [24] AXISS Wiki Page kernel description. https://en.wikipedia.org/wiki/Advanced_eXtensible_Interface. Accessed: 2010-09-30.
- [25] Siemens EDA. *TessentTM Shell Reference Manual*. Siemens, 3rd edition, 2022.
- [26] What is a makefile and how does it work? <https://opensource.com/article/18/8/what-how-makefile>. Accessed: 2010-09-30.
- [27] Makefile tutorial kernel description. <https://makefiletutorial.com/>. Accessed: 2010-09-30.
- [28] Chris Spear. *SystemVerilog for verification: a guide to learning the testbench language features*. Springer Science & Business Media, 2008.

Anexa A

Cod

```

1 add_scan_groups grp1 dummy_design.testproc
2 add_scan_chains chain1 grp1      edt_si[0]      edt_so[0]
3 add_scan_chains chain2 grp1      edt_si[1]      edt_so[1]
4 add_scan_chains chain3 grp1      edt_si[2]      edt_so[2]
5 add_scan_chains chain4 grp1      edt_si[3]      edt_so[3]
6 add_scan_chains chain5 grp1      edt_si[4]      edt_so[4]
7 add_scan_chains chain6 grp1      edt_si[5]      edt_so[5]
8 add_scan_chains chain7 grp1      edt_si[6]      edt_so[6]
9 add_scan_chains chain8 grp1      edt_si[7]      edt_so[7]
10 add_scan_chains chain9 grp1      edt_si[8]      edt_so[8]
11 add_scan_chains chain10 grp1     edt_si[9]      edt_so[9]
12 add_scan_chains chain11 grp1     edt_si[10]     edt_so[10]
13 add_scan_chains chain12 grp1     edt_si[11]     edt_so[11]
14 add_scan_chains chain13 grp1     edt_si[12]     edt_so[12]
15 add_scan_chains chain14 grp1     edt_si[13]     edt_so[13]
16 add_scan_chains chain15 grp1     edt_si[14]     edt_so[14]
17 add_scan_chains chain16 grp1     edt_si[15]     edt_so[15]
18 add_scan_chains chain17 grp1     edt_si[16]     edt_so[16]
19 add_scan_chains chain18 grp1     edt_si[17]     edt_so[17]
20 add_scan_chains chain19 grp1     edt_si[18]     edt_so[18]
21 add_scan_chains chain20 grp1     edt_si[19]     edt_so[19]
22 add_scan_chains chain21 grp1     edt_si[20]     edt_so[20]
23 add_scan_chains chain22 grp1     edt_si[21]     edt_so[21]
24 add_scan_chains chain23 grp1     edt_si[22]     edt_so[22]
25 add_scan_chains chain24 grp1     edt_si[23]     edt_so[23]
26 add_scan_chains chain25 grp1     edt_si[24]     edt_so[24]
27 add_scan_chains chain26 grp1     edt_si[25]     edt_so[25]
28 add_scan_chains chain27 grp1     edt_si[26]     edt_so[26]
29 add_scan_chains chain28 grp1     edt_si[27]     edt_so[27]
30 add_scan_chains chain29 grp1     edt_si[28]     edt_so[28]
31 add_scan_chains chain30 grp1     edt_si[29]     edt_so[29]
32 add_scan_chains chain31 grp1     edt_si[30]     edt_so[30]
33 add_scan_chains chain32 grp1     edt_si[31]     edt_so[31]
34 add_scan_chains chain33 grp1     edt_si[32]     edt_so[32]
35 add_scan_chains chain34 grp1     edt_si[33]     edt_so[33]
36 add_scan_chains chain35 grp1     edt_si[34]     edt_so[34]
37 add_scan_chains chain36 grp1     edt_si[35]     edt_so[35]
38 add_scan_chains chain37 grp1     edt_si[36]     edt_so[36]
39 add_scan_chains chain38 grp1     edt_si[37]     edt_so[37]
40 add_scan_chains chain39 grp1     edt_si[38]     edt_so[38]
41 add_scan_chains chain40 grp1     edt_si[39]     edt_so[39]
42 add_scan_chains chain41 grp1     edt_si[40]     edt_so[40]
43 add_scan_chains chain42 grp1     edt_si[41]     edt_so[41]
44 add_scan_chains chain43 grp1     edt_si[42]     edt_so[42]
45 add_scan_chains chain44 grp1     edt_si[43]     edt_so[43]
46 add_scan_chains chain45 grp1     edt_si[44]     edt_so[44]
47 add_scan_chains chain46 grp1     edt_si[45]     edt_so[45]
48 add_scan_chains chain47 grp1     edt_si[46]     edt_so[46]
49 add_scan_chains chain48 grp1     edt_si[47]     edt_so[47]
50 add_scan_chains chain49 grp1     edt_si[48]     edt_so[48]
51 add_scan_chains chain50 grp1     edt_si[49]     edt_so[49]
52 add_scan_chains chain51 grp1     edt_si[50]     edt_so[50]
53 add_scan_chains chain52 grp1     edt_si[51]     edt_so[51]
54 add_scan_chains chain53 grp1     edt_si[52]     edt_so[52]
55 add_scan_chains chain54 grp1     edt_si[53]     edt_so[53]
56 add_scan_chains chain55 grp1     edt_si[54]     edt_so[54]
57 add_scan_chains chain56 grp1     edt_si[55]     edt_so[55]
58 add_scan_chains chain57 grp1     edt_si[56]     edt_so[56]
59 add_scan_chains chain58 grp1     edt_si[57]     edt_so[57]
60 add_scan_chains chain59 grp1     edt_si[58]     edt_so[58]
61 add_scan_chains chain60 grp1     edt_si[59]     edt_so[59]
62 add_scan_chains chain61 grp1     edt_si[60]     edt_so[60]
63 add_scan_chains chain62 grp1     edt_si[61]     edt_so[61]

```

```

64 add_scan_chains chain63 grp1    edt_si[62]    edt_so[62]
65 add_scan_chains chain64 grp1    edt_si[63]    edt_so[63]
66 add_scan_chains chain65 grp1    edt_si[64]    edt_so[64]
67 add_scan_chains chain66 grp1    edt_si[65]    edt_so[65]
68 add_scan_chains chain67 grp1    edt_si[66]    edt_so[66]
69 add_scan_chains chain68 grp1    edt_si[67]    edt_so[67]
70 add_scan_chains chain69 grp1    edt_si[68]    edt_so[68]
71 add_scan_chains chain70 grp1    edt_si[69]    edt_so[69]
72 add_clocks 0 clk

```

Listing A.1: dummy_design.dofile

```

1  system date
2
3  set_context dft -edt
4  read_cell_library dummy_design.atpglib
5  read_verilog ./dummy_design.v
6
7  dofile dummy_design.dofile
8
9
10 set_edt_options -input_channels 1 -output_channels 1 -pipeline 0 -bypass_logic off
    -reset_signal asynchronous
11
12 // for 30 chains
13 //set_edt_options -longest_chain_range 150 200
14 // for 40 chains
15 //set_edt_options -longest_chain_range 110 160
16 // for 50 chains
17 //set_edt_options -longest_chain_range 90 140
18 // for 60 chains
19 set_edt_options -longest_chain_range 70 120
20 // for 70 chains
21 set_edt_options -longest_chain_range 50 90
22
23 set_edt_pins output_channel -change_edge_at_compactor_output any
24
25 // scan chains
26 set_compactor_connection -channel 1 -chains chain1 chain2 chain3 chain4 chain5 chain6
    chain7 chain8 chain9 chain10 \
27 chain11 chain12 chain13 chain14 chain15 chain16
    chain17 chain18 chain19 chain20 \
28 chain21 chain22 chain23 chain24 chain25 chain26
    chain27 chain28 chain29 chain30 \
29 chain31 chain32 chain33 chain34 chain35 chain36
    chain37 chain38 chain39 chain40 \
30 chain41 chain42 chain43 chain44 chain45 chain46
    chain47 chain48 chain49 chain50 \
31 chain51 chain52 chain53 chain54 chain55 chain56
    chain57 chain58 chain59 chain60
32
33
34
35
36 set_system_mode atpg
37
38 report_edt_lockup_cells
39
40 report_edt_configuration -verbose
41
42 write_edt_files gen/created -verilog -replace -nortl_prefix
43
44 system date
45 exit

```

Listing A.2: edt_ip.dofile

```

1  #!/bin/bash
2

```



```

3 # EDT regeneration:
4 # edit or create skeleton_input_file (chains working on falling and rising edge)
5
6 # !!!!!!!!!!! add module load dft-mentor/2022.3 if not loaded !!!!!!!!!!!!!!!!!!!!!!!
7 moduleunload dft-mentor
8 module load dft-mentor/2022.3
9
10 cleartool co -unres -nc skeleton_input_file
11 cleartool co -unres -nc dummy_design*
12 cleartool co -unres -nc edt_ip.dofile
13
14 echo -e "\nDon't forget to change number_of_chains variable here, in skeleton_input_file and
    in source/verilog/rtl/soc_student_edt.v !!!\n"
15
16 declare -i number_of_chains=70
17 echo -e "\nCreating skeleton_input_file for $number_of_chains chains...\n"
18 create_skeleton_design -i skeleton_input_file -o dummy_design
19
20
21 echo -e "\nExecuting edt_ip.dofile in 5 seconds...\n"
22 echo -e "5....."
23 sleep 1
24 echo -e "4...."
25 sleep 1
26 echo -e "3..."
27 sleep 1
28 echo -e "2.."
29 sleep 1
30 echo -e "1."
31 sleep 1
32 tesseract -shell -dofile edt_ip.dofile -replace -logfile edt_ip.log
33
34
35 # generated EDT RTL: gen/created_edt.v
36 # copy EDT RTL to source dir
37 echo -e "\nCheckout and copying generated RTL in my source folder for edt design later...\n"
38 cleartool co -unres -nc ../source/verilog/rtl/edt.v
39 echo -e "\nStill passed, continuing...\n"
40 cp gen/created_edt.v ../source/verilog/rtl/edt.v
41
42
43 # copy and process gen/created_edt.dofile
44 # !!!ATTENTION testpattern/edt/edt_atpg/edt_cfg.dofile will be overwritten
45 cleartool co -nc ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "/edt_cfg.dofile
46 cp -f gen/created_edt.dofile ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
47
48 echo -e "\nDeleting lines with add_clocks and the others...\n"
49
50 sed -i '/^add_scan_groups/d' ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
51 sed -i '/^add_scan_chains/d' ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
52 sed -i '/^add_clocks/d' ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
53 sed -i '/^add_pin_constraints/d' ../testpattern/edt/edt_"$number_of_chains"_chains/edt_atpg_"
    "$number_of_chains"/edt_cfg.dofile
54
55 echo -e "\nScript finished!\n"
56
57 echo -e "Now make synthesis on the new edt with $number_of_chains chains"

```

Listing A.3: Bash Script for generating EDT

```

1 //////////////////////////////////////
2 //Skeleton Design Input File
3 //////////////////////////////////////
4 scan_chain_input  edt_si bused
5 scan_chain_output edt_so bused
6 begin_clocks
7 clk 0
8 end_clocks

```

```

9 begin_chains
10 //1 30 200 LE clk LE clk
11 //1 40 150 LE clk LE clk
12 //1 50 120 LE clk LE clk
13 //1 60 100 LE clk LE clk
14 1 70 90 LE clk LE clk
15 end_chains

```

Listing A.4: Skeleton Input File

```

1 #!/bin/bash
2
3 cleartool co -unres -nc irun.log
4 cleartool co -unres -nc waves.shm
5 cleartool co -unres -nc irun.key
6 cleartool co -unres -nc INCA_libs/
7 cleartool co -unres -nc INCA_libs/*
8
9 rm -rf *.log *.elog *.err *.shm *.wlf *.diag *.key INCA_libs/ .simvision/ all_signals.
   shm/

```

Listing A.5: clean.sh

```

1
2 =====
3 -- LIB CELLS
4 =====
5
6
7 =====
8 -- MEMORY MODELS
9 =====
10 /var/vob/testb/m5362/vob/units/soc_student/resources/verilog/icache_tag_ram.v
11 /var/vob/testb/m5362/vob/units/soc_student/resources/verilog/icache_data_ram.v
12 /var/vob/testb/m5362/vob/units/soc_student/resources/verilog/generic_data_ram_32_32.v
13 =====
14
15
16 =====
17 -- NETLIST
18 =====
19 // -- FOR NORMAL USE --
20 ../../testpattern/netlists/soc_student_edt_30_backup.v
21
22 // -- FOR DIAGNOSIS --
23 ../../testpattern/netlists/soc_student_edt_30.v
24 =====
25
26
27 =====
28 -- T E S T B E N C H
29 =====
30 soc_student_dft_30_stuckat_results.v
31 =====

```

Listing A.6: ordered_list_diagnose.txt

```

1
2 =====
3 -- LIB CELLS
4 =====
5
6
7 =====
8 -- MEMORY MODELS
9 =====

```

```

10 /var/vob/testb/m5362/vob/units/soc_student/resources/verilog/icache_tag_ram.v
11 /var/vob/testb/m5362/vob/units/soc_student/resources/verilog/icache_data_ram.v
12 /var/vob/testb/m5362/vob/units/soc_student/resources/verilog/generic_data_ram_32_32.v
13 =====
14
15
16 -----
17 --      NETLIST
18 -----
19 //      -- FOR NORMAL USE --
20 ../../testpattern/netlists/soc_student_edt_30_backup.v
21
22 //      -- FOR DIAGNOSIS --
23 ../../testpattern/netlists/soc_student_edt_30.v
24 -----
25
26
27 -----
28 --      T E S T B E N C H
29 -----
30 soc_student_dft_30_stuckat_results.v
31 -----

```

Listing A.7: ordered_list_normal.txt

```

1
2 # NC-Sim Command File
3 # TOOL: ncsim 15.10-s007
4 #
5 #
6 # You can restore this configuration with:
7 #
8 #      irun -top soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl -f ordered_list.txt
9 #      -access +c -timescale 1ns/1ps -s -sv -notimezeroasrtmsg -input restore_global.tcl
10
11 set tcl_prompt1 {puts -nonewline "ncsim> "}
12 set tcl_prompt2 {puts -nonewline "> "}
13 set vlog_format %h
14 set vhdl_format %v
15 set real_precision 6
16 set display_unit auto
17 set time_unit module
18 set heap_garbage_size -200
19 set heap_garbage_time 0
20 set assert_report_level note
21 set assert_stop_level error
22 set autoscope yes
23 set assert_1164_warnings yes
24 set pack_assert_off {}
25 set severity_pack_assert_off {note warning}
26 set assert_output_stop_level failed
27 set tcl_debug_level 0
28 set relax_path_name 1
29 set vhdl_vcdmap XX01ZX01X
30 set intovf_severity_level ERROR
31 set probe_screen_format 0
32 set rangeinst_severity_level ERROR
33 set textio_severity_level ERROR
34 set vital_timing_checks_on 1
35 set vlog_code_show_force 0
36 set assert_count_attempts 1
37 set tcl_all64 false
38 set tcl_runerror_exit false
39 set assert_report_incompletes 0
40 set show_force 1
41 set force_reset_by_reinvoke 0
42 set tcl_relaxed_literal 0
43 set probe_exclude_patterns {}
44 set probe_packed_limit 4k
45 set probe_unpacked_limit 16k
46 set assert_internal_msg no
47 set svseed 1

```

```

48 set assert_reporting_mode 0
49 alias . run
50 alias iprof profile
51 alias quit exit
52 database -open -shm -into waves.shm waves -default
53 probe -create -database waves
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.clk_i
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_channels_in

    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_channels_out
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_clock
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_reset
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_update
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.rst_n_i
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.scan_enable_cg_i

    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.scan_enable_i
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.scanmode_i
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._skipped_patterns
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._fail_pattern_cnt
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._pattern_count
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._pat_num
54 probe -create -database waves
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.clk_i
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_channels_in

    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_channels_out
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_clock
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_reset
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.edt_update
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.rst_n_i
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.scan_enable_cg_i

    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.scan_enable_i
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.scanmode_i
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._skipped_patterns
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._fail_pattern_cnt
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._pattern_count
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._pat_num
55 probe -create -database waves
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._vec_pat_count
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._save_cycle_count
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl._cycle_count
56 probe -create -database waves
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.u_soc_student

57 .u_soc.u_gpio.@{\gpio_direction_output_q_reg[0] }.Q
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.u_soc_student

58 .u_soc.u_gpio.@{\gpio_direction_output_q_reg[1] }.Q
    soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl.soc_student_edt_inst.u_soc_student

59 .u_soc.u_gpio.@{\gpio_direction_output_q_reg[2] }.Q
60 probe -create -database waves soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl -all
    -variables -memories -sc_processes -depth all
61
62 simvision -input restore_global.tcl.svcf

```

Listing A.8: restore_global.tcl

```

1 #!/bin/tcsh
2
3 module load INCISIV
4
5 irun -top soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl -f ordered_list_diagnose.
    txt -gui -access +c -timescale 1ns/1ps -s -sv -notimezeroasrtmsg -input "restore.tcl"

```

Listing A.9: run_diagnose.sh

```

1 #!/bin/tcsh
2
3 module load INCISIV
4
5 irun -top soc_student_edt_soc_student_dft_30_stuckat_results_v_ctl -f ordered_list_normal.txt
   -gui -access +c -timescale 1ns/1ps -s -sv -notimezeroasrtmsg -input "restore.tcl"

```

Listing A.10: run_normal.sh

```

1 add_clock 1 rst_n_i
2 add_clock 0 clk_i
3 add_clock 0 edt_clock
4
5 add_input_constraint scanmode_i      -C1
6 add_input_constraint edt_channels_in -C0
7 add_input_constraint scan_enable_i   -C0
8 add_input_constraint edt_clock       -C0
9 add_input_constraint edt_update      -C0
10 add_input_constraint edt_reset       -C0
11 add_input_constraint test_se        -C0
12
13 if { $EXECUTE_STUCKAT == "ON" } {
14     add_scan_group group1 edt_atpg_stuckat.testproc
15 }
16 if { $EXECUTE_TRANSITION == "ON" } {
17     add_scan_group group1 edt_atpg_transition.testproc
18 }
19
20 set NUMBER_OF_CHAINS 30
21
22 for {set i 1} {$i <= $NUMBER_OF_CHAINS} {incr i} {
23     add_scan_chain -internal chain$i group1 edt_i/edt_scan_in[$i] edt_i/edt_scan_out[$i]
24 }
25
26
27 set_edt_options -channels 1
28
29 set_edt_pins clock      edt_clock
30 set_edt_pins update     edt_update
31 set_edt_pins scan_enable scan_enable_i
32 set_edt_pins input 1    edt_channels_in
33 set_edt_pins output 1   edt_channels_out
34
35
36 dofile edt_cfg.dofile
37

```

Listing A.11: design_setup_edt.dofile

```

1 // command: diagnose_failures final.txt
2 // Note: Skipping pattern verification (diagnosis_verify_patterns = 0).
3 // Warning: Failure truncation was not specified in failure file 'final.txt'.
4 //         Using default failure truncation handling 'failure_buffer_limit_reached unknown'
   which could impact diagnosis resolution.
5 //         Please use appropriate truncation annotation in the failure file.
6 // Warning: Failure truncation was not specified in failure file 'final.txt'.
7 //         Using default failure truncation handling 'failure_buffer_limit_reached unknown'
   which could impact diagnosis resolution.
8 //         Please use appropriate truncation annotation in the failure file.
9 Tessent Shell 2022.3   Thu Aug 11 22:57:52 GMT 2022
10
11 tracking_info_begin
12 tracking_info_end
13
14 #diagnosis_modes=1
15 #logic_symptoms=1
16 #total_symptoms=1 #total_suspects=1 total_CPU_time=7.14sec
17
18 diagnosis_mode=logic

```

```

19 #symptoms=1 #suspects=1 CPU_time=7.14sec fail_log=final.txt
20 #failing_patterns=42, #passing_patterns=1030
21 #unexplained_failing_patterns=30
22 0      54      77      139      157      186      215      219      355      364
23 443    449    522    600    700    741    768    811    851    857
24 879    897    910    918    932    1013   1044   1048   1058   1071
25
26 symptom=1 #suspects=1 #explained_patterns=12
27 12      90      142      211      216      303      320      427      845      901
28 961      970
29 suspect  score fail_match pass_mismatch type          value  pin_pathname cell_name
   net_pathname
30 -----
31 1          80      12          2          OPEN/DOM          both    /u_soc_student/u_core/u3/Z
   UH12DW040L_USBUX040 /u_soc_student/u_core/n13
32 -----
33
34
35 Internal failing cells profiling
36 Total failing patterns=42, Total external compressed bits=792
37 Failing Pattern 0 has 17 compressed failing bits
38 unexplained failing pattern
39 Failing Pattern 12 has 22 compressed failing bits
40 partial explained failing pattern
41 Failing Pattern 54 has 13 compressed failing bits
42 unexplained failing pattern
43 Failing Pattern 77 has 15 compressed failing bits
44 unexplained failing pattern
45 Failing Pattern 90 has 16 compressed failing bits
46 partial explained failing pattern
47 Failing Pattern 139 has 18 compressed failing bits
48 unexplained failing pattern
49 Failing Pattern 142 has 12 compressed failing bits
50 partial explained failing pattern
51 Failing Pattern 157 has 20 compressed failing bits
52 unexplained failing pattern
53 Failing Pattern 186 has 25 compressed failing bits
54 unexplained failing pattern
55 Failing Pattern 211 has 16 compressed failing bits
56 partial explained failing pattern
57 Failing Pattern 215 has 20 compressed failing bits
58 unexplained failing pattern
59 Failing Pattern 216 has 15 compressed failing bits
60 partial explained failing pattern
61 Failing Pattern 219 has 20 compressed failing bits
62 unexplained failing pattern
63 Failing Pattern 303 has 15 compressed failing bits
64 partial explained failing pattern
65 Failing Pattern 320 has 15 compressed failing bits
66 partial explained failing pattern
67 Failing Pattern 355 has 14 compressed failing bits
68 unexplained failing pattern
69 Failing Pattern 364 has 24 compressed failing bits
70 unexplained failing pattern
71 Failing Pattern 427 has 14 compressed failing bits
72 partial explained failing pattern
73 Failing Pattern 443 has 16 compressed failing bits
74 unexplained failing pattern
75 Failing Pattern 449 has 19 compressed failing bits
76 unexplained failing pattern
77 Failing Pattern 522 has 18 compressed failing bits
78 unexplained failing pattern
79 Failing Pattern 600 has 31 compressed failing bits
80 unexplained failing pattern
81 Failing Pattern 700 has 18 compressed failing bits
82 unexplained failing pattern
83 Failing Pattern 741 has 36 compressed failing bits
84 unexplained failing pattern
85 Failing Pattern 768 has 26 compressed failing bits
86 unexplained failing pattern
87 Failing Pattern 811 has 15 compressed failing bits
88 unexplained failing pattern
89 Failing Pattern 845 has 11 compressed failing bits
90 partial explained failing pattern

```

```

91 Failing Pattern 851 has 26 compressed failing bits
92   unexplained failing pattern
93 Failing Pattern 857 has 13 compressed failing bits
94   unexplained failing pattern
95 Failing Pattern 879 has 14 compressed failing bits
96   unexplained failing pattern
97 Failing Pattern 897 has 12 compressed failing bits
98   unexplained failing pattern
99 Failing Pattern 901 has 10 compressed failing bits
100  partial explained failing pattern
101 Failing Pattern 910 has 29 compressed failing bits
102   unexplained failing pattern
103 Failing Pattern 918 has 6 compressed failing bits
104   unexplained failing pattern
105 Failing Pattern 932 has 18 compressed failing bits
106   unexplained failing pattern
107 Failing Pattern 961 has 12 compressed failing bits
108   partial explained failing pattern
109 Failing Pattern 970 has 16 compressed failing bits
110   partial explained failing pattern
111 Failing Pattern 1013 has 15 compressed failing bits
112   unexplained failing pattern
113 Failing Pattern 1044 has 35 compressed failing bits
114   unexplained failing pattern
115 Failing Pattern 1048 has 43 compressed failing bits
116   unexplained failing pattern
117 Failing Pattern 1058 has 29 compressed failing bits
118   unexplained failing pattern
119 Failing Pattern 1071 has 13 compressed failing bits
120   unexplained failing pattern
121 // command: exit

```

Listing A.12: diag.log

```

1 system echo Starting edt_atpg
2 system date
3 //
4 //Setup Design name Variable
5 //
6 set TOPMODULE soc_student_dft
7 set OUTPUTFILE $TOPMODULE
8
9 //
10 //Tessent Shell context and loading design, libs and netlist
11 //
12 set_context patterns -scan
13 read_cell_library dft_mentor.lib
14 read_verilog ../../netlists/soc_student_edt_30_backup.v
15
16 set file_list [lsort -unique [get_attribute_value [get_modules -of_type design] -name
17   file_path_name]]
17 foreach i $file_list {puts $i}
18
19 set_current_design soc_student_edt
20
21 //
22 // Invoke user-defined setup dofile
23 //
24 dofile setup_user.dofile
25
26 //
27 // Invoke design setup, generated by stil2mgc
28 //
29 dofile design_setup_edt.dofile
30
31 //
32 // Check DFT design rules
33 //
34 report_black_boxes
35 add_black_boxes -auto
36
37 set_system_mode analysis
38 //

```

```

39 // Report scan chains, scan cells, clocks, pin constraints,
40 // write/read controls and DRC summary
41 //
42
43 report_clocks
44 report_input_constraints
45 report_write_control
46 report_read_control
47 report_drc_rules -verbose
48
49 if { $EXECUTE_STUCKAT == "ON" } {
50     report_scan_chain > reports/scan_chains_30_stuckat.rpt
51     report_scan_cells > reports/scan_cells_30_stuckat.rpt
52
53     report_drc_rules C3 > reports/drc/drc_stuckat/
54         C3_clock_may_capture_data_affected_by_its_captured_data.rpt
55     report_drc_rules C4 > reports/drc/drc_stuckat/
56         C4_clock_may_be_affected_by_its_captured_data.rpt
57     report_drc_rules C5 > reports/drc/drc_stuckat/
58         C5_clock_is_connected_to_multiple_ports_of_same_latch.rpt
59     report_drc_rules C7 > reports/drc/drc_stuckat/C7_scan_cell_capture_ability_check.rpt
60     report_drc_rules D5 > reports/drc/drc_stuckat/D5_non_scan_memory_element.rpt
61     report_drc_rules D6 > reports/drc/drc_stuckat/D6_non_transparent_non_scan_latches.rpt
62     report_drc_rules D7 > reports/drc/drc_stuckat/D7_stable_high_edge_triggered_clock_ports.rpt
63     report_drc_rules E5 > reports/drc/drc_stuckat/E5_X_state_propagation.rpt
64     report_drc_rules C6 > reports/drc/drc_stuckat/C6.rpt
65 }
66
67 if { $EXECUTE_TRANSITION == "ON" } {
68     report_scan_chain > reports/scan_chains_30_transition.rpt
69     report_scan_cells > reports/scan_cells_30_transition.rpt
70
71     report_drc_rules C3 > reports/drc/drc_transition/
72         C3_clock_may_capture_data_affected_by_its_captured_data.rpt
73     report_drc_rules C4 > reports/drc/drc_transition/
74         C4_clock_may_be_affected_by_its_captured_data.rpt
75     report_drc_rules C5 > reports/drc/drc_transition/
76         C5_clock_is_connected_to_multiple_ports_of_same_latch.rpt
77     report_drc_rules C7 > reports/drc/drc_transition/C7_scan_cell_capture_ability_check.rpt
78     report_drc_rules D5 > reports/drc/drc_transition/D5_non_scan_memory_element.rpt
79     report_drc_rules D6 > reports/drc/drc_transition/D6_non_transparent_non_scan_latches.rpt
80     report_drc_rules D7 > reports/drc/drc_transition/
81         D7_stable_high_edge_triggered_clock_ports.rpt
82     report_drc_rules E5 > reports/drc/drc_transition/E5_X_state_propagation.rpt
83     report_drc_rules C6 > reports/drc/drc_transition/C6.rpt
84 }
85
86 //
87 // Invoke user-defined atpg dofiles based on selected faults
88 // (stuck-at / transition)
89 //
90 if { $EXECUTE_STUCKAT == "ON" } {
91     dofile stuckat_user.dofile
92 }
93
94 if { $EXECUTE_TRANSITION == "ON" } {
95     dofile transition_user.dofile
96 }
97
98 //
99 system date
100 report_resources
101 //exit

```

Listing A.13: edt.atpg.dofile

```

1 set_context patterns -scan_diagnosis
2 read_flat_model results/soc_student_dft_30_stuckat_all.flat.gz
3 set_diagnosis -expected_value off -verify_pattern off -BRIDGE_ANALYSIS OFF -max_suspect 50
4   -internal_failing_cells on
5 set_contention_check off
6
7 read_patterns results/soc_student_dft_30_stuckat_results.bin
8

```



```
8 diagnose_failures final.txt
```

Listing A.14: edt_diag.dofile

```
1 #!/bin/tcsh
2
3 setenv TESSENT_EXECUTE_STUCKAT      1
4 setenv TESSENT_EXECUTE_TRANSITION  0
5
6
7 ## RUN CMD:
8 ifxlsf -eh_dispatch LS_INTERACTIVE_EXEC -eh_queue batch \
9 "tessent -shell -dofile ./edt_atpg.dofile -logfile logs/RUN_tessent_30_Stuckat.log -replace"
10
11 ## for just entering the environment (to simulate patterns / diagnosis)
12 ## ifxlsf -eh_dispatch LS_INTERACTIVE_EXEC -eh_queue batch "tessent -shell"
```

Listing A.15: run_tessent_stuckat

```
1 #!/bin/tcsh
2
3 setenv TESSENT_EXECUTE_STUCKAT      0
4 setenv TESSENT_EXECUTE_TRANSITION  1
5
6
7 ## RUN CMD:
8 ifxlsf -eh_dispatch LS_INTERACTIVE_EXEC -eh_queue batch \
9 "tessent -shell -dofile ./edt_atpg.dofile -logfile logs/RUN_tessent_30_Transition.log -
  replace"
```

Listing A.16: run_tessent_transition

```
1 #!/bin/bash
2
3 rm -rf final.txt
4
5 pcregrep -M "Expected response.+\n.+\n.+\n" irun.log > final.txt
6
7 ## then search and replace
8 ## "^.+pattern\s+(\d+).+\n.+\n.+shift cycle:\s+(\d+)\n"
9 ## with the pattern \1 edt_channels_out \2\n :)
```

Listing A.17: script_regex_patterns.sh

```
1 set ADVANCED_POWER_REPORT "OFF"
2 set HIGH_EFFORT_ATPG "OFF"
3 set EXECUTE_STUCKAT "OFF"
4 set EXECUTE_IDDQ "OFF"
5 set EXECUTE_NDETECT "OFF"
6 set EXECUTE_TRANSITION "OFF"
7 set EXECUTE_PATHDELAY "OFF"
8 set EXECUTE_FSIM "OFF"
9
10 if {[info exists env(TESSENT_EXECUTE_STUCKAT)] && $env(TESSENT_EXECUTE_STUCKAT) == 1} {
11     set EXECUTE_STUCKAT "ON"
12 }
13
14 if {[info exists env(TESSENT_EXECUTE_TRANSITION)] && $env(TESSENT_EXECUTE_TRANSITION) == 1} {
15     set EXECUTE_TRANSITION "ON"
16 }
17
18 //
```

```

19 //
20 // Don't consider internal faults of the cells
21 //
22 set_internal_fault off
23
24 //
25 // Creates pattern without bus contention
26 //
27 set_contention_check on -atpg
28
29 //
30 // Do not identify shadow elements to enable parallel resimulation
31 //
32 set_shadow_check off
33
34 //
35 // ZHOLD states are not allowed to retain their previous values
36 // Do not change, otherwise mismatches can occur
37 //
38 set_zhold_behavior off
39
40 //
41 // Enable the calculation of switching activity for shift and capture
42 //
43 set_power_metrics -shift on -capture on -composite_shift_power on

```

Listing A.18: setup_user.dofile

```

1 //
2 // Set logfile handling
3 //
4 set_logfile_handling logs/edt_atpg_30_stuckat.log -replace
5 report_version_data
6 system date
7
8 //
9 // Generate the fault list
10 //
11 add_faults -all
12
13 //
14 // Save every day (60 minutes * 12) the last pattern in binary format
15 //
16 set_checkpoint_options on -pattern_fi checkpoint_stuckat.bin -pattern_format binary -replace
   -period 720
17
18 //-----
19
20 add_input_constraints -slow_pad -all
21
22 //
23 // Set the procedures defined in testproc
24 //
25
26 //-----
27
28 // BASIC
29
30 set_pattern_type -multiple_load off -ram_sequential off -sequential 1
31
32 set_capture_proc off -all
33 set_capture_proc on capt_scan_reset_n
34 create_patterns -no_auto
35 report_statistics
36
37 set_capture_proc off -all
38 set_capture_proc on capt_basic_clk
39 reset_au_faults
40 create_pattern -no_auto
41 report_statistics
42
43 //-----
44

```

```

45 // SEQUENTIAL
46
47 set_pattern_type -sequential 2
48 set_capture_proc off -all
49 set_capture_proc on capt_seq_2clk
50 reset_au_faults
51 create_patterns -no_auto
52 report_statistics
53
54 set_pattern_type -sequential 3
55 set_capture_proc off -all
56 set_capture_proc on capt_seq_3clk
57 create_patterns -no_auto
58 report_statistics
59
60 set_pattern_type -sequential 4
61 set_capture_proc off -all
62 set_capture_proc on capt_seq_4clk
63 create_patterns -no_auto
64 report_statistics
65
66 //-----
67
68 // MULTIPLE LOAD
69
70 set_pattern_type -sequential 4 -multiple_load on
71
72 set_capture_proc off -all
73 set_capture_proc on capt_mul_2pat_1clk
74 reset_au_faults
75 create_patterns -no_auto
76 report_statistics
77
78 set_pattern_type -sequential 4 -multiple_load on
79
80 set_capture_proc off -all
81 set_capture_proc on capt_mul_3pat_1clk
82 reset_au_faults
83 create_patterns -no_auto
84 report_statistics
85
86 set_pattern_type -sequential 4 -multiple_load on
87
88 set_capture_proc off -all
89 set_capture_proc on capt_mul_4pat_1clk
90 reset_au_faults
91 create_patterns -no_auto
92 report_statistics
93
94 set_pattern_type -sequential 4 -multiple_load on
95
96 set_capture_proc off -all
97 set_capture_proc on capt_mul_2pat_2clk
98 reset_au_faults
99 create_patterns -no_auto
100 report_statistics
101
102 //
103 // Optimize pattern using static pattern compression
104 //
105 set_dofile_abort off
106 identify_redundant_faults
107 //
108 compress_patterns -multiple_detection
109 set_dofile_abort on
110
111 compress_patterns 3
112 set_dofile_abort on
113
114 order_pattern 3
115
116 //
117 // Optional step to reach maximum coverage
118 //
119 if { $HIGH_EFFORT_ATPG == "ON" } {
120     create_patterns -no_terminate_ineffective_atpg -coverage_effort high

```

```

121 }
122
123 //
124 // save flattend model for diagnosis flow
125 //
126 write_flat_model results/${OUTPUTFILE}_30_stuckat_all.flat.gz -all -replace
127
128
129 //
130 // Save reports
131 //
132 report_scan_volume > reports/report_scan_volume_30_stuckat.rpt
133 report_statistics -detailed > reports/report_statistics_summary_30_stuckat.rpt
134
135 report_faults > reports/faults_all_30_stuckat.rpt
136 report_faults -class AU > reports/faults_AU_30_stuckat.rpt
137 report_faults -class UC > reports/faults_UC_30_stuckat.rpt
138 report_faults -class UO > reports/faults_UO_30_stuckat.rpt
139
140 report_statistics -hierarchy -level 4 -format "#f %t %tcl uo au" -CSV -Use_full_instance_path
141 > reports/report_statistic_30_stuckat_hier4.rpt
142 report_statistics -hierarchy -level 5 -format "#f %t %tcl uo au" -CSV -Use_full_instance_path
143 > reports/report_statistic_30_stuckat_hier5.rpt
144 report_statistics -hierarchy -level 6 -format "#f %t %tcl uo au" -CSV -Use_full_instance_path
145 > reports/report_statistic_30_stuckat_hier6.rpt
146
147 //
148 // Save the patterns
149 //
150 write_patterns results/${OUTPUTFILE}_30_stuckat_results.bin -bin -scan -noz -replace
151 write_patterns results/${OUTPUTFILE}_30_stuckat_results.WGL.gz -procfile -WGL -scan -noz
152 -replace -mode_internal
153
154 //
155 // for a more advanced power report (optional)
156 //
157 if { $ADVANCED_POWER_REPORT == "ON" } {
158 //
159 // Report detailed switching activity for all patterns
160 //
161 report_power_metrics -patterns all -sort_capture wsa_peak
162 //
163 // Save min and max power patterns for analysis
164 //
165 // Save the 100 patterns with lowest capture activity
166 //
167 order_pattern -capture_power wsa_peak
168 write_patterns results/${OUTPUTFILE}_min_power_30_stuckat.bin -bin -scan -noz -replace -end
169 99
170 write_patterns results/${OUTPUTFILE}_min_power_30_stuckat.WGL.gz -procfile -WGL -scan
171 -parameter fsatpg_parameterfile -noz -replace -end 99
172 //
173 // Save the 100 patterns with highest capture activity
174 //
175 order_pattern -capture_power wsa_peak -descending
176 write_patterns results/${OUTPUTFILE}_max_power_30_stuckat.bin -bin -scan -noz -replace -end
177 99
178 write_patterns results/${OUTPUTFILE}_max_power_30_stuckat.WGL.gz -procfile -WGL -scan
179 -parameter fsatpg_parameterfile -noz -replace -end 99
180 }
181
182 //
183 // Set logfile handling
184 //
185 system date
186 set_logfile_handling -resume
187 //
188 report_statistics -detailed
189 report_scan_volume

```

Listing A.19: stuckat_user.dofile

```

1 set_logfile_handling logs/edt_atpg_30_transition.log -replace
2 report_version_data
3 system date
4 //
5 // Generate the fault list
6 //
7 add_faults -all
8
9 set_fault_type transition -no_shift_launch
10
11 //
12 // no need to delete those faults, only for transition because
13 // there the reset has action only at the beginning of the simulation
14 // so you have to delete asynchronous-controls, which only refer to
15 // set/reset ports of state elements and RAMs.
16 //
17 delete_faults -ASynchronous_controls
18
19 //
20 // Save every day (60 minutes * 12) the last pattern in binary format
21 //
22 set_checkpoint_options on -pattern_fi checkpoint_transition.bin -pattern_format binary
   -replace -period 720
23
24 //-----
25
26 set_pattern_type -multiple_load off -ram_sequential off -sequential 1
27
28 //
29 // Report clock information
30 //
31 report_clocks
32
33 //-----
34
35 // //
36 // // Define initial capture procedures
37 // //
38
39 // SEQUENTIAL
40
41 set_pattern_type -sequential 2
42 set_capture_proc off -all
43 set_capture_proc on capt_seq_2clk
44 reset_au_faults
45 create_patterns -no_auto
46 report_statistics
47
48 set_pattern_type -sequential 3
49 set_capture_proc off -all
50 set_capture_proc on capt_seq_3clk
51 create_patterns -no_auto
52 report_statistics
53
54 set_pattern_type -sequential 4
55 set_capture_proc off -all
56 set_capture_proc on capt_seq_4clk
57 create_patterns -no_auto
58 report_statistics
59
60 //-----
61
62 // MULTIPLE LOAD
63
64 set_pattern_type -multiple_load on
65
66 set_capture_proc off -all
67 set_capture_proc on capture_mul_1slow_2fast
68 reset_au_faults
69 create_patterns -no_auto
70 report_statistics
71
72 //
73 // Optimize pattern using static pattern compression
74 //
75 set_dofile_abort off

```

```

76 identify_redundant_faults
77 //
78 compress_patterns -multiple_detection
79 set_dofile_abort on
80
81 compress_patterns 3
82 set_dofile_abort on
83
84 order_pattern 3
85
86 //
87 // Optional step to reach maximum coverage
88 //
89 if { $HIGH_EFFORT_ATPG == "ON" } {
90     create_patterns -no_terminate_ineffective_atpg -coverage_effort high
91 }
92 //
93 // save flattend model for diagnosis flow
94 //
95 write_flat_model results/${OUTPUTFILE}_30_transition.flat.gz -all -replace
96 write_patterns results/${OUTPUTFILE}_30_transition.bin -bin -scan -noz -replace
97 write_patterns results/${OUTPUTFILE}_30_transition.WGL.gz -procfile -WGL -scan -noz -replace
98     -mode_internal
99 //
100 // Save reports
101 //
102 report_scan_volume > reports/report_scan_volume_30_transition.rpt
103 report_statistics -detailed > reports/report_statistics_summary_30_transition.rpt
104
105 report_faults > reports/faults_all_30_transition.rpt
106 report_faults -class AU > reports/faults_AU_30_transition.rpt
107 report_faults -class UC > reports/faults_UC_30_transition.rpt
108 report_faults -class UO > reports/faults_UO_30_transition.rpt
109
110 report_statistics -hierarchy -level 4 -format "#f %t %tcl uo au" -CSV -Use_full_instance_path
111     > reports/report_statistic_30_transition_hier4.rpt
112 report_statistics -hierarchy -level 5 -format "#f %t %tcl uo au" -CSV -Use_full_instance_path
113     > reports/report_statistic_30_transition_hier5.rpt
114 report_statistics -hierarchy -level 6 -format "#f %t %tcl uo au" -CSV -Use_full_instance_path
115     > reports/report_statistic_30_transition_hier6.rpt
116
117 //
118 // Save the patterns
119 //
120 write_patterns results/${OUTPUTFILE}_30_transition.bin -bin -scan -noz -replace
121 write_patterns results/${OUTPUTFILE}_30_transition.WGL.gz -procfile -WGL -scan -noz -replace
122     -mode_internal
123
124 //
125 // for a more advanced power report (optional)
126 //
127 if { $ADVANCED_POWER_REPORT == "ON" } {
128     report_power_metrics
129     //
130     // Report detailed switching activity for all patterns
131     //
132     report_power_metrics -patterns all -sort_capture wsa_peak
133     //
134     // Save min and max power patterns for analysis
135     //
136     // Save the 100 patterns with lowest capture activity
137     //
138     order_pattern -capture_power wsa_peak
139     write_patterns results/${OUTPUTFILE}_min_power_30_transition.bin -bin -scan -noz -replace
140     -end 99
141     write_patterns results/${OUTPUTFILE}_min_power_30_transition.WGL.gz -procfile -WGL -scan
142     -parameter fsatpg_parameterfile -noz -replace -end 99
143     //
144     // Save the 100 patterns with highest capture activity
145     //
146     order_pattern -capture_power wsa_peak -descending
147     write_patterns results/${OUTPUTFILE}_max_power_30_transition.bin -bin -scan -noz -replace
148     -end 99

```

```

143   write_patterns results/${OUTPUTFILE}_max_power_30_transition.WGL.gz -procfile -WGL -scan
      -parameter fsatpg_parameterfile -noz -replace -end 99
144 }
145
146 //
147 // Set logfile handling
148 //
149 system date
150 set_logfile_handling -resume
151 //
152 report statistics -detailed
153 report_scan_volume

```

Listing A.20: transition_user.dofile

```

1 add_clock 1 rst_n_i
2 add_clock 0 clk_i
3
4 add_input_constraint scan_enable_i -C0
5 add_input_constraint scanmode_i -C1
6
7 if { $EXECUTE_STUCKAT == "ON" } {
8     add_scan_group group1 fsatpg_stuckat.testproc
9 }
10 if { $EXECUTE_TRANSITION == "ON" } {
11     add_scan_group group1 fsatpg_transition.testproc
12 }
13
14 add_scan_chain chain1 group1 test_si1 test_so1

```

Listing A.21: design_setup.dofile

```

1 system echo Starting fsatpg
2 system date
3 //
4 //Setup Design name Variable
5 //
6 set TOPMODULE soc_student
7 set OUTPUTFILE $TOPMODULE
8
9 //
10 //Tessent Shell context and loading design, libs and netlist
11 //
12 set_context patterns -scan
13 read_cell_library dft_mentor.lib
14 read_verilog ../../rtl2gds/dc/working/soc_student.v
15
16 set file_list [lsort -unique [get_attribute_value [get_modules -of_type design] -name
      file_path_name]]
17 foreach i $file_list {puts $i}
18
19 set_current_design soc_student
20
21 //
22 // Invoke user-defined setup dofile
23 //
24 dofile setup_user.dofile
25
26 //
27 // Invoke design setup, generated by stil2mgc
28 //
29 dofile design_setup.dofile
30
31 //
32 // Check DFT design rules
33 //
34 report_black_boxes
35 add_black_boxes -auto
36

```

```

37 set_system_mode analysis
38
39 //
40 // Report scan chains, scan cells, clocks, pin constraints,
41 // write/read controls and DRC summary
42 //
43 report_clocks
44 report_input_constraints
45 report_write_control
46 report_read_control
47 report_drc_rules -verbose
48
49 if { $EXECUTE_STUCKAT == "ON" } {
50     report_scan_chain > reports/scan_chains_fs_tuckat.rpt
51     report_scan_cells > reports/scan_cells_fs_stuckat.rpt
52
53     report_drc_rules C3 > reports/drc/drc_stuckat/
54     C3_clock_may_capture_data_affected_by_its_captured_data.rpt
55     report_drc_rules C4 > reports/drc/drc_stuckat/
56     C4_clock_may_be_affected_by_its_captured_data.rpt
57     report_drc_rules C5 > reports/drc/drc_stuckat/
58     C5_clock_is_connected_to_multiple_ports_of_same_latch.rpt
59     report_drc_rules C7 > reports/drc/drc_stuckat/C7_scan_cell_capture_ability_check.rpt
60     report_drc_rules D5 > reports/drc/drc_stuckat/D5_non_scan_memory_element.rpt
61     report_drc_rules D6 > reports/drc/drc_stuckat/D6_non_transparent_non_scan_latches.rpt
62     report_drc_rules D7 > reports/drc/drc_stuckat/D7_stable_high_edge_triggered_clock_ports.rpt
63     report_drc_rules E5 > reports/drc/drc_stuckat/E5_X_state_propagation.rpt
64     report_drc_rules C6 > reports/drc/drc_stuckat/C6.rpt
65 }
66
67 if { $EXECUTE_TRANSITION == "ON" } {
68     report_scan_chain > reports/scan_chains_fs_transition.rpt
69     report_scan_cells > reports/scan_cells_fs_transition.rpt
70
71     report_drc_rules C3 > reports/drc/drc_transition/
72     C3_clock_may_capture_data_affected_by_its_captured_data.rpt
73     report_drc_rules C4 > reports/drc/drc_transition/
74     C4_clock_may_be_affected_by_its_captured_data.rpt
75     report_drc_rules C5 > reports/drc/drc_transition/
76     C5_clock_is_connected_to_multiple_ports_of_same_latch.rpt
77     report_drc_rules C7 > reports/drc/drc_transition/C7_scan_cell_capture_ability_check.rpt
78     report_drc_rules D5 > reports/drc/drc_transition/D5_non_scan_memory_element.rpt
79     report_drc_rules D6 > reports/drc/drc_transition/D6_non_transparent_non_scan_latches.rpt
80     report_drc_rules D7 > reports/drc/drc_transition/
81     D7_stable_high_edge_triggered_clock_ports.rpt
82     report_drc_rules E5 > reports/drc/drc_transition/E5_X_state_propagation.rpt
83     report_drc_rules C6 > reports/drc/drc_transition/C6.rpt
84 }
85
86 //
87 // Invoke user-defined atpg dofiles based on selected faults
88 // (stuck-at / transition)
89 //
90 if { $EXECUTE_STUCKAT == "ON" } {
91     dofile stuckat_user.dofile
92 }
93 if { $EXECUTE_TRANSITION == "ON" } {
94     dofile transition_user.dofile
95 }
96
97 //
98 system date
99 report_resources
100 //exit

```

Listing A.22: fsatpg.dofile

```

1 #!/bin/tcsh
2
3 setenv TESSENT_EXECUTE_STUCKAT 1
4 setenv TESSENT_EXECUTE_TRANSITION 0
5
6 ## RUN CMD:
7 cleartool co -unres -nc logs/*

```



```

8 cleartool co -unres -nc results/*
9 cleartool co -unres -nc reports/*
10
11 ifxlsf -eh_dispatch LS_INTERACTIVE_EXEC -eh_queue batch \
12 "tessent -shell -dofile ./fsatpg.dofile -logfile logs/fsatpg_run_stuckat.log -replace"

```

Listing A.23: run_tessent_fastscan_stuckat

```

1 #!/bin/tcsh
2
3 setenv TESSENT_EXECUTE_STUCKAT      0
4 setenv TESSENT_EXECUTE_TRANSITION  1
5
6 ## RUN CMD:
7 cleartool co -unres -nc logs/*
8 cleartool co -unres -nc results/*
9 cleartool co -unres -nc reports/*
10
11 ifxlsf -eh_dispatch LS_INTERACTIVE_EXEC -eh_queue batch \
12 "tessent -shell -dofile ./fsatpg.dofile -logfile logs/fsatpg_run_delay.log -replace"

```

Listing A.24: run_tessent_fastscan_transition

```

1 module wrap ( sysclk_i, rst_n_i, spi_miso_i, uart_txd_i,
2               gpio_input_i, scanmode_i, spi_clk_o, spi_mosi_o, spi_cs_o, uart_rxd_o,
3               gpio_output_o, gpio_output_enable_o, test_si1, test_so1, scan_enable_i,
4               scan_enable_cg_i );
5 input [31:0] gpio_input_i;
6 output [31:0] gpio_output_o;
7 output [31:0] gpio_output_enable_o;
8 input sysclk_i, rst_n_i, spi_miso_i, uart_txd_i, scanmode_i, test_si1,
9        scan_enable_i, scan_enable_cg_i;
10 output spi_clk_o, spi_mosi_o, spi_cs_o, uart_rxd_o, test_so1;
11
12
13
14 soc_student u_soc_student
15 (
16   .clk_i          (sysclk_i          ),
17   .rst_n_i        (rst_n_i          ),
18   .spi_miso_i     (spi_miso_i        ),
19   .uart_txd_i     (uart_txd_i        ),
20   .gpio_input_i   (gpio_input_i      ),
21   .scanmode_i     (scanmode_i        ),
22   .spi_clk_o      (spi_clk_o         ),
23   .spi_mosi_o     (spi_mosi_o        ),
24   .spi_cs_o       (spi_cs_o          ),
25   .uart_rxd_o     (uart_rxd_o        ),
26   .gpio_output_o  (gpio_output_o     ),
27   .gpio_output_enable_o (gpio_output_enable_o ),
28   .test_si1       (test_si1          ),
29   .test_so1       (test_so1          ),
30   .scan_enable_i  (scan_enable_i     ),
31   .scan_enable_cg_i (scan_enable_cg_i )
32 );
33
34 endmodule

```

Listing A.25: wrap.v